


# Crea tus propios juegos para Game Boy

**Autor:** Alberto Rius Poveda

**Fecha:** 23/05/2024

ElegantLaTeX Program

# Índice general

**Capítulo 1 Preparación** **1**

1.1 BGB . . . 1
1.2 GBTD y GBMB . . . 5
1.3 VS Code . . . 5
1.4 RGBDS . . . 6
1.5 Boilerplate . . . 9
1.6 Make . . . 9

**Capítulo 2 Gráficos** **11**

2.1 Primer paso, dibujar un tile . . . 11
2.2 Segundo paso, experimenta . . . 16
2.3 Primer obstáculo, dibuja una N . . . 17
2.4 Aprende nuevas instrucciones . . . 18
2.5 Flags . . . 20
2.6 Tercer paso, usa los bucles . . . 21
2.7 Segundo obstáculo, dibuja un triángulo . . . 24
2.8 Cuarto paso, crea una función . . . 25
2.9 La pila y el registro SP . . . 27
2.10 El registro STAT . . . 28
2.11 Haz tu propio dibujo . . . 32

**Capítulo 3 Diálogos** **33**

3.1 Crear una fuente . . . 33
3.2 Cargar los tiles . . . 36
3.3 Guarda una línea en la ROM . . . 38
3.4 Escribe una línea . . . 39
3.5 Organicemos el código . . . 40
3.6 Escribir más despacio . . . 41
3.7 Escribe múltiples líneas . . . 45
3.8 Control por botones . . . 48
3.9 Otras mejoras . . . 50
3.10 Usando la ventana . . . 51

**Capítulo 4 Minijuego** **53**

4.1 Diseñar personajes . . . 53
4.2 Dibujar el primer sprite . . . 55
4.3 Paletas y sprites más grandes . . . 58
4.4 OAM DMA . . . 60
4.5 Crear estructuras . . . 63
4.6 Moviendo sprites . . . 64

4.7 Desacopla el gestor y los sistemas . . . 65
4.8 Crea entidades . . . 67
4.9 Borra entidades . . . 68
4.10 Tipos de entidades y animaciones . . . 69
4.11 Usando macros y moviendo la pila . . . 75
4.12 Mover entidades con los botones y cambiar animaciones . . . 78
4.13 Enemigos y sistema de borrado de entidades . . . 79
4.14 Sistema de IA . . . 81
4.15 Gestor de partida . . . 81
4.16 Colisiones . . . 84
4.17 Final de la partida . . . 86

**Capítulo 5 Sonido** **88**
5.1 Jugando con el sonido . . . 88

**Capítulo 6 Hagamos un Sokoban** **92**
6.1 Metatiles . . . 92
6.2 Tilemaps . . . 98
6.3 Movimiento en cuadrícula . . . 105
6.4 Reestructurar el bucle principal . . . 106
6.5 Crea las cajas y los botones . . . 108
6.6 Codificación de niveles . . . 109

**Apéndice A Sistemas de numeración** **111**
A.1 Hexadecimal . . . 111
A.2 Binario . . . 111

**Apéndice B RGBASM** **113**
B.1 Símbolos . . . 113
B.2 Secciones . . . 114
B.3 Cadenas constantes . . . 115
B.4 Macros . . . 115

**Apéndice C Instrucciones** **119**
C.1 Instrucciones de carga (load) de 8 bits . . . 119
C.2 Instrucciones de carga (load) de 16 bits . . . 121
C.3 Instrucciones aritméticas de 8 bits . . . 121
C.4 Instrucciones aritméticas de 16 bits . . . 124
C.5 Instrucciones de rotación y desplazamiento . . . 124
C.6 Instrucciones de bit . . . 126
C.7 Instrucciones de salto . . . 127
C.8 Instrucciones de control de la CPU . . . 128

# Capítulo 1 Preparación

El primer paso antes de comenzar nuestro primer viaje en el mundo del desarrollo para Game Boy, es instalar las herramientas necesarias. Empezaremos instalando un emulador de Game Boy.

## 1.1 BGB

El emulador más recomendado para nuestro propósito es BGB, pues tiene incluido un muy buen debugger que nos ayudará a examinar nuestros juegos en ejecución, e interactuar directamente con el código.

El proceso de instalación es muy sencillo:

- Primero, accedemos a la página oficial del emulador: https://bgb.bircd.org
- Después bajamos hasta la sección de descargas y elegimos la última versión. Si tu sistema operativo es Windows, puedes elegir la versión de 64 bits, en caso contrario, descarga la indicada en 1.2.
- Descomprimimos el archivo .zip, y ya tenemos el emulador instalado.

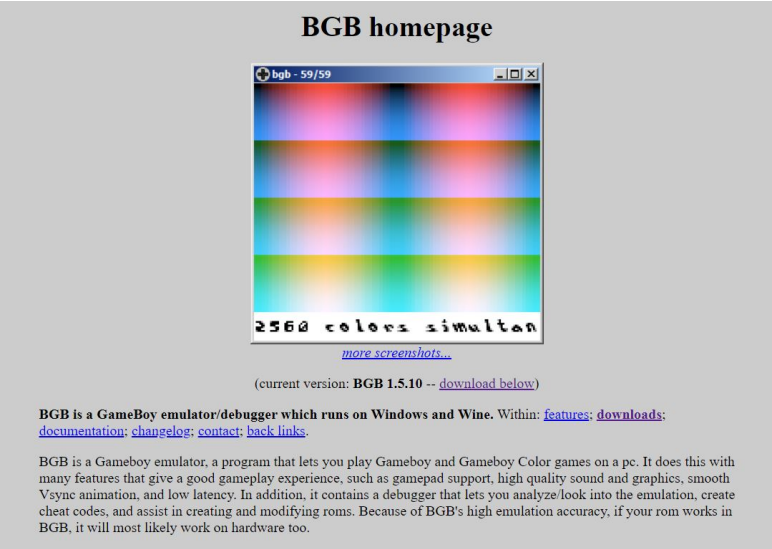
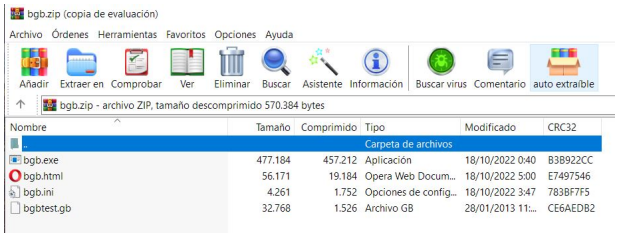



**Figura 1.1:** Página principal de BGB.




**(a)** Enlaces de descarga de BGB.




**(b)** Marin hablando con Link.

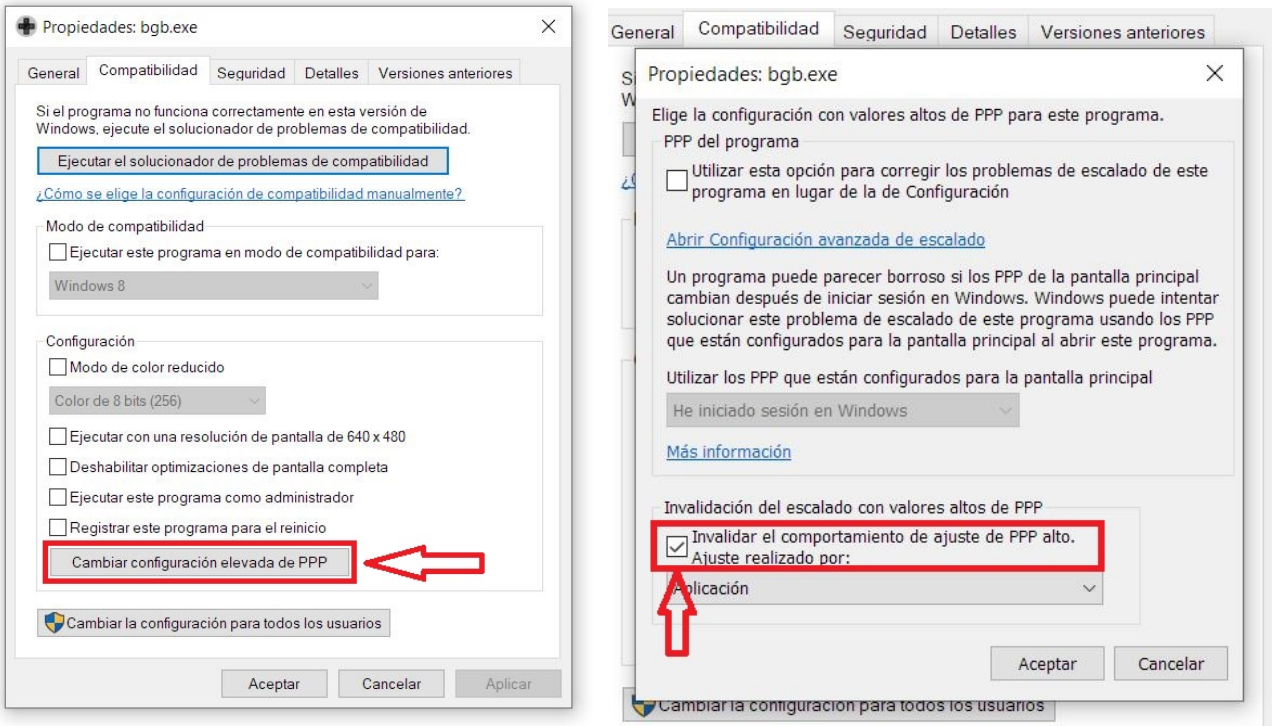
**Figura 1.2:** Instalación de BGB.

En caso de estar utilizando un sistema operativo Linux, o Mac, se deberá ejecutar el programa con Wine. Puedes probar a ejecutar el emulador y comprobar que funciona, la ventana debería mostrar un color verde pálido 1.3.




**Figura 1.3:** Estado inicial de BGB

Al pulsar al tecla *Esc*, se abrirá el debugger. En caso de que la ventana se vea borrosa en Windows, opcionalmente se puede arreglar modificando las propiedades del ejecutable, haciendo clic derecho en el archivo, eligiendo la opción *Propiedades*, y siguiendo los pasos mostrados en 1.4.




**(a)** Propiedades **(b)** Ajustes de PPP.

**Figura 1.4:** Cambiar escalado de BGB.

Para terminar, vamos a configurar la bootrom que el emulador usará cuando ejecute un juego. La bootrom es una sección de código que en una consola real se ejecuta antes de empezar a leer el juego. En el caso de la Game Boy, estos datos ocupan 256 Bytes, y son los encargados de verificar que el cartucho es original y mostrar una animación con el logo de Nintendo al arrancar. Podemos descargar la bootrom del siguiente enlace: https://gbdev.gg8.se/files/roms/bootroms/. El archivo específico es *dmg_boot.bin*, indicado en la figura 1.5.

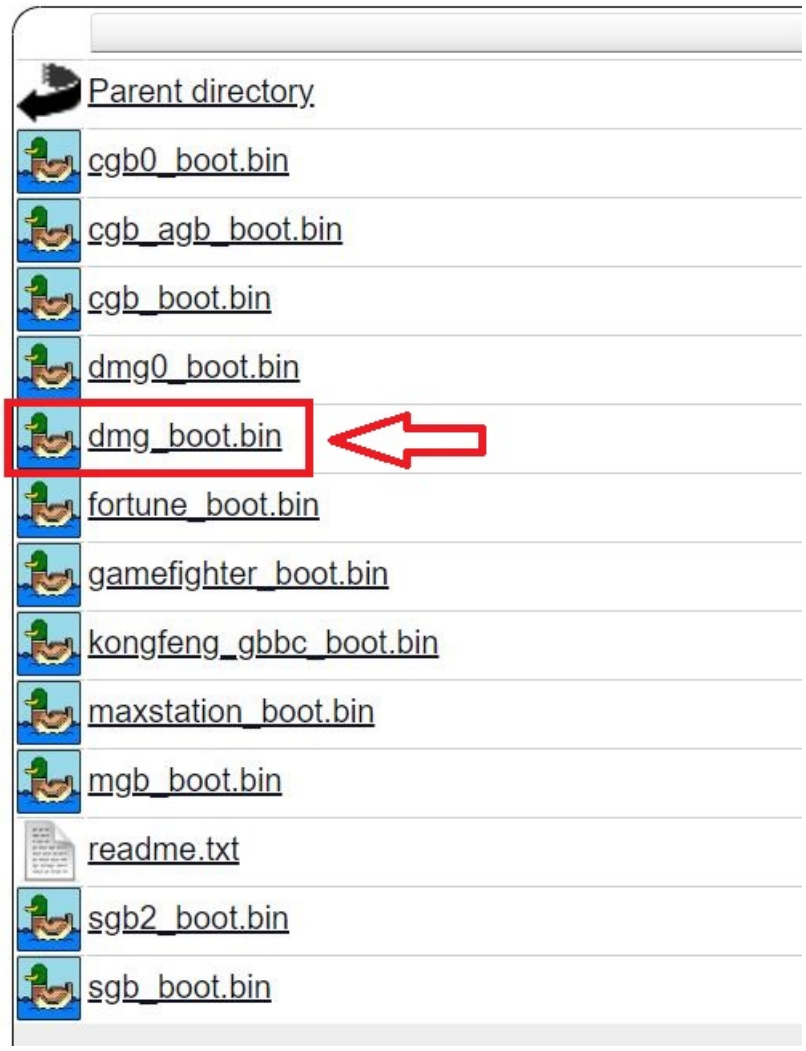



**Figura 1.5:** Archivo bootrom para Game Boy

Para añadirlo a la configuración de BGB, con el emulador abierto, pulsamos F11 para abrir las opciones, seleccionamos la pestaña *System*, y hacemos clic en el botón indicado en la figura 1.6, bajo la opción *DMG bootrom*, elegimos el archivo, y hacemos clic en *Abrir*.

Con esto hemos terminado de configurar el emulador.

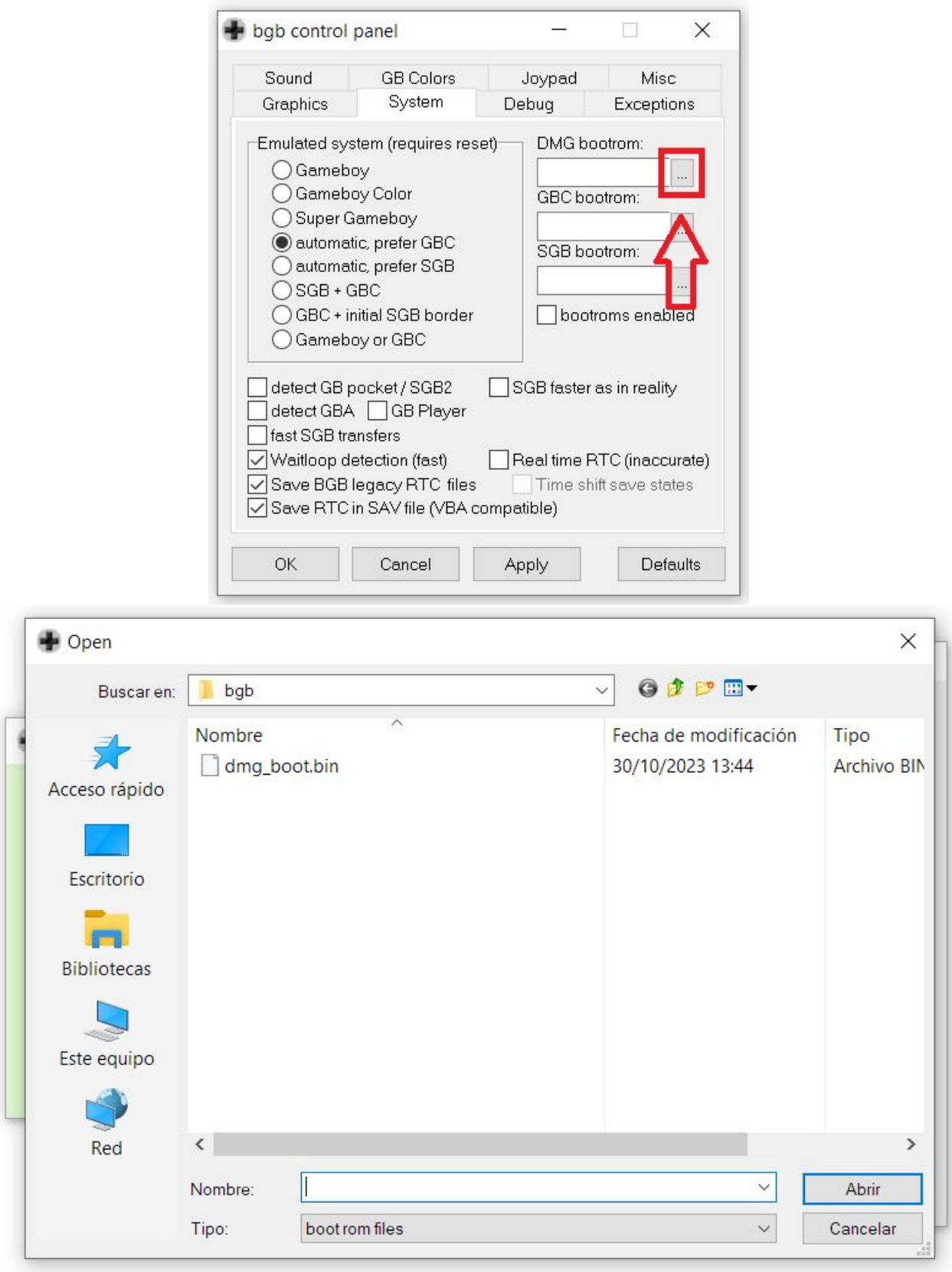



**Figura 1.6:** Configurar bootrom

## 1.2 GBTD y GBMB

Lo siguiente que instalaremos son Gameboy Tile Designer (GBTD) y Gameboy Map Builder (GBMB). Estos programas servirán para crear nuestros propios gráficos y escenarios. GBTD y GBMB pueden ser descargados del siguiente enlace: https://github.com/gbdk-2020/GBTD_GBMB/releases, haciendo clic dónde indica la figura 1.7.

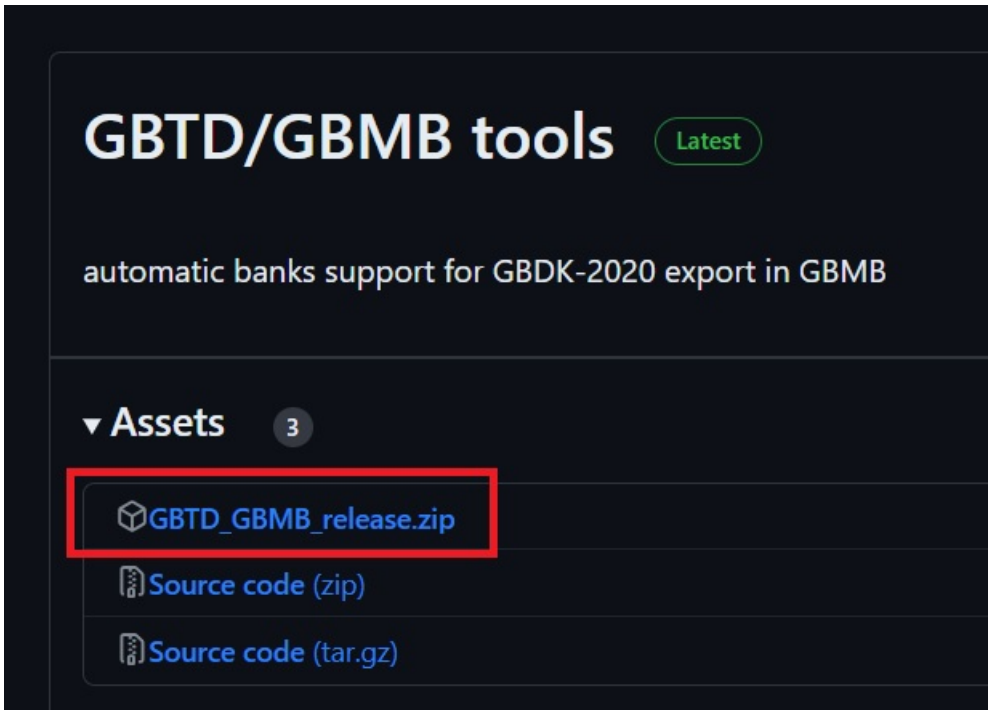



**Figura 1.7:** Enlace de descarga de GBTD y GBMB

Descarga el archivo y descomprímelo, tras esto, su instalación estará terminada. Al igual que con BGB, será necesario usar Wine para ejecutar ambos en Linux o Mac, y en Windows puede ser necesario cambiar las propiedades de la misma forma que en la figura 1.4 para que los programas no se vean borrosos.

## 1.3 VS Code

El siguiente paso será configurar nuestro entorno de desarrollo, que será Visual Studio Code (VS Code). Descarga la versión adecuada para tu sistema operativo de https://code.visualstudio.com/download y sigue las instrucciones de instalación. Tras la instalación, abriremos VS Code e instalaremos las extensiones *RGBDS Z80* y *Z80 Instruction Set*. Para instalarlas, hacemos clic en la pestaña de extensiones en la barra izquierda, usamos la barra de búsqueda para encontrarlas, y hacemos clic en instalar. La figura 1.8 muestra el proceso.

Ahora que tenemos VS Code instalado, lo siguiente que vamos a hacer es descargar y configurar RGBDS.

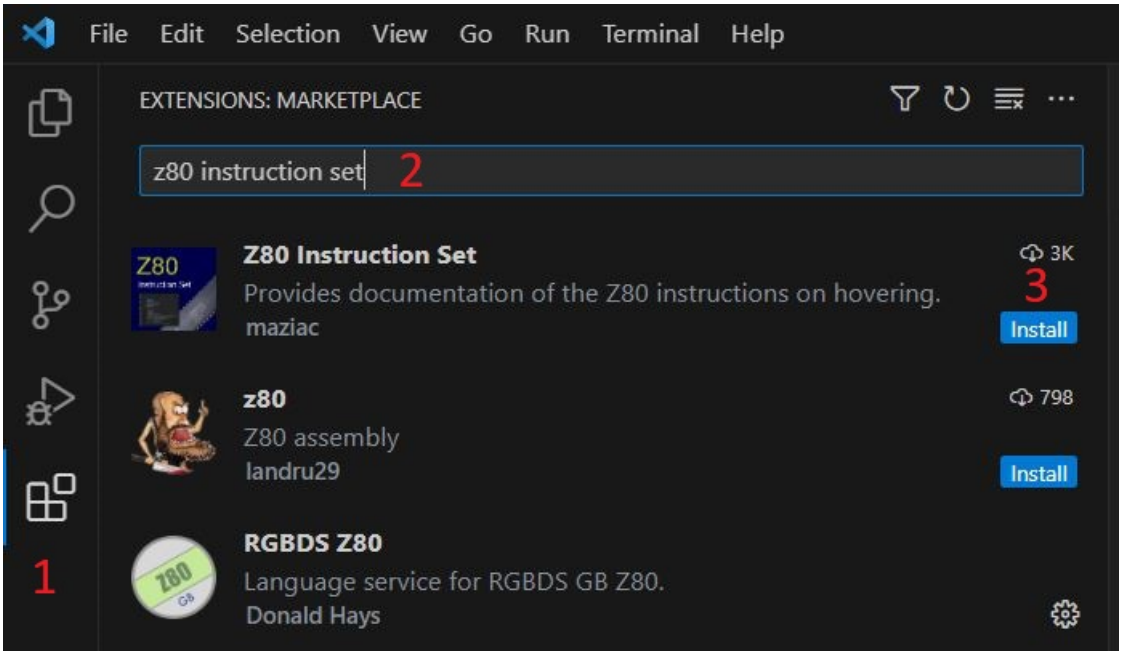



**Figura 1.8:** Extensiones de VS Code usadas

## 1.4 RGBDS

RGBDS es un conjunto de ensamblador y linker para Game Boy y Game Boy Color. Este programa convertirá nuestro código en un programa que una Game Boy pueda leer y ejecutar. El modo de instalación dependerá del sistema operativo que queramos utilizar.

Para instalarlo, accede al repositorio oficial en https://github.com/gbdev/rgbds/releases y descarga el paquete adecuado a tu sistema operativo. Para Windows, sirve cualquiera de las dos versiones siempre que tu sistema operativo soporte arquitectura de 64 bits. En caso de duda, descarga la versión de 32 bits, indicado al final del nombre del archivo.

- Linux: `rgbds-version.tar.gz`
- MacOS: `rgbds-version-macos-x86-64.zip`
- Windows: `rgbds-version-winXX.zip`

Tras la descarga, descomprime el archivo para obtener los ejecutables de `rgbasm`, `rgbfix`, `rgbgfx` y `rgblink`. Para terminar la instalación añadiremos la carpeta dónde se encuentran los ejecutables a la variable de entorno PATH. La variable PATH contiene una serie de rutas a carpetas en las que el sistema operativo busca un programa cuando recibe la orden de abrirlo desde la línea de comandos.

### 1.4.1 Ubuntu Linux

Para añadir el directorio de instalación de RGBDS en PATH en Ubuntu, editamos el archivo `~/.bashrc` con un editor de texto

```
nano ~/.bashrc
```

y añadimos la siguiente línea al final:

```
export PATH="directorio:$PATH"
```

Dónde `directorio` es la ruta a la carpeta de isntalación de RGBDS.

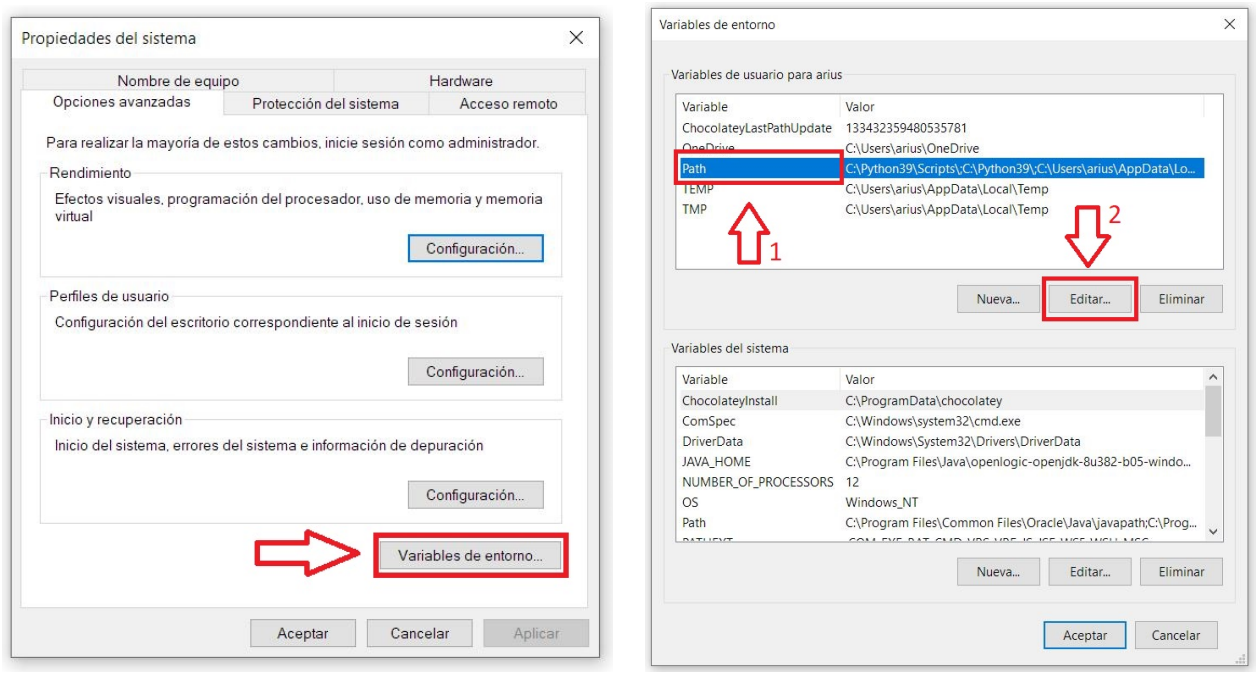
### 1.4.2 MacOS

Para añadir una ruta a PATH en MacOS, crea un nuevo archivo en en la carpeta `/etc/paths.d` y escribe en él la ruta.

```
sudo nano /etc/paths.d/rgbds
```

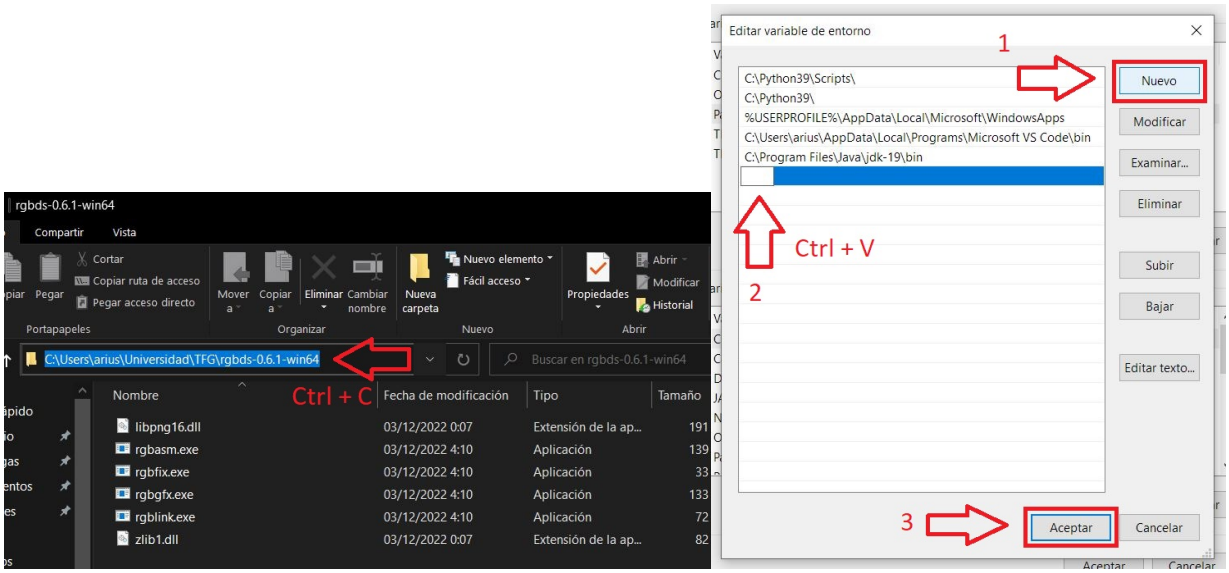
### 1.4.3 Windows

Para añadir una ruta a PATH en Windows, usa el buscador de Windows y escribe *variables de entorno*, debería aparecer una opción llamada *Editar las variables de entorno del sistema*. Selecciónala y en la ventana que habrá abierto, haz clic en la pestaña de opciones avanzadas y luego en *variables de entorno* 1.9a. Tras esto haz clic en Path, Editar, Nuevo, y entonces añade la ruta a la carpeta de instalación. Para escribir la ruta, puedes ir a la carpeta de instalación, hacer clic en la barra superior, copiar la ruta, y pegarla en la ventana de la variable. Las figuras 1.9 y 1.10 ilustran los pasos a seguir.




**(a)** Propiedades. **(b)** Seleccionar la variable.

**Figura 1.9:** Acceder a la variable *Path*.




**(a)** Carpeta de instalación de RGBDS. **(b)** Pegar ruta de instalación.

**Figura 1.10:** Modificar la variable *Path*.

Por último, haz clic en aceptar hasta cerrar todas las ventanas.

## 1.5 Boilerplate

Para terminar la configuración, descargaremos un proyecto base. De ahora en adelante empezaremos nuestros proyectos haciendo copias de este.

Descarga el proyecto base de https://github.com/ISSOtm/gb-boilerplate/tree/master y el archivo *hardware.inc* de https://github.com/gbdev/hardware.inc. Para descargar el proyecto, hacemos clic en el botón <>*Code* y después en *Download ZIP*. Tras descargarlo, descomprimimos el ZIP, borramos la carpeta que hay en *include*, y en su lugar guardamos el archivo *hardware.inc*. Ahora borramos el archivo *build_date.asm* de *src/res/* y hacemos las siguientes modificaciones al archivo *Makefile*:

- Añade

```
$(wildcard src/**/*.asm)
```

  al final de la línea 44. El resultado debería ser

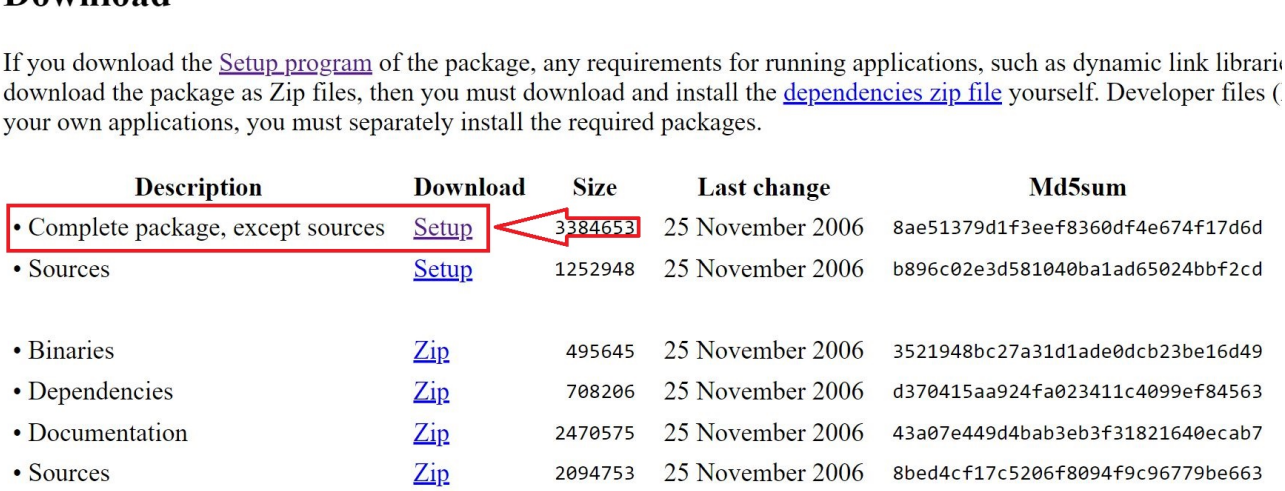
```
SRCS = $(wildcard src/*.asm) $(wildcard src/**/*.asm)
```

- En la línea 85 borra

```
$(OBJDIR)/build_date.o
```

  y borra la línea 84 entera.

Por último, si usas Windows, en las carpetas *obj* y *dep* deberías crear una subcarpeta llamada *res*. En general, por cada subcarpeta del directorio *src* debería haber una igual en los otros dos. Esto sólo es un problema en Windows, en Linux y Mac estas carpetas se generan automáticamente según van haciendo falta.



**Download**

If you download the Setup program of the package, any requirements for running applications, such as dynamic link librari
download the package as Zip files, then you must download and install the dependencies zip file yourself. Developer files (
your own applications, you must separately install the required packages.

| Description | Download | Size | Last change | Md5sum |
|---|---|---|---|---|
| • Complete package, except sources | Setup | 3384653 | 25 November 2006 | 8ae51379d1f3eef8360df4e674f17d6d |
| • Sources | Setup | 1252948 | 25 November 2006 | b896c02e3d581040ba1ad65024bbf2cd |
| • Binaries | Zip | 495645 | 25 November 2006 | 3521948bc27a31d1ade0dcb23be16d49 |
| • Dependencies | Zip | 708206 | 25 November 2006 | d370415aa924fa023411c4099ef84563 |
| • Documentation | Zip | 2470575 | 25 November 2006 | 43a07e449d4bab3eb3f31821640ecab7 |
| • Sources | Zip | 2094753 | 25 November 2006 | 8bed4cf17c5206f8094f9c96779be663 |

**Figura 1.11:** Página de descarga de make para Windows.

## 1.6 Make

Para poder ensamblar el proyecto, deberás instalar *make*. Si estás usando Linux o MacOS, probablemente ya estará instalado en tu máquina. Si estás usando Windows, accede a https://gnuwin32.sourceforge.net/packages/make.htm y descarga el paquete completo, señalado en 1.11, y ejecuta el instalador.

Añade la ruta de instalación al PATH igual que con rgbds. La ruta por defecto debería ser `C:\Program Files (x86)\Gr`

Con esto podemos probar el proyecto base que hemos descargado. Navega hasta la carpeta del proyecto y abre una terminal, entonces ejecuta `make`, y debería crearse una carpeta llamada `bin` en el directorio raíz del proyecto con tres archivos en ella, entre ellos `boilerplate.gb`. Abre BGB y haz clic derecho en la ventana, y clic en *Load ROM*. Navega hasta el archivo y ábrelo. Si lo has hecho todo correctamente, la pantalla de BGB debería mostrar el logo de Nintendo, como se ve en 1.12.

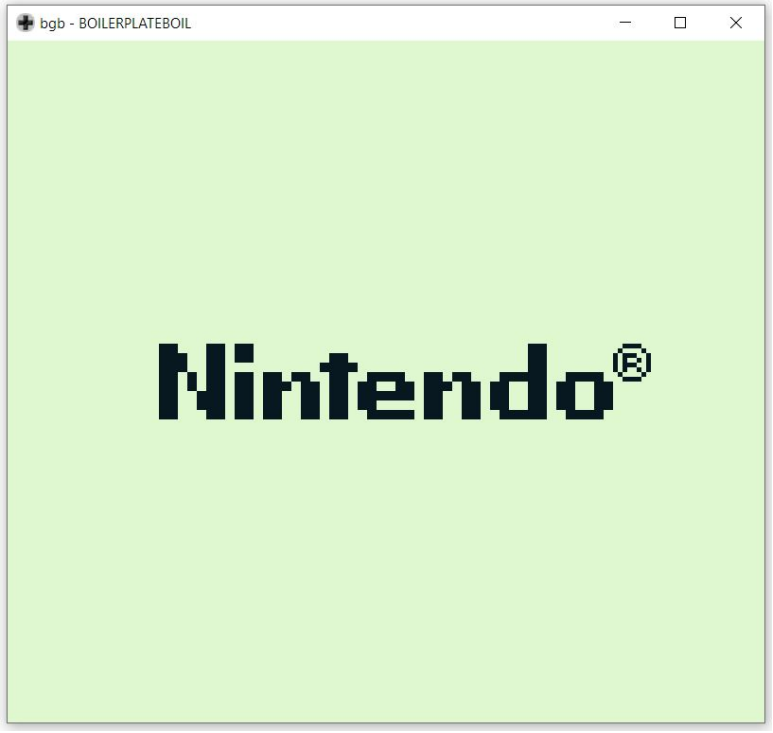
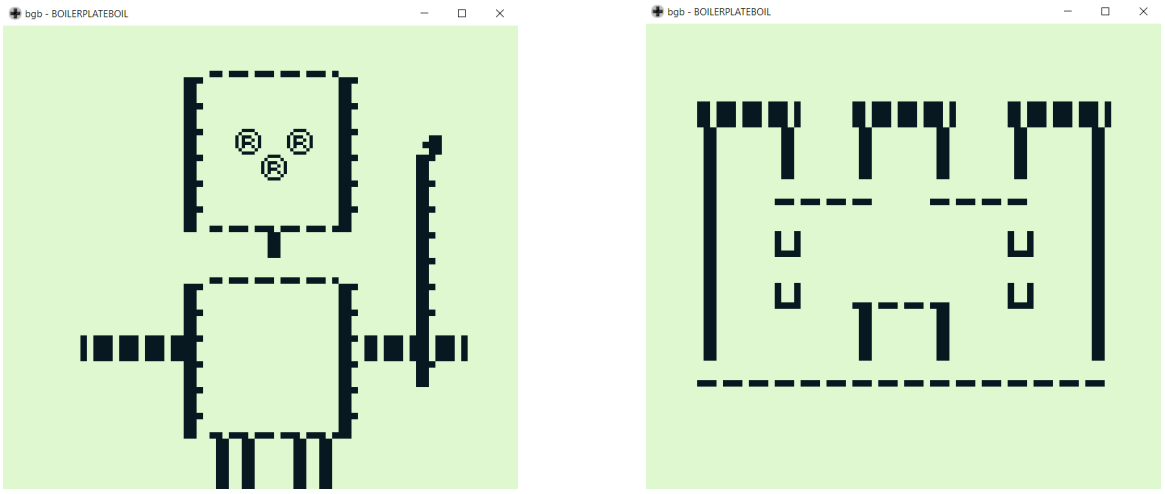



**Figura 1.12:** Emulador con proyecto base.

A partir de ahora, antes de empezar un nuevo proyecto, crea una copia del proyecto base que descargaste durante la instalación y cámbiale el nombre. Entonces abre VS Code y haz clic en *Archivo* y después en *Abrir carpeta*, navega hasta la ubicación de la carpeta que has creado, y haz clic en *Seleccionar carpeta*. También puedes cambiar el nombre que tendrá el archivo del juego desde *project.mk*, cambiando la parte de la derecha en la asignación a *ROMNAME*. Con esto ya podrás empezar a programar.

# Capítulo 2 Gráficos

Es hora de comenzar tu viaje en el mundo del desarrollo para Game Boy, nuestro destino es terminar un juego, pero para llegar a esta meta tendrás que pasar por varias etapas que te ayudarán a cumplir este objetivo. La primera de ellas será aprender a dibujar en la pantalla. Al final de esta etapa serás capaz de crear dibujos como los de las imágenes 2.1a y 2.1b. Por el camino, aprenderás las bases del lenguaje de programación que usaremos para desarrollar nuestro juego.




**(a)** Dibujo de una persona. **(b)** Dibujo de un castillo.

**Figura 2.1:** Dibujos de ejemplo

## 2.1 Primer paso, dibujar un tile

Como primer paso para conseguir llegar a nuestra meta, dibujaremos un tile en la pantalla. Un tile es una agrupación de píxeles en forma de cuadrado con dimensiones de 8x8 píxeles. En este caso empezaremos dibujando en la esquina superior izquierda la R de marca registrada que hay al final del logo de Nintendo.

Crea un nuevo proyecto y abre el archivo *header.asm*, en la carpeta *src*. Escribe el siguiente código entre `EntryPoint:` y `jr @`, en las líneas 19 y 21 respectivamente:

```
ld a, $19
ld [$9800], a
```

El resultado debería ser el mostrado en la figura 2.2

```
SECTION "Header", ROM0[$100]

    ; This is your ROM's entry point
    ; You have 4 bytes of code to do... something
    di
    jp EntryPoint

    ; Make sure to allocate some space for the header, so no important
    ; code gets put there and later overwritten by RGBFIX.
    ; RGBFIX is designed to operate over a zero-filled header, so make
    ; sure to put zeros regardless of the padding value. (This feature
    ; was introduced in RGBDS 0.4.0, but the -MG etc flags were also
    ; introduced in that version.)
    ds $150 - @, 0

SECTION "Entry point", ROM0

EntryPoint:
    ; Here is where the fun begins, happy coding :)
    ld a, $19
    ld [$9800], a
    jr @

```

**Figura 2.2:** *header.asm* inicial.

El símbolo *$* que hay delante de algunos números tanto en el resto de este libro como en el código indica que ese valor está escrito en el sistema hexadecimal. Este símbolo y algunos otros que son necesarios en el código para diferenciar el formato numérico se usarán en lugar de los habituales a lo largo de este libro para facilitar el traslado de números y conceptos desde estas páginas al código que escribas. Estos símbolos vienen dados por RGBASM, el ensamblador que usaremos a lo largo de todo el libro, y puedes consultar documentación más en detalle en el anexo B. El sistema hexadecimal es un sistema de numeración posicional de base 16, en contraste con el decimal, que es de base 10. El sistema decimal es el sistema de numeración que usamos habitualmente. Este sistema consta de 10 símbolos distintos, los números del 0 al 9, mientras que el hexadecimal usa las letras de la A a la F para representar un total de 16 símbolos. Puedes consultar el anexo A para ver una explicación más detallada de los sistemas hexadecimal y binario.

No trates de traducir los números hexadecimales que usaremos a base decimal, sólo debes conocer que a 9 le sigue A, y que a F le sigue 10. Trata de no referirte a los números hexadecimales por cómo se leerían en decimal, pues puede llevar a confusión, al no coincidir sus valores. Es mejor nombrarlos cifra a cifra. Es importante que comprendas esta base, pues todas las direcciones de memoria que usaremos estarán escritas en hexadecimal.

Ahora haz clic en la opción *Terminal* de la barra superior, y después en *Nueva terminal*. En la terminal que se ha abierto en la zona inferior, escribe `make` y pulsa *Enter*. Ahora abre el emulador, haz clic derecho sobre la ventana principal y elige *Load ROM*. Navega hasta el la carpeta *bin* del proyecto y elige el archivo con extensión `.gb`.

Al abrirse, deberías ver cómo en la ventana del emulador aparece el logo de Nintendo, y una R en la esquina superior izquierda, como en la imagen 2.3.

Ahora pulsa *Esc* para abrir la ventana del debugger y poner en pausa la ejecución, entonces haz clic derecho en la ventana principal del emulador y vuelve a cargar la ROM de la misma forma que antes.




**Figura 2.3:** Dibujada una R en la esquina superior izquierda

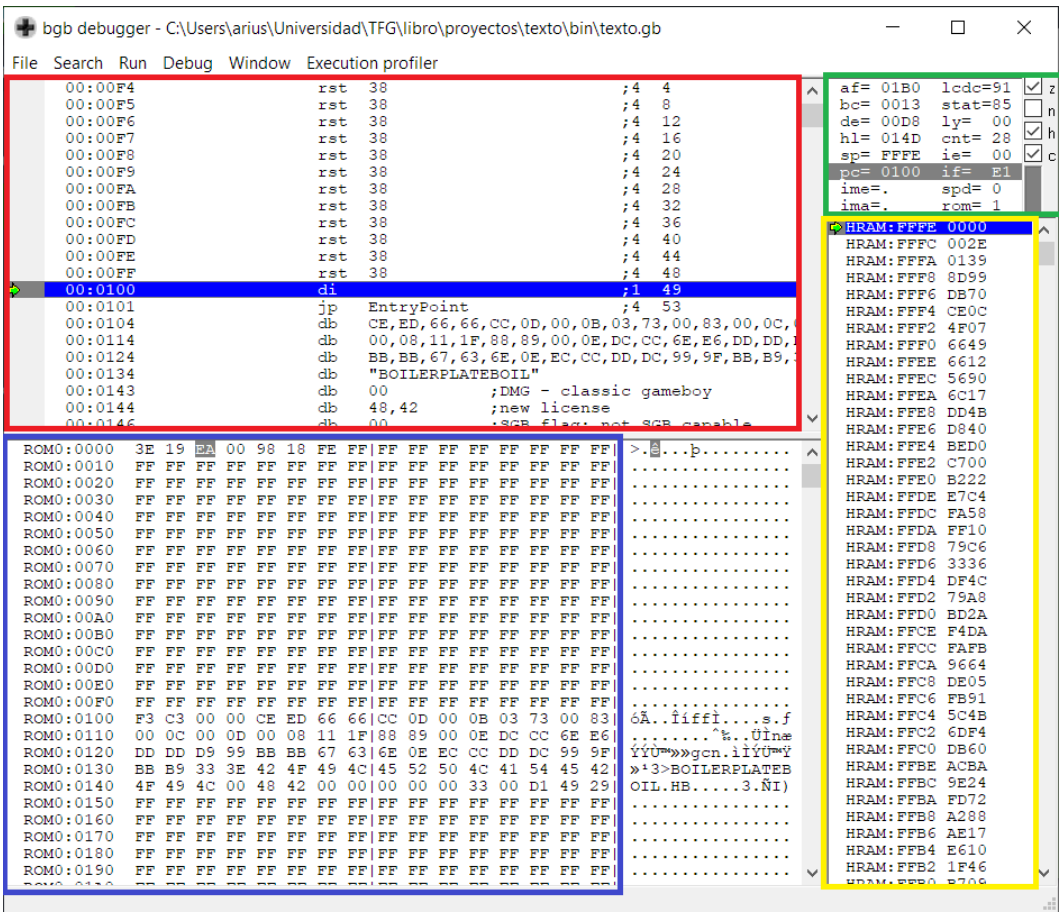



**Figura 2.4:** Debugger de BGB

Examinemos ahora la ventana del debugger. El debugger es una herramienta que nos permite ver el estado de la CPU emulada en todo momento. Esto refleja el estado que tendría una Game Boy real en las mismas situaciones, con la diferencia de que con el emulador podemos pausar la ejecución, y hacer que el programa se ejecute línea a línea pulsando una tecla.

Pasemos a analizar la ventana del debugger, la cual tiene 4 secciones, encuadradas en la figura 2.4:

- En la esquina inferior izquierda se encuentra la memoria. Aquí puede verse toda la información contenida entre el cartucho de juego, y la memoria RAM de la consola. Esto incluye nuestro código y los datos que hayamos guardado. Más adelante veremos dónde se encuentra nuestro código en esta sección. La Game Boy tiene un total de $2^{16}$ direcciones de memoria, por lo que se necesitan 16 bits de datos para poder representar cualquier dirección.
- En la esquina superior izquierda se encuentra el código. El emulador lee los datos de la memoria y los interpreta como si fueran código. Esta interpretación es la que aparece en la ventana superior. Más adelante veremos como aparece nuestro código (casi) tal cual lo hemos escrito en esta sección.
- En la esquina superior derecha están los registros de la CPU, y algunos otros de la consola. Un registro es una unidad de memoria de poca capacidad, generalmente integrados en la CPU, y de muy rápida lectura y escritura. La Game Boy tiene una combinación de registros de 8 y 16 bits. Más adelante explicaremos en detalle cada registro y su uso. Por ahora los únicos que nos interesan son los registros A (acumulador) que puede almacenar 8 bits de datos, y PC (program counter), que tiene 16 bits de espacio que representan una dirección de memoria.
- En la esquina inferior derecha se encuentra la pila. La función de la pila está fuera del alcance de este apartado, pero no tardaremos en hablar de ella.

El último elemento que nos queda por explicar es la línea resaltada de la sección de código, en la esquina superior izquierda. Esta línea indica cuál es la siguiente instrucción que ejecutará la CPU. En la imagen 2.4 podemos ver que la instrucción de esta línea es `di`, la primera línea de código que teníamos en nuestro programa, como en la figura 2.2. En este estado, podemos pulsar la tecla *F7* para que el emulador ejecute una instrucción. Prueba a pulsar *F7* y verás como la línea resaltada avanza una fila. Si vuelves a pulsar la tecla, se resaltará la primera línea de código que hemos escrito. Aquí es cuando nuestro código empezará a ejecutarse.

Vamos a analizar nuestro código y lo que hace.

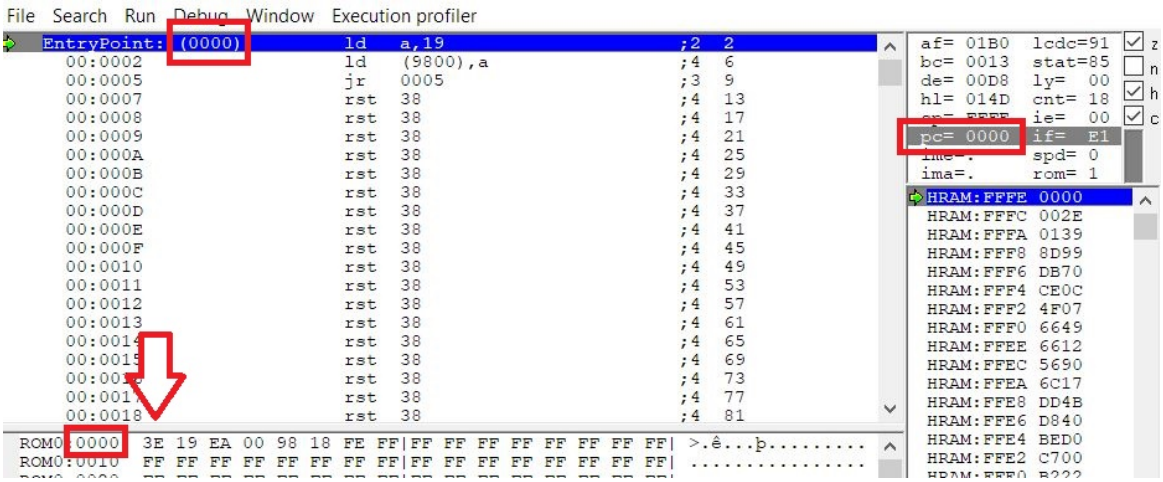



**Figura 2.5:** Inicio del programa

Empecemos mirando la sección de los registros, y centremos nuestra atención en el registro PC, resaltado en la figura 2.5. Este registro es el que indica a la CPU la dirección de memoria dónde se encuentra la siguiente instrucción o dato que debe leer. Podemos ver en la sección de código que la dirección de línea resaltada, a la izquierda de esta, coincide con el valor de PC. El ciclo de ejecución es entonces el siguiente:

- La CPU lee un byte de la dirección de memoria que indica PC, y aumenta el valor del registro en 1.
- La CPU entonces decodifica el byte leído para saber qué acción debe realizar.
- Si la instrucción decodificada necesita de más datos, los lee siguiendo el mismo proceso, leyendo de la dirección que indica PC y aumentando su valor en 1 cada vez.
- Una vez ha leído todos los bytes de datos necesarios, ejecuta la instrucción.
- Entonces la CPU vuelve al estado inicial y se empieza de nuevo desde el primer punto.

Veamos un ejemplo de este ciclo con la ejecución de nuestro programa. En la figura 2.5 podemos ver los datos en memoria que representan nuestro programa señalados por la flecha roja. Como se puede ver a la izquierda de estos, el primer byte de dicha fila corresponde a la dirección de memoria desde $0000, el de su derecha está entonces en la dirección $0001, el siguiente $0002, y así el resto de la fila hasta llegar a $000F. Las tres líneas de nuestro programa están en los 7 primeros bytes de memoria, con valores $3E $19 $EA $00 $98 $18 $FE.

1. PC=$0000. La CPU lee la dirección de memoria $0000 y aumenta PC en 1. El byte obtenido tiene un valor de $3E. Esto se decodifica como la instrucción `ld a, XX`. Esta instrucción guarda un valor de 8 bits en el registro A. El valor a guardar se obtiene de la siguiente posición en memoria.
2. PC=$0001. La CPU lee el byte de la dirección $0001 ($19), lo guarda en un buffer interno, y aumenta PC en 1.
3. PC=$0002. La CPU ejecuta la instrucción, y copia el valor $19 del buffer al registro A. Pulsa F7 para ejecutar esta instrucción.
4. PC=$0002. La CPU lee la dirección de memoria $0002 y aumenta PC en 1. El byte obtenido tiene un valor de $32. Esto se decodifica como la instrucción `ld [YYXX], a`. Esta instrucción copia el valor contenido en el registro A en la dirección de memoria YYXX.
5. PC=$0003. La dirección de memoria consta de 16 bits, o 2 bytes, por lo que la CPU leerá dos bytes más de memoria, de las direcciones $0003 y $0004. El primer byte leído ($0003) corresponde al byte bajo de la dirección (XX), y el segundo ($0004) al alto (YY). Este tipo de ordenación, almacenar primero el byte menos significativo se llama *little endian*, y es como la CPU de la Game Boy interpreta los datos de más de un byte. En nuestro ejemplo, estos dos bytes tienen valores de $00 y $98. Como la CPU ha leído dos bytes, el PC aumenta en 2 en total.
6. PC=$0005. La CPU ejecuta la instrucción, y almacena el valor de A ($19) en $9800. Pulsa F7 para ejecutar esta instrucción.
7. PC=$0005. La CPU lee la dirección de memoria $0005 y aumenta PC en 1. El byte obtenido tiene un valor de $18. Esto se decodifica como la instrucción `jr, XX`. Esta instrucción suma al registro PC el valor XX. `jr` es un acrónimo de Jump Relative (salto relativo). Al hecho de modificar el valor de PC se le suele llamar ”saltar”, puesla ejecución del programa deja de ser continua, la siguiente instrucción a ejecutar (la señalada por PC) ya no es la inmediatamente posterior a la actual en memoria.
8. PC=$0006. La CPU lee el byte de la dirección $0006 y almacena el valor ($FE) en un buffer interno, y aumenta PC en 1.
9. PC=$0007. La CPU ejecuta la instrucción y suma $FE a PC. Es importante tener en cuenta que la instrucción jr suma un valor con signo, por lo que $FE tiene un valor equivalente a -2. Esto es equivalenta a sumar $FF en el byte alto si el número es negativo, y $00 si es positivo. Al sumar el valor, PC pasa a valer $0005. Pulsa F7 para ejecutar esta instrucción.
10. PC=$0005. Ahora volvemos a estar en la misma situación que en el paso 7. El programa vuelve a ejecutar

los pasos 7 a 9 una y otra vez, ha entrado en un bucle infinito.

En definitiva, lo que hace el programa es copiar un valor de $19 e la dirección $9800 y después se queda atascado. Ahora sabemos que copiar ese valor en esa dirección provoca que aparezca una R igual a la del logo de Nintendo en la esquina superior izquierda.

## 2.2 Segundo paso, experimenta

Ya sabes que escribir un $19 en $9800 dibuja una R en la esquina superior izquierda, pero qué sucede si cambias esos valores? Una de las ventajas de usar un emulador con debugger como BGB es la posibilidad de cambiar la memoria y los registros manualmente durante la ejecución. Abre el debugger y ve a tu código del apartado anterior. Ahora mismo PC debería valer $0005 y la siguiente línea a ejecutar debería ser `jr 0005`. Ve al código en la sección de memoria, la serie de números que hemos visto antes, que empezaban en $0000. Ya sabes que de esos, el $19 indica el dato a copiar y $00 y $98 indican dónde. Cambia los valores del $19 y $00 manualmente. Simplemente haz clic en el byte que quieres cambiar, escribe el nuevo valor (en hexadecimal, no hace falta poner $ delante), y pulsa *Enter*. Usa valores menores de $19 para el dato a copiar, y no elijas un valor impar para la cifra de la izquierda de la memoria (10 a 19, 30 a 39, 50 a 59...). Tampoco modifiques el byte con valor $98. Ahora mismo la memoria estará modificada, y podrás ver encima de esta, en la sección de código, cómo este también ha cambiado, sin embargo aunque le des a ejecutar no sucederá nada, porque PC sigue valiendo $0005 y sigue atascado en un bucle. Haz doble clic sobre el registro PC en la sección superior derecha y escribe el nuevo valor, que deberá ser el del principio del código que quieres que se ejecute. En nuestro ejemplo, el de la figura 2.5, este valor es 0000 (de nuevo en hexadecimal, sin $ delante). Verás que al hacer esto, la línea resaltada de la sección de código cambia de posición. Una vez esté todo preparado, pulsa *F9* y observa el resultado.

Veamos algunos ejemplos. Si Partiendo del estado anterior (una R en la esquina) hacemos que se escriba un $12 en $9846, el resultado es el de 2.6a. Si después hacemos que se escriba un $0C en $9889, el resultado es el de 2.6b.




**(a)** $12 en $9846. **(b)** $0C en $9889.

**Figura 2.6:** Modificando el código

Puedes repetir este proceso tantas veces como quieras. Tras varias pruebas, intenta predecir qué va a suceder antes de ejecutar el nuevo código.

## 2.3 Primer obstáculo, dibuja una N

Llegamos entonces al primer obstáculo de nuestro viaje, es la hora de que dibujes algo de forma controlada, no al azar como en la sección anterior. El código de la solución no se mostrará en este libro, únicamente se mostrará el resultado esperado, y tendrás que ser tú quien compruebe que has superado el obstáculo y puedes proseguir al siguiente apartado.

Para que puedas afrontar este reto, ha llegado la hora de explicar cómo funciona parte de la memoria de vídeo, responsable de mostrar los gráficos en pantalla.

Abre el debugger de BGB y hac en la barra superior haz clic en la pestaña *Window* y después en *VRAM viewer* (también puedes pulsar *F5*). Debería haberse abierto una ventana como la de la figura 2.7.

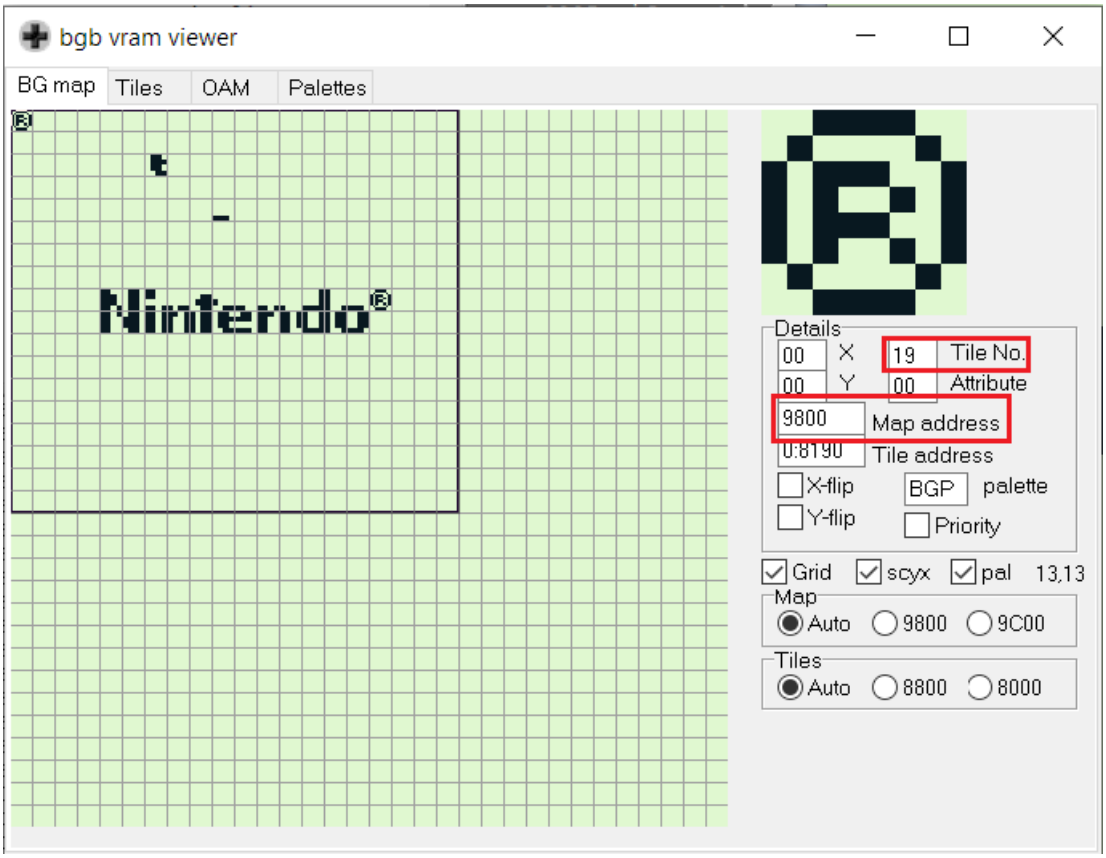



**Figura 2.7:** Memoria de vídeo

La sección de la izquierda representa un total de 1024 bytes de memoria, cada cuadrado de la cuadrícula equivale a un bytes de la memoria de vídeo. Ahora mismo sólo nos interesa la sección marcada con un rectángulo con el logo de Nintendo en el medio. Este rectángulo representa la pantalla de la consola, todo lo que está dentro de él es lo que se vemos al mirar a la pantalla principal del emulador. Al pasar el ratón por encima de cada casilla verás que la información mostrada en la derecha cambia, por ejemplo, al seleccionar la casilla superior izquierda, que contiene una R como en la imagen 2.7, podemos ver que el atributo *Map address* es igual a 9800, y el atributo *Tile No.* es 19.

Como habrás deducido, es muestra qué dirección de memoria corresponde a cada casilla, y qué valor corresponde a cada tile. Si pones el ratón sobre cualquier casilla vacía, verás que el *Tile No.* es 0, mientras que en ambas R, es 19. Puedes averiguar el valor de otros tiles que estén en pantalla del mismo modo.

Ahora te toca completar el reto, tu objetivo es dibujar la N mayúscula del logo, un total de 4 tiles, en la esquina inferior derecha de la pantalla. Vuelve al archivo *header.asm*, borra las dos líneas de código que hiciste y escribe el código necesario para que aparezca la N. Una vez lo hayas escrito, ejecuta `make` en la terminal, carga la ROM en el emulador y deja que se ejecute. Si aparece la N en la esquina habrás superado el obstáculo y podrás seguir avanzando, si no, vuelve al código y modifícalo, no modifiques la memoria directamente desde el emulador. Recuerda las dos instrucciones de las que dispones, `ld a, d8` (d8 significa dato de 8 bits) y `ld [a16], a` (a16 significa dirección de 16 bits), y no olvides de poner un $ delante de un número si lo escribes en hexadecimal. El resultado que deberías obtener es el de la figura 2.8.

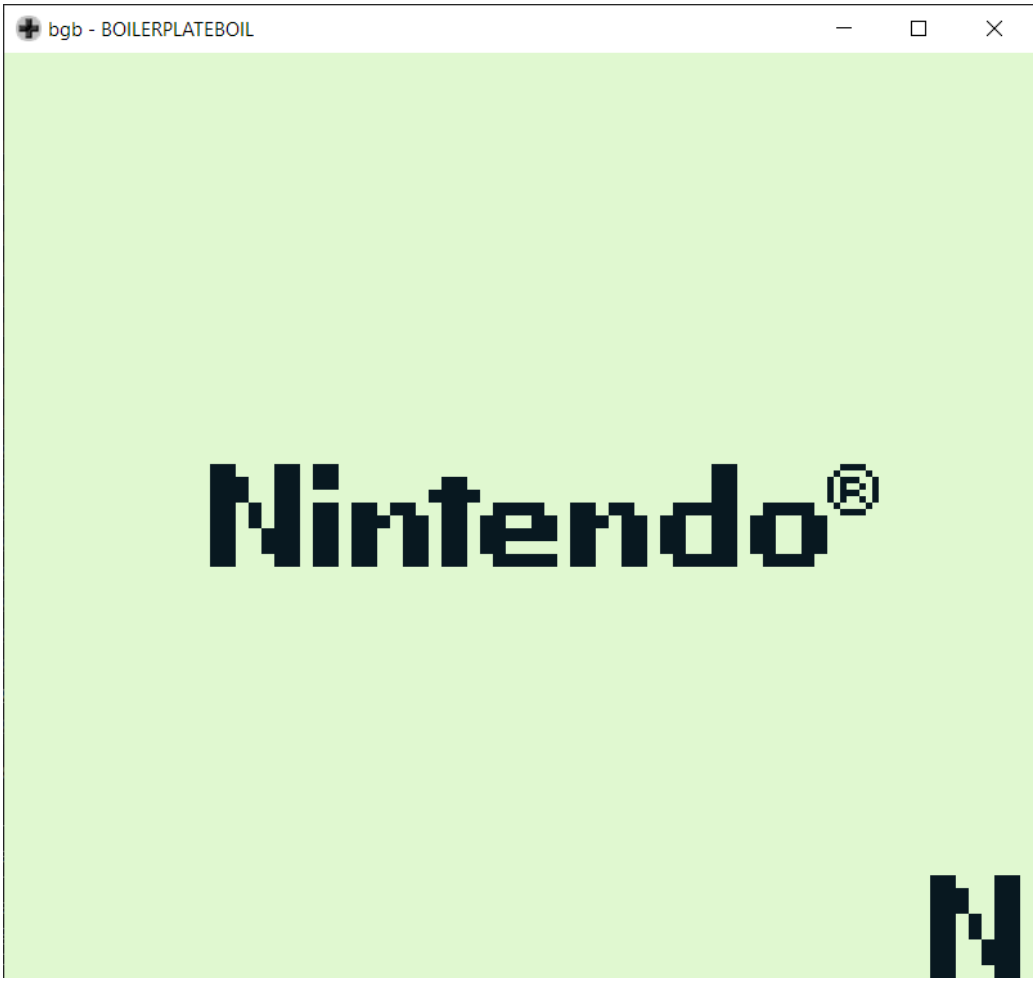



**Figura 2.8:** N mayúscula dibujada en la esquina.

## 2.4 Aprende nuevas instrucciones

Ya has comprendido las bases de escribir en memoria, pero acceder directamente no siempre será posible, habrá ocasiones en las que no sepas dónde tienes que escribir los datos antes de ejecutar el programa. Es por es-

to que también se puede usar el valor guardado en un registro como la dirección de memoria sobre la que escribir.

Ya conoces los registros A y PC. En la figura 2.9 se muestran varios registros tal y como aparecen en BGB. De estos, los encuadrados son los de la CPU, los que no requieren de una dirección de memoria para ser leídos y escritos. De estos, los registros B, C, D, E, H y L se denominan registros de propósito general, porque pueden ser usados tanto para almacenar datos como direcciones de memoria.

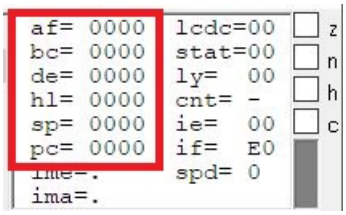



**Figura 2.9:** Registros de la CPU

En la figura 2.9 se puede ver como están agrupados dos a dos, en los pares BC, DE y HL. De las cuatro cifras que representan a cada par de registros, las dos de la izquierda pertenecen a B, D y H, y las dos de la derecha a C, E y L. Esta agrupación se debe a que por si solos, cada uno de ellos funciona como un registro de 8 bits, como A, aunque con algunas limitaciones más, pero pueden usarse a pares como un único registro de 16 bits, en el que se puede almacenar una dirección de memoria para leer y escribir sobre ella.

Podemos ver ahora las siguientes instrucciones:

- `ld r8, d8`: Almacena el valor de 8 bits d8 en el registro r8, siendo éste cualquiera de los registros de 8 bits ya mencionados (A, B, C, D, E, H, L).
- `ld r16, d16`: Almacena el valor de 16 bits d16 en el registro r16, siendo éste una pareja de registros (BC, DE o HL).
- `ld [hl], d8`: Interpreta el valor contenido en HL como una dirección de memoria y escribe en ella el valor de 8 bits d8.
- `ld [hl], r8`: Interpreta el valor contenido en HL como una dirección de memoria y escribe en ella el valor almacenado en el registro r8, siendo éste cualquiera de los registros de 8 bits ya mencionados.
- `ld [r16], A`: Interpreta el valor contenido en el registro r16 como una dirección de memoria y escribe en ella el valor del registro A en ella.
- `inc REG`: Aumenta el valor del registro REG en uno. REG puede ser cualquier registro ya mencionado excepto PC.
- `dec REG`: Reduce el valor del registro REG en uno. REG puede ser cualquier registro ya mencionado excepto PC.

Pongamos un ejemplo, el primer código que escribimos fue el siguiente:

```
ld a, $19
ld [$9800], a
```

Este código cumple la misma función que el siguiente:

```
ld hl, $9800
ld [hl], $19
```

Intenta completar el reto anterior, dibujar una N en la esquina inferior derecha, pero sin usar el registro A, y usando solo dos veces la instrucción `ld hl, d16`. Si quieres ver todas las instrucciones de las que dispone el procesador puedes consultarlas en el anexo C.

## 2.5 Flags

Uno de los últimos registros de la CPU que aún no hemos explicado es el registro F, o también llamado de *Flags* (banderas). La función de este registro es indicar cuando se dan situaciones especiales al realizar una operación aritmética. Estas situaciones son:

- Flag Z: Se activa si el resultado de la operación es cero.
- Flag C: Se activa si ha habido overflow en la operación (el resultado tras una suma es mayor que $FF, o tras una resta es menor que $00).
- Flag N: Se activa si la operación ha sido una resta.
- Flag H: Se activa si en la última operación ha habido acarreo desde los cuatro bits bajos a los altos.

Estos flags pueden verse en el margen derecho de la figura 2.9, y corresponden a los 4 bits más altos del registro F. Un flag activado se representa con la caja del flag marcado, e internamente significa que en ese bit hay guardado un 1. Los flags N y H se usan con una única instrucción, y no necesitas recordar cuando se activan o desactivan.

Las instrucciones `inc` y `dec` del apartado anterior, son operaciones aritméticas y modifican los flags Z, N y H cuando se usan con registros de 8 bits, pero no modifican ninguno cuando se usan con registros de 16 bits. El resto de operaciones aritméticas se aplican casi todas sobre el registro A, es decir, el registro A se usa tanto de operador como de almacenamiento del resultado. La instrucción `add` suma un registro, dirección de memoria o dato inmediato al registro A, guarda el resultado en A, y actualiza los flags. Para usarla sólo tienes que escribir `add n`, dónde n es el registro o valor que quieres sumar, o `[hl]` para usar como operador el valor en la dirección de memoria indicada por HL.

Vamos a hacer un programa de ejemplo para ver varias de estas instrucciones y como modifican los flags. El código que deberás escribir es:

```
;Probando INC
ld c, $FE
inc c  ;Resultado: (C=FF) Z=0, N=0
inc c  ;Resultado: (C=00) Z=1, N=0

;Probando DEC
ld b, $02
dec b  ;Resultado: (B=01) Z=0, N=1
dec b  ;Resultado: (B=00) Z=1, N=1

;Probando ADD
ld d, $80
ld a, $00
add d  ;Resultado: (A=80) Z=0, N=0, C=0
add d  ;Resultado: (A=00) Z=1, N=0, C=1
```

```
ld a, $C0
add d  ;Resultado: (A=40) Z=0, N=0, C=1
```

Recuerda poner el código entre `EntryPoint:` y `jr @`. Ensambla el programa como siempre y ejecútalo paso a paso en el emulador. Comprueba que los flags y los registros tienen los valores esperados tras cada paso.

## 2.6 Tercer paso, usa los bucles

A la hora de escribir un programa hay muchas acciones que se repiten constantemente, o que apenas cambian. Por ejemplo, si quieres dibujar una línea entera de la pantalla, sería muy molesto tener que escribir cada dirección de memoria a mano. Para esto podemos usar bucles, que hagan la misma acción de manera continua. Para programar un bucle, se requiere saltar atrás en el código, es decir, modificar el registro PC, y como hemos visto en apartados anteriores, la instrucción `jr` tiene esta función. Ahora mismo la única instrucción `jr` de nuestro programa provoca que éste se quede atascado, le hace saltar a esa misma instrucción continuamente. El símbolo `@` que hay tras la instrucción es una forma de indicar que la línea a la que saltar es la actual. A la hora de ensamblar el programa, el símbolo adquiere el valor de la dirección de memoria en la que se encontrará esa línea. Para poder controlar el bucle, tenemos que poner después de jr la dirección de memoria a la que queremos que el programa salte.

Ya habrás notado que el programa empieza ejecutando las dos instrucciones de arriba (`di` y `jp EntryPoint`) y entonces salta a la sección de abajo. Esto es porque `jp` también es una instrucción de salto y `EntryPoint` es una dirección de memoria. Al escribir una cadena de texto con un punto en ella, o con dos puntos al final (la diferencia entre ambos se verá más adelante), como le pasa a `EntryPoint`, ésta se define como una dirección de memoria, y su valor es la dirección en la que empieza la siguiente instrucción. A esto se le llama etiqueta. Al saltar a la etiqueta desde la instrucción `jp`, la siguiente línea en ejecutarse es la de abajo, que en nuestro primer programa era `ld a, $19`.

Podemos entonces crear un bucle cambiando el símbolo `@` por `EntryPoint`. Para ver este bucle, crea un programa sencillo en el que sólo se aumente el valor del registro A en uno en cada pasada, y ejecútalo paso a paso. Deberías ver cómo el registro A va aumentando. En este momento puedes probar a cambiar `jr` por `jp` y ver las diferencias en la memoria, además del código de la instrucción, que como ya vimos era $18 para `jr`.

Hacer bucles está muy bien, pero sin una forma de salir de ellos no son muy útiles porque el programa se quedaría atascado, y aquí es donde entran los flags. Además de instrucciones para hacer saltos siempre, existe un conjunto de instrucciones que realizan o no el salto en función del estado de algún flag, es decir, si se cumple la condición, la ejecución seguirá dónde diga la instrucción, habiendo ésta modificado PC, mientras que si no, PC habrá aumentado de forma natural y la ejecución seguirá con la siguiente instrucción. La sintaxis para las instrucciones de salto condicionales es: `jp/jr cc, n16`. `jp` y `jr` diferencian entre salto absoluto y salto relativo, internamente, las instrucciones `jp` almacenan la dirección de destino completa y la copian en PC, mientras que `jr` almacena un byte y lo suma a PC. `jr` está limitada a hacer saltos de máximo 128 bytes, mientras que `jp` puede saltar a cualquier parte de la memoria. cc representa la condición, que pueden ser las siguientes:

- z: La condición se cumple si el flag z está a 1 (la última operación dio 0 de resultado).
- c: La condición se cumple si el flag c está a 1 (la última operación sobrepasó el límite de 8 bits del registro).
- nz: La condición se cumple si el flag z está a 0.
- nc: La condición se cumple si el flag c está a 0.

Pot último, n16 es la dirección a la que hacer el salto, que prácticamente siempre será una etiqueta.

Vamos a mostrar un ejemplo, el siguiente código dibuja 3 tiles de R en horizontal empezando en la esquina superior derecha de la pantalla:

```
EntryPoint:
    ld a, [rLY]
    cp 144
jr nz, EntryPoint

    ld de, $9800
    ld a, $19
    ld b, 5
.loop
    ld [de], a
    inc de
    dec b
jr nz, .loop

jr @
```

Las indentaciones (o sangrado, las líneas que están más a la derecha que el resto) sólo tiene la función de dar claridad al código. Escribe también lo siguiente en la primera línea del archivo:

```
include "include/hardware.inc"
```

El resultado tras ejecutar el programa debería ser el de la figura 2.10.




**Figura 2.10:** Ejemplo de un bucle.

Vamos a analizar qué hace el código, recomiendo que pruebes a ejecutarlo línea a línea mientras sigues la explicación y compruebes qué está cambiando como se esperaría.

```
EntryPoint:
    ld a, [rLY]
    cp 144
jr nz, EntryPoint
```

Esta sección no la explicaremos por ahora, sólo debes saber que si no estuviera ahí, no se podrían haber dibujado los 5 tiles seguidos con el código que viene a continuación. Para saltar esta sección (que es un bucle largo) sin tener que pulsar *F7* muchas veces, haz doble clic en la línea siguiente en el debugger, debería cambiar a color rojo, como en la figura 2.11. Si ahora dejas que el programa siga ejecutándose en tiempo real, cerrando la ventana del debugger o pulsando *F9*, el programa se detendrá y volverá a abrir el debugger al alcanzar la línea marcada en rojo. A esta característica se la llama ”breakpoint”.

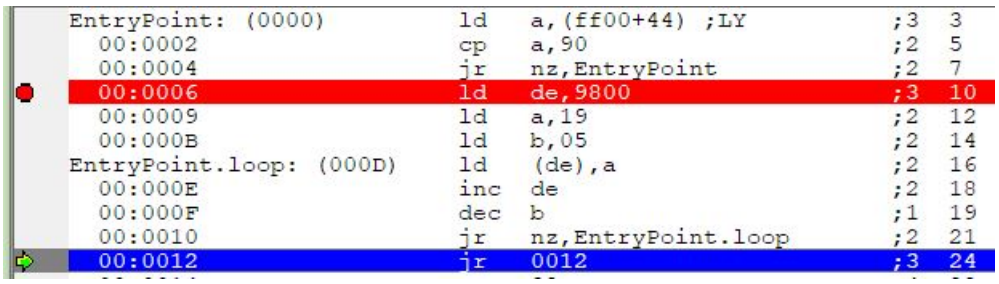



**Figura 2.11:** Ejemplo de un breakpoint.

```
    ld de, $9800
    ld a, $19
    ld b, 5
```

En las tres líneas anteriores guardamos en DE la dirección de memoria de la esquina superior izquierda, en A el identificador del tile que vamos a dibujar, y en B el número de veces que vamos a dibujarlo. Esta sección prepara el estado inicial del bucle. B tiene la función de almacenar el número de iteraciones.

```
.loop
   ld [de], a
   inc de
   dec b
jr nz, .loop
```

Empezamos declarando la etiqueta `.loop`, que marcará el punto de inicio del bucle. Las declaraciones de etiquetas no ocupan memoria en el programa. Después, con `ld [de], a`, escribimos en la dirección de memoria indicada por DE ($9800) el valor que hay en A ($19). Esto dibuja el primer tile en la esquina superior izquierda. La siguiente instrucción, `inc de`. Aumenta el valor de DE en uno, haciendo que pase a ser $9801. `dec b` reduce el valor de B en uno, siendo ahora 4, y por último `jr nz, .loop` comprueba si la última operación (reducir B en uno) ha dado de resultado cero, y si no ha sido así, cambia el valor de PC para que sea el de la etiqueta `.loop`. Como el resultado ha sido 4, la ejecución del programa vuelve a la primera línea del bucle. Esta vez, DE vale $9801, por lo que se dibuja el tile adyacente al anterior. El bucle se repite hasta que B pasa a valer 0, en cuyo caso no se hace el salto y la ejecución sigue con la línea siguiente.

La última línea es `jr @`, y como ya sabes, esta línea hace un salto a sí misma, y el programa se queda en un bucle infinito.

## 2.7 Segundo obstáculo, dibuja un triángulo

Ha llegado la hora del siguiente obstáculo, te toca a ti hacer el programa. Tu objetivo esta vez es llenar una esquina de la pantalla, como en la figura 2.12.




**Figura 2.12:** Rellenada una esquina de tiles.

Son un total de cinco líneas, todas empiezan en el borde izquierdo, y su longitud va decreciendo. No es necesario que uses el mismo tile que en el ejemplo, ni que todas las líneas usen el mismo tile, puedes ir cambiándolo. La única limitación es que sólo puedes usar como máximo cinco instrucciones en las que escribas en memoria. Recuerda incluir al principio del código la misma sección que en el apartado anterior:

```
   EntryPoint:
      ld a, [rLY]
      cp 144
   jr nz, EntryPoint
```

Como última anotación, en caso de que necesites crear varias etiquetas, no se pueden declarar dos con el mismo nombre, debes darles nombres distintos a todas ellas. Cuando hayas conseguido completar el objetivo, puedes pasar al siguiente apartado.

## 2.8 Cuarto paso, crea una función

En el último ejercicio muy probablemente habrás repetido la misma estructura en el programa varias veces, primero asignas valores a varios registros, y luego ejecutas un bucle que usa esos valores. Los bucles probablemente tengan todos las mismas instrucciones, y como mucho cambiará el nombre de las etiquetas. En este apartado veremos como ahorrar memoria y evitar repetir código usando funciones.

Una función, es una rutina, una serie de instrucciones, que puede ser invocada desde otra parte del código. Todo lo que se necesita para llamarlas es la dirección de memoria en la que se encuentran, comúnmente usando una etiqueta. Para nuestra función, que será para dibujar líneas con el mismo tile, usaremos una global para su nombre, y locales si hacen falta para algún bucle. Si necesitas más información sobre los distintos tipos de etiquetas, puedes consultar el anexo B.1.1.

También debemos tener claro en qué registros irá cada variable de la función, es recomendable ponerlo en un comentario antes de ésta, lo cual nos ayudará en el futuro cuando queramos saber qué parámetros pasarle a alguna función cuando vayamos a invocarla sin necesidad de revisar el código. Para escribir un comentario, introduce un punto y coma en una línea, todo lo que vaya a continuación en esa línea será un comentario y se ignorará al ensamblar.

El siguiente segmento sería una función que copia el mismo valor almacenado en A, sobre una cantidad B de bytes seguidos empezando en HL. Nosotros lo usamos para dibujar una línea dibujando en memoria de vídeo, pero puede usarse en cualquier dirección de memoria.

```
;Escribe un mismo valor en varios bytes seguidos.
;Input A: Valor a escribir.
;Input B: Número de veces a escribir el valor, debe ser al menos 1.
;Input HL: Dirección dónde empezar a escribir.
EscribeLinea:
   ld [hl+], a
   dec b
   jr nz, EscribeLinea
ret
```

Verás dos instrucciones nuevas en este fragmento de código. La primera de ellas, `ld [hl+], a`, almacena el valor de A en la dirección de memoria indicada por HL, y aumenta HL en uno. Es equivalente a usar `ld [hl], a` seguida de `inc hl`. El bucle va copiando A sobre la dirección indicada en HL y aumentando su valor para escribir en el siguiente byte en la siguiente iteración. La otra nueva instrucción, `ret`, sirve para que cuando la CPU la ejecute, salte a la dirección desde dónde se llamó a la función. Cómo hace esto lo veremos en el siguiente apartado.

Incluye esta función en tu proyecto, para llamarla, sólo tienes que escribir la instrucción `call EscribeLinea`, y el programa saltará a la función cuando se ejecute esa línea. Ahora puedes sustituir los bucles repetidos por llamadas a esa función (si estabas usando el registro DE en vez de HL, cámbialo, porque no existe la instrucción `ld [de+], a`, déjalo como `ld [de], a` más `inc de`). El código debería ser ahora mucho más claro, aunque como la función era pequeña, no se habrá ahorrado mucho espacio. Puedes crear más funciones si quieres y llamarlas desde dentro de otras, pero debes recordar añadir la instrucción `ret` al final, o la CPU no volverá al lugar desde el que se llamó y leerá de la siguiente dirección de memoria.

Vuelve a ejecutar el programa y comprueba que sigue funcionando igual, y dibuja la esquina como querías.

## 2.9 La pila y el registro SP

Para realizar un salto a una función, como ya sabes se debe cambiar el registro PC, pero si sólo se hiciera esto la CPU no sabría volver al punto desde el que se hizo la llamada, necesitaría almacenar la dirección desde la que se llamó a la función y restaurarla al final de su ejecución. Además, debe ser posible llamar a funciones dentro de otras, por lo que es necesario tener una lista en la que se vayan añadiendo las direcciones desde las que se invocan funciones y eliminando una vez se vuelva al punto de la llamada. Para esto (y para otras funciones) existe la pila, también llamada stack. Vamos a ver el último registro de la CPU y la última sección del debugger de BGB que aún no hemos visto.

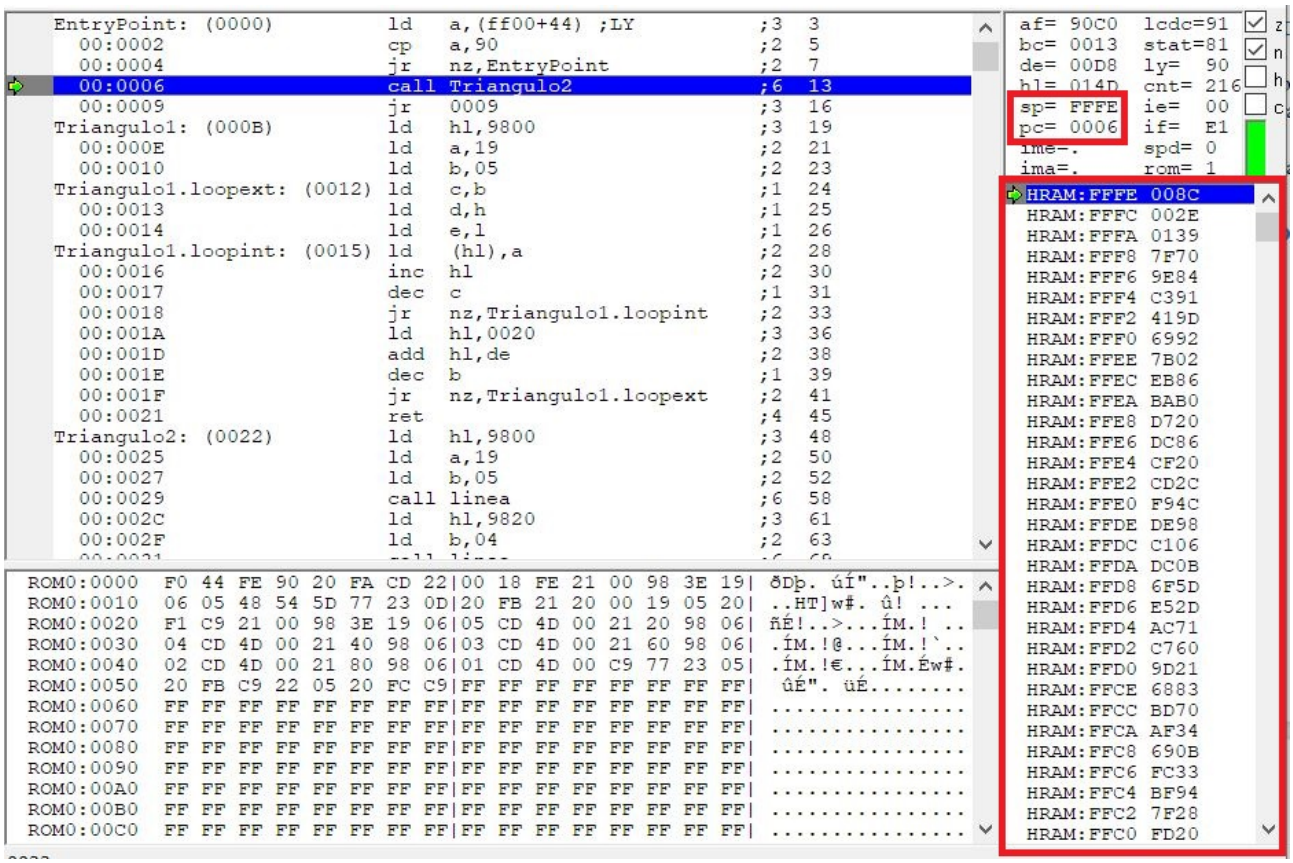



**Figura 2.13:** Ejemplo de uso de la pila.

Vuelve a ejecutar el código del apartado anterior y presta atención a la pila (la sección inferior derecha) y los registros PC y SP. Puedes ver cómo desde un principio hay una línea resaltada en azul en la pila, cuya dirección coincide con el valor de SP. El registro SP se llama ”stack pointer”, o puntero de pila, y almacena la dirección de la cima de la pila. La estructura de datos se llama pila, porque funciona igual que si apilaras objetos, cuando añades uno, se añade en la parte de arriba, y cuando quitas uno, se quita el de más arriba, el último añadido, o el siguiente anterior si quitas varios objetos seguidos. En este caso funciona igual, cuando añades un objeto a la pila, SP se reduce en dos, para apuntar a la nueva cima de la pila.

Fíjate en lo que sucede cuando se ejecuta una instrucción `call`, como es de esperar, PC cambia a la dirección de la etiqueta de la llamada, pero también se añaden dos bytes a la pila, y consecuentemente se reduce SP en dos. Ahora sigue ejecutando hasta llegar a una instrucción `ret`. Cuando esta instrucción se ejecuta, SP aumenta en dos, lo cual es equivalente a ”quitar”dos bytes de la pila, y PC cambia a la dirección siguiente a la

llamada que inicialmente movió la ejecución a esa función.

Puedes ahora fijarte mejor y ver que lo que hace `call` es reducir SP en dos y copiar ahí la dirección de la instrucción siguiente a la suya (internamente, PC se ha ido aumentando automáticamente al leer la instrucción `call`, por lo que no hay que hacer ningún cálculo adicional), y `ret` copia en PC el valor que en los bytes a los que apunta SP, y aumenta este registro en dos.

De este modo se pueden llamar funciones dentro de otras y no perderse el hilo de ejecución, las funciones que fueron llamadas primero serán las últimas de las que se extraiga el punto de origen de la pila. La pila mantiene un registro de desde dónde se han hecho saltos y en qué orden, para poder restaurar PC cada vez.

## 2.10 El registro STAT

Recordarás hace un par de secciones incluimos en el código algunas líneas sin la que supuestamente no se habría podido dibujar varios tiles seguidos. El código en concreto era el siguiente:

```
EntryPoint:
    ld a, [rLY]
    cp 144
jr nz, EntryPoint
```

Prueba a eliminar las tres líneas del bucle (obviamente no elimines `EntryPoint:` o el programa no se ejecutará) y ejecuta el código del apartado anterior, que dibujaba una sección triangular en una esquina. Ahora el resultado debería ser el de la figura 2.14.

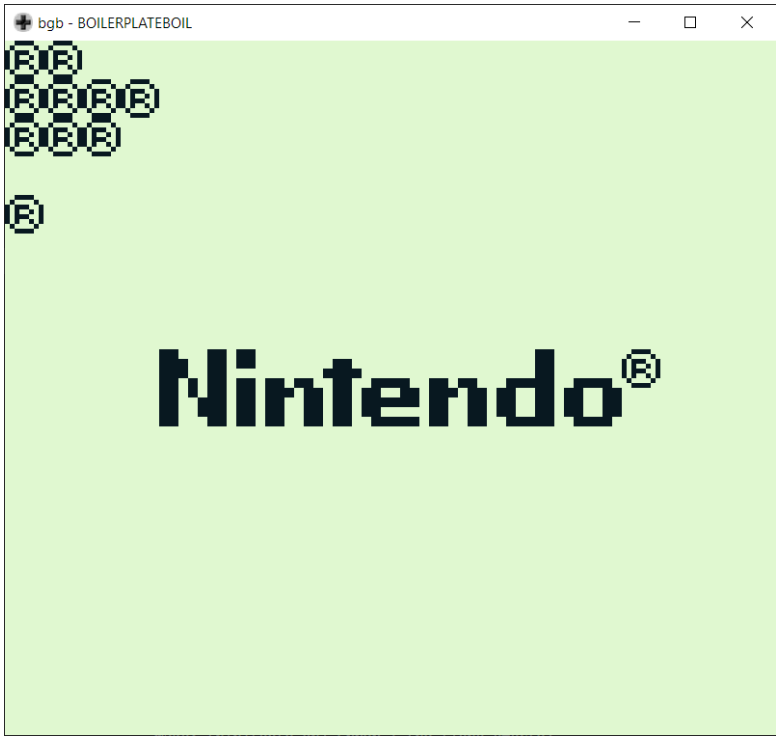



**Figura 2.14:** Esquina parcialmente dibujada.

Esto sucede porque no se puede escribir en la memoria de vídeo en todo momento. La consola va leyendo

la memoria de vídeo y dibujando la pantalla intermitentemente, y en estos periodos en los que está dibujando la pantalla no es posible escribir ni leer la memoria de vídeo. Puedes ver esto mismo en el emulador, si desplazas la vista de la sección de memoria hasta la de vídeo (con una rom cargada, haz clic derecho sobre la sección y haz clic en *Go to...* o pulsa *Control* + *G*, introduce 9800 y haz clic en *Ok*). Si mantienes pulsado *F7* podrás ver cómo la memoria de vídeo cambia a FF cada cierto tiempo, y es en estos periodos en los que no se puede leer ni escribir sobre ella.

La unidad encargada de dibujar la pantalla es la Unidad de Procesamiento de Imagen (PPU por sus siglas en inglés) y en todo momento puede estar en cuatro estados, también llamados modos, distintos. La PPU va dibujando línea tras línea, alternando entre 3 de estos modos en cada una, y finalmente está un tiempo equivalente a dibujar 10 líneas en el cuarto modo. Estos modos son:

- Modo 3: Este es el momento en el que la PPU está dibujando, y la memoria de vídeo es totalmente inaccesible.
- Modo 2: Cuando la PPU está en este modo, está buscando los sprites que intersectan con la siguiente línea. Los sprites, también llamados objetos, son un elemento gráfico independiente del fondo que no veremos hasta mucho más adelante. Toda la memoria de vídeo es accesible en este modo a excepción de la perteneciente a los objetos.
- Modo 1: Este es el modo en el que se encuentra la PPU durante el periodo de diez líneas posterior a dibujar la pantalla entera, mientras espera al siguiente frame. En este momento es accesible toda la memoria de vídeo. A este modo se le suele llamar VBlank, o vertical blank.
- Modo 0: Tras terminar de dibujar una línea en el modo 2, la PPU pasa a este modo mientras espera al principio de la siguiente línea. También se puede acceder a la memoria de vídeo en este modo. A este modo se le suele llamar HBlank o horizontal blank.

Durante el dibujado de un frame, la PPU alterna entre estos modos en el orden *2 >3 >0* 144 veces, una por cada línea de la pantalla, y luego cambia al modo 1. Tras esto el proceso se repite. Volviendo al código que prevenía que dibujáramos durante el modo 3:

```
EntryPoint:
    ld a, [rLY]
    cp 144
jr nz, EntryPoint
```

La primera línea, como supondrás, carga en el registro A el valor almacenado en la dirección de memoria `rLY`. `rLY` es un símbolo (un nombre que representa un número) que equivale a la dirección de memoria dónde se encuentra el registro `LY`, y que está definido en el fichero `harware.inc`, que incluimos en la primera línea del código. La directiva `include` es equivalente a copiar y pegar el contenido completo del archivo en esa línea, y cómo `rLY` se define en ese fichero, también se define en el nuestro. Este registro es de sólo lectura, y contiene la línea que la PPU está dibujando. El registro toma valores entre 0 y 153. Los valores de 144 a 153 representan el periodo de diez líneas del modo 1, en el que toda la memoria de vídeo es accesible. La siguiente línea, la instrucción `cp N` (N puede ser tanto un valor como un registro de 8 bits, excluyendo F. También puede ser el valor del byte indicado por HL), realiza la operación de resta con el registro A, sin guardar el resultado, pero modificando los flags en función de este. Es decir, la instrucción está comparando el valor de A (línea actual) con 144, y si el resultado no es cero, la siguiente instrucción hace que el programa vuelva atrás y repita la comparación. El resultado es que la ejecución entra en bucle hasta alcanzar el modo 1 al llegar a la línea 144,

dejándonos un tiempo más largo para modificar la memoria de vídeo de forma seguida que el que tenemos si escribimos mientras la PPU va alternando entre los otros tres modos.

Sin embargo el periodo VBlank puede no ser suficiente para dibujar todo lo que queramos, y tendríamos que seguir comprobando que no se sale de este intervalo en mitad de dibujar, por lo que vamos a escribir una nueva forma de comprobar si la memoria es accesible. Podemos dibujar entonces en los modos 0, 1 y 2, sin embargo podría darse el caso de que comprobáramos el modo actual estando casi al final del 2, y entonces al intentar escribir hubiera cambiado al 3. Por este motivo consideraremos que es seguro escribir sobre memoria de vídeo cuando al leer comprobemos que la PPU se encuentra en los modos 0 o 1.

El modo actual podemos verlo en el registro STAT. El emulador muestra el estado de este registro en la sección superior derecha 2.15.

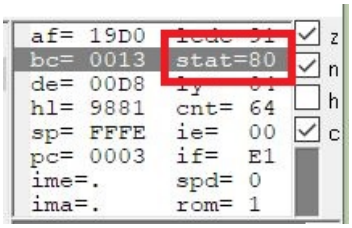



**Figura 2.15:** Registro STAT

Los dos bits más bajos, el 0 y el 1 (los bits están numerados de 0 a 7, empezando por la derecha) representan el modo. En el caso de la imagen, se encuentra en el modo 0.

Es hora de empezar a escribir el código. Nuestro objetivo es crear un bucle en el que se quede el programa hasta que la PPU se encuentre en los modos 0 o 1. Para esto empezaremos guardando el registro STAT en A. Podemos leer STAT usando la dirección $FF41, pero usar un símbolo es mucho más sencillo, y no requiere de recordar este valor o revisarlo cada vez que lo necesitemos. Para este propósito, `hardware.inc` ya incluye la definición del símbolo `rSTAT`, con valor $FF41, por lo que todo la línea que necesitamos es `ld a, [rSTAT]`. Ahora queremos comprobar que se encuentre en los modos 0 o 1, o lo que es lo mismo, no se encuentre ni en el modo 2 ni en el 3. Podemos observar en las representaciones binarias de los cuatro números (0:00, 1:01, 2:10, 3:11) que el bit de la izquierda (el bit 1 del registro) es 0 en los modos 0 y 1, y 1 en los modos 2 y 3. Por lo que nuestro código debe quedarse en bucle mientras el valor de ese bit sea 1, y salir de él cuando sea 0.

Vamos a aprovechar esta comprobación para introducir las operaciones aritméticas. Todas las operaciones aritméticas toman el registro A como primero operando y como destino del resultado, con unas pocas excepciones que no veremos ahora mismo. En todos los casos siguientes, N puede ser cualquier valor o registro de 8 bits, a excepción de F, y también puede ser el valor almacenado en la dirección de memoria HL, que se representaría como [HL].

- ADD N: Suma A y N y guarda el resultado en A.
- ADC N: Suma A, N y el valor del bit del flag C, y guarda el resultado en A.
- SUB N: Le resta N a A y guarda el resultado en A.
- SCB N: Le resta N y el valor del bit del flag C a A, y guarda el resultado en A.
- AND N: Realiza la operación A and N y guarda el resultado en A.

- XOR N: Realiza la operación A xor N y guarda el resultado en A.
- OR N: Realiza la operación A or N y guarda el resultado en A.
- CP N: Le resta N a A, actualizando los flags, pero no guarda el resultado.

Vamos a ver más en detalle las operaciones lógicas and, or y xor, ya que vamos a necesitarlas para obtener el valor del bit 1. Cada una de estas operaciones toma dos bits y devuelve otro. No es diferente a una operación matemática habitual, como add, que recibe dos números y devuelve otro (la suma) El valor del bit devuelto para cada operación en función de los de entrada es el siguiente:

| A | B | A and B |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

**Cuadro 2.1:** Operación and.

| A | B | A or B |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

**Cuadro 2.2:** Operación or.

| A | B | A xor B |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

**Cuadro 2.3:** Operación xor.

En el caso de operadores de 8 bits, los números se comprueban bit a bit.

Supongamos que la siguiente es una representación del valor del registro *STAT*: `nnnnnnXn`. Sólo nos interesa el valor del bit marcado con una X, el resto son irrelevantes. Necesitamos alguna forma de extraer únicamente el valor de ese bit. Si nos fijamos en la operación and, podemos ver que siempre que alguno de los bits sea 0, el resultado será 0, y si uno de los bits es 1, el resultado es igual al otro bit. Podemos entonces realizar una operación and entre el registro y el número binario `%00000010`. Al realizar esta operación, el resultado será `000000X0`, obteniendo el valor del bit X. A este concepto se le llama máscara. Si realizamos una operación and podemos poner a 0 los bits que queramos de un operador y mantener el resto usando como segundo operador un número que tenga a 0 los bits que queremos descartar y a 1 los que queremos mantener. Con las operaciones or y xor podemos hacer algo similar. Con or podemos poner a 1 los bits que queramos manteniendo el resto poniendo en el segundo operador a 1 los bits a escribir y a 0 los que se quiera mantener. Con xor podemos invertir el valor de los bits que queramos usando unos en el operador.

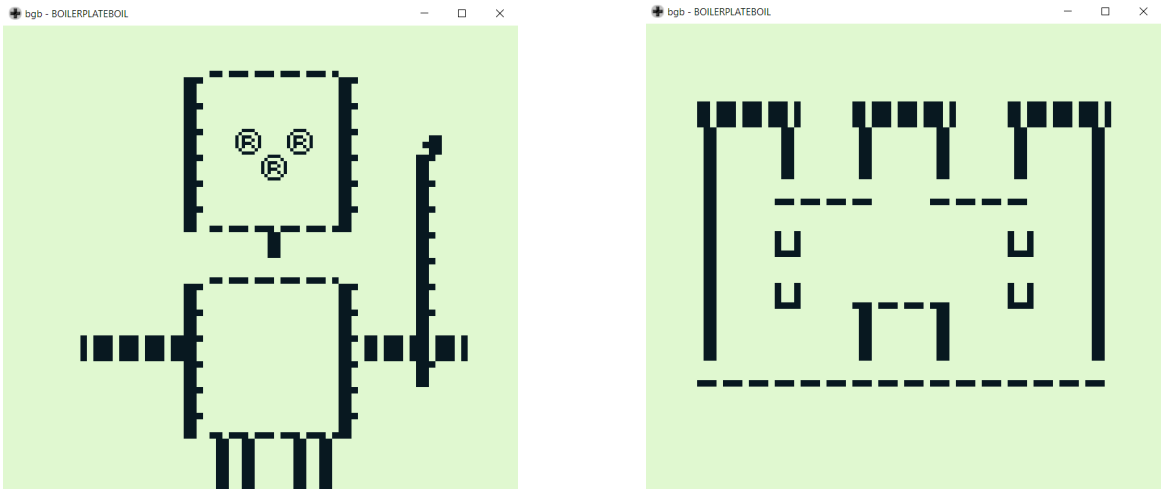
En definitiva, la operación que necesitamos, y la instrucción que irá en la siguiente línea, es `and %00000010`. El resultado de la operación será 0 si la PPU se encuentra en los modos 0 o 1, y distinto de 0 si está en los modos 2 o 3. Entonces podemos comprobar el flag Z para decidir si saltar atrás o no con la instrucción `jr nz, etiqueta`. Define tú mismo la etiqueta en la línea anterior a la que lee el registro *STAT* y llámala cómo quieras. Ahora puedes poner este código dentro de la función que dibuja una línea, justo antes de la escritura en memoria, para que el programa no dibuje durante los modos 2 o 3. Si quieres puedes en su lugar crear una función con el código y llamarla siempre que quieras escribir en memoria de vídeo.

Si guardabas el valor a escribir en el registro A, este se perderá cuando hagas la lectura del registro *STAT*. Cambia el registro por otro de 8 bits. Recuerda que entonces el único registro de 16 bits que se puede usar como puntero para escribir el valor de cualquier registro de 8 bits es HL, el resto sólo funcionan con A. Después de hacer los cambios necesarios a la función, no olvides actualizar el comentario que indica en qué registros se pasan las variables, ni cambiar en las llamadas a la función los registros en los que se guardan. Una vez esté todo hecho, genera la ROM y comprueba que esta vez la esquina se dibuja entera.

Ahora ya puedes dibujar en cualquier parte de la pantalla sin inconvenientes.

## 2.11 Haz tu propio dibujo

Ya dispones de todas las herramientas para dibujar en pantalla sin problemas (aunque sólo con los tiles iniciales que forman el logo de Nintendo), pero no puedes ponerte a dibujar y dejar el logo en medio, así que empieza por borrarlo. Una vez hayas hecho esto, prueba a hacer alguna figura de al menos 16 tiles de ancho o alto. En las figuras 2.16a y 2.16b tienes algunos ejemplos. Como anotación final, en caso de que quisieras hacer líneas verticales, dos tiles adyacentes verticalmente están separados por 32 bytes, y puedes realizar una operación de suma sobre el registro HL usando uno de los otros registros de 16 bits, con `add hl, r16`, que suma HL y el registro r16 y guarda el resultado en HL. Esta es una de las pocas excepciones en las que una operación aritmética no usa el registro A.



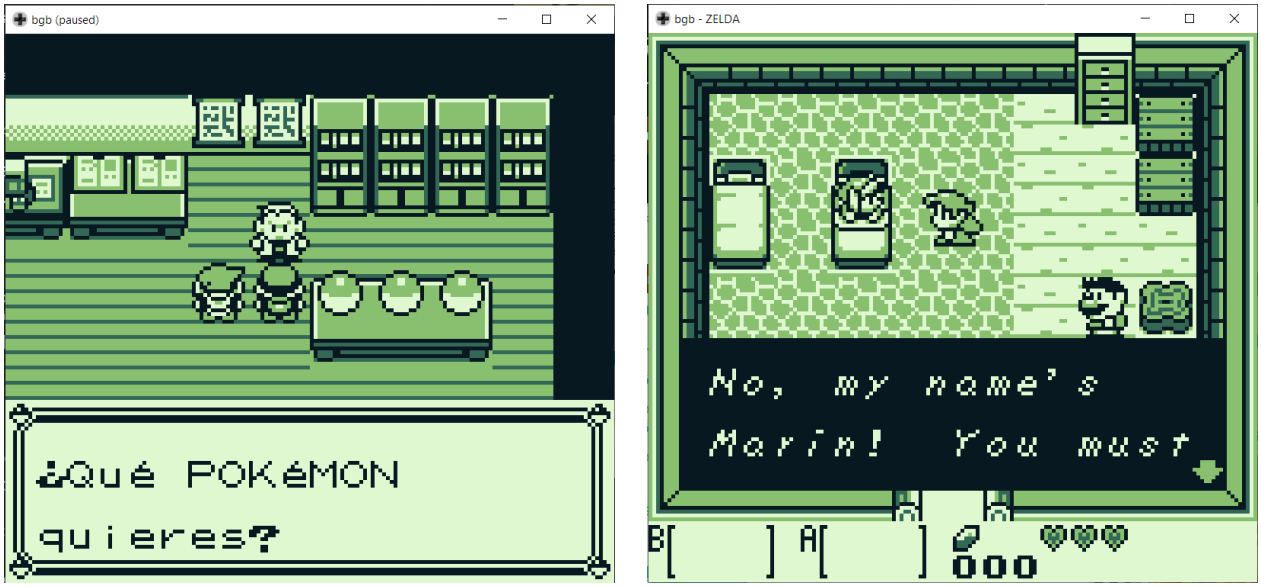

**(a)** Dibujo de una persona. **(b)** Dibujo de un castillo.

**Figura 2.16:** Dibujos de ejemplo

Cuando consideres que el resultado es satisfactorio habrás completado esta sección, y ya dominarás las bases de la programación en ensamblador para Game Boy y del dibujado de gráficos.

# Capítulo 3 Diálogos

Tras completar el capítulo anterior, ya te has armado con las herramientas necesarias para poder dibujar lo que quieras en la pantalla. Nuestro siguiente destino será conseguir mostrar diálogos, es decir, representar conversaciones en pantalla entre personajes. El uso de diálogos es una herramienta muy útil para dar vida a nuestros personajes y contar una historia. Podemos ver ejemplos de esto en gran variedad de juegos, como por ejemplo en 3.1.




**(a)** Profesor Oak hablando con el jugador. **(b)** Marin hablando con Link.

**Figura 3.1:** Conversaciones en juegos.

Nuestro objetivo será conseguir dibujar un cuadro de texto similar en el que se muestren varias líneas de diálogo seguidas.

## 3.1 Crear una fuente

Lo primero que necesitaremos para poder dibujar letras, es crear nuestros propios tiles con las letras que necesitamos. Es hora de abrir Game Boy Tile Designer, uno de los programas que descargaste durante la instalación.

Las distintas zonas de edición, según se definen en la imagen 3.2 son:

- 1 - Selección de herramienta: En esta sección se puede elegir entre pintar píxel a píxel con el lápiz, o rellenar una zona con el cubo. Las otras herramientas sirven para desplazar o rotar el dibujo.
- 2 - Lienzo: Cada casilla de está región representa un píxel del tile.
- 3 - Paleta: Haciendo clic en los colores de aquí se puede cambiar el color seleccionado.
- 4 - Lista de tiles: Cada casilla de esta zona es un tile distinto. Puedes cambiar de diseño haciendo clic en un tile.

Queremos diseñar una fuente, un conjunto de letras. Los tiles con estas letras estarán dispuestos uno al lado de otro cuando formen el texto. Si no queremos que se toquen entre sí, dificultando diferenciar las letras en

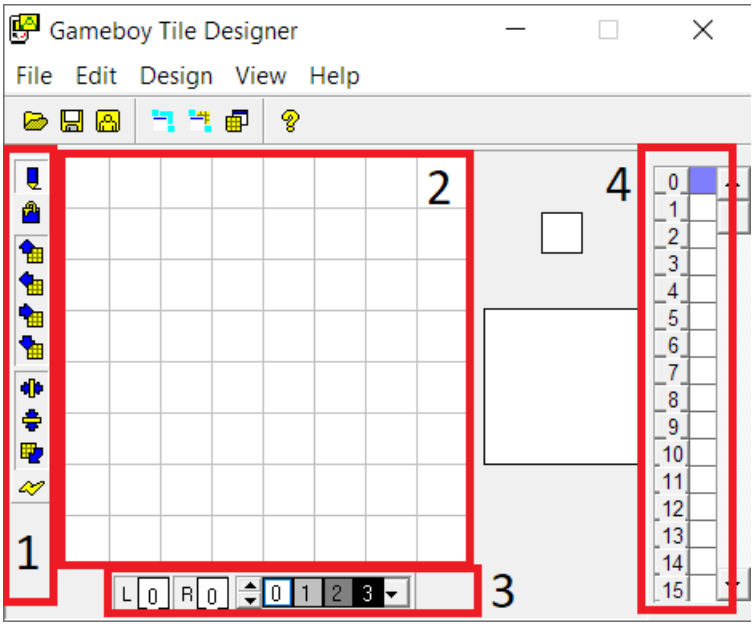



**Figura 3.2:** Game Boy Tile Designer

el texto, hay que dejar los bordes libres. Únicamente es necesario dejar en blanco dos bordes, uno lateral y el superior o inferior, pero puedes dejar los cuatro bordes en blanco si quieres.

Empieza a dibujar en el tile 0 la letra A mayúscula, la B en el 1, y sigue así con el resto del abecedario sin incluir la Ñ. Deberías haber acabado con la Z en el tile 25. Ahora dibuja las letras minúsculas de la misma forma empezando por el tile 26, es decir en el 26 irá la a, en el 27 la b, así hasta llegar a la z en el tile 51.

Una vez hayas dibujado todas las letras, haz clic en *File*, y después en *Export to...*. En la nueva ventana, siguiendo el orden indicado en 3.3, primero selecciona el destino del archivo a guardar, después introduce el nombre de la etiqueta que usarás para referirte a los datos de los tiles, selecciona el rango de tiles a exportar, que debería ser del 0 al 51, y haz clic en *OK* para exportar el archivo. No olvides hacer también un guardado normal haciendo clic en la opción *Save* en vez de *Export to...*.

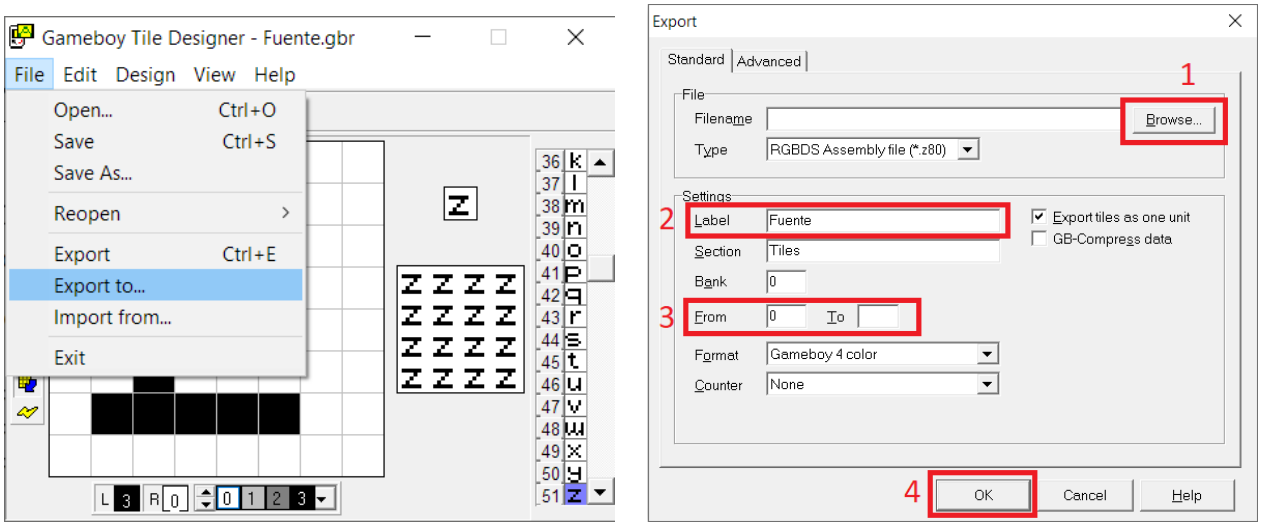



**Figura 3.3:** Exportar un conjunto de tiles.

Si guardas el archivo, más adelante podrás modificarlo para añadir nuevos caracteres, como signos de puntuación o letras con acento. Tras exportar deberían haberse creado dos archivos, uno *.z80* y otro *.inc*. Sólo nos interesa el primero. Copiálo en la carpeta *src/res/* del proyecto y cámbiale la extensión de z80 a asm. Ahora abre el archivo en VSCode.

```
; FUENTE.Z80
;
; Tile Source File.
;
; Info:
;   Section              : Tiles
;   Bank                 : 0
;   Form                 : All tiles as one unit.
;   Format               : Gameboy 4 color.
;   Compression          : None.
;   Counter              : None.
;   Tile size            : 8 x 8
;   Tiles                : 0 to 57
;
;   Palette colors       : None.
;   SGB Palette          : None.
;   CGB Palette          : None.
;
;   Convert to metatiles : No.
;
; This file was generated by GBTD v2.2
```

```
SECTION "Tiles", ROM0

; Start of tile array.
Fuente::
DB $00,$00,$10,$10,$28,$28,$28,$28
DB $44,$44,$7C,$7C,$82,$82,$82,$82
DB $00,$00,$FC,$FC,$82,$82,$82,$82
DB $FC,$FC,$82,$82,$82,$82,$FC,$FC
DB $00,$00,$3E,$3E,$40,$40,$80,$80
DB $80,$80,$80,$80,$40,$40,$3E,$3E
DB $00,$00,$F8,$F8,$84,$84,$82,$82
DB $82,$82,$82,$82,$84,$84,$F8,$F8
DB $00,$00,$FE,$FE,$80,$80,$80,$80
DB $FC,$FC,$80,$80,$80,$80,$FE,$FE
DB $00,$00,$FE,$FE,$80,$80,$80,$80
DB $FC,$FC,$80,$80,$80,$80,$80,$80
DB $00,$00,$3C,$3C,$42,$42,$80,$80
DB $8E,$8E,$82,$82,$42,$42,$3C,$3C
DB $00,$00,$82,$82,$82,$82,$82,$82
DB $FE,$FE,$82,$82,$82,$82,$82,$82
DB $00,$00,$FE,$FE,$10,$10,$10,$10
```

**Figura 3.4:** Datos de tiles.

Como se puede ver en la figura 3.4, la cabecera del archivo consiste de un comentario con información sobre este, como el tamaño de los tiles y su cantidad, mientras que la segunda sección incluye todos los datos que definen los 52 tiles que hemos exportado, precedidos por un etiqueta. Esta etiqueta lleva dos puntos dobles para poder usarse fuera del propio archivo.

`DB` es una directiva que le dice al ensamblador que en ese punto de la memoria debe emitir una serie de bytes con los valores que van a continuación. En este caso, cada fila representa un total de 8 bytes en hexadecimal.

Cada tile ocupa 16 bytes en memoria, los primeros 2 bytes representan la fila fila de 8 píxeles de más arriba del tile, los dos siguientes la segunda fila, y así hasta la última. Dado que la consola puede mostrar un máximo de 4 colores, cada píxel está representado por dos bits que forman el identificador del color de la paleta que le corresponde (00, 01, 10, 11). Para cada fila de 8 píxeles de un tile, los 8 bits del primer byte representan el bit bajo del color de cada píxel de esa fila, y los bits del segundo byte, el alto. El bit más bajo de cada byte va asociado al píxel de más a la derecha. La figura 3.5 muestra un ejemplo de representación de un tile.

Cada dos filas de 8 bytes del archivo que hemos creado representan un tile. Como hemos exportado 52 tiles, tenemos un total de 104 filas y 832 bytes.

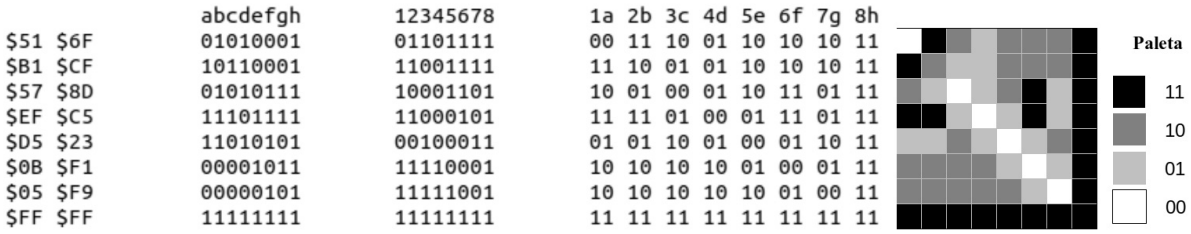



**Figura 3.5:** Representación binaria de un tile.

## 3.2 Cargar los tiles

En el capítulo anterior nos dedicamos a dibujar usando los tiles que venían cargados de base en la consola. Ha llegado la hora de guardar en memoria nuestros propios diseños de tiles para poder dibujar con más libertad. Si abres el visor de VRAM de BGB, y haces clic en la pestaña *Tiles*, podrás ver la sección de Tile Data.

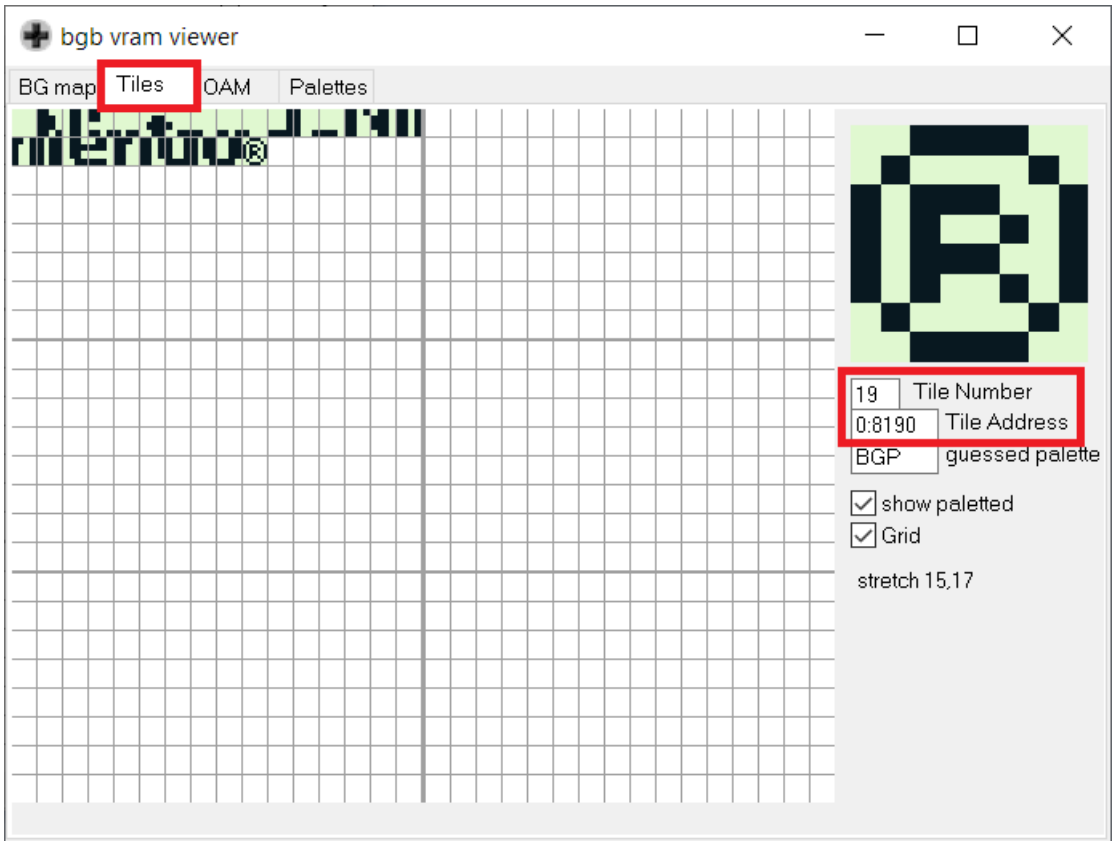



**Figura 3.6:** Tile Data

Como podrás ver en la figura 3.6, el número de tile que usábamos para referirnos a cada uno de ellos en el capítulo anterior depende de su posición en memoria. Por ejemplo, el tile de la R de marca registrada es el vigésimo sexto empezando por el principio, y está guardado en la posición de memoria $8190, por tanto le corresponde el número de tile $19 (25 en decimal). Esto significa que los 16 bytes que representan el tile están almacenados entre la dirección $8190 y $819F, ambas incluidas. Si queremos poder usar nuestros diseños, debemos escribir en esta región los datos de los tiles que vayamos a usar.

Verás que la mitad derecha de la sección tiene los mismos números de tiles y direcciones de memoria que la izquierda, siendo la única diferencia que antes de la dirección de memoria hay un 1 en vez de un 0. Esta mitad representa un segundo banco de memoria que tienen las consolas Game Boy Color, que este libro no cubre. También verás que aunque las direcciones de memoria del tercio inferior son distintas al tercio superior, ambos tienen los mismos números de tiles. Esto se debe a que con dos tercios ya se cubren los 256 posibles identificadores diferentes, por lo que la consola incluye una función que permite cambiar entre usar el tercio superior y el inferior, mientras que el tercio del medio siempre es accesible.

Vamos a escribir nuestros datos empezando desde el tile $80, o 128, que se encuentra en la dirección de memoria $8800. Para copiar todos los datos necesitamos:

- Guardar en un registro de 16 bits la dirección de origen de los datos (la etiqueta `Fuente`).
- Guardar en otro la dirección del primer byte de destino.
- Guardar en otro la cantidad de bytes a copiar (como son más de 256, necesitaremos un registro de 16 bits).
- Comprobar antes de copiar cada byte que la memoria de vídeo se encuentra accesible.
- Y lo más importante, tener cuidado de no sobrescribir ningún registro que contenga datos importantes para la ejecución de la función.

El código de la función y de la llamada a esta tendría la siguiente estructura:

```
ld r1, Fuente
ld r2, Destino
ld r3, número de bytes
call copia
```

```
copia:
esperar a memoria accesible
copiar un byte y aumentar registros puntero
si no era el último, volver al principio
si era el último, ret
```

El estándar para copiar datos de un lugar de memoria a otro es usar el registro HL como puntero a los datos de origen, el registro DE como el puntero de destino, y BC como contador si hay que copiar más de 256 bytes. Esto deja libre A para hacer la transferencia entre HL y DE, y para comprobar cuando está disponible la memoria de vídeo. Para esta última función ya deberías tener una rutina que hicimos cerca del final del capítulo anterior, simplemente introduce una llamada en la función de copia. Como siempre, no olvides comentar las funciones que crees indicando qué variables de entrada y salida tiene, así como los registros que se modifican al ejecutarse.

Ten en cuenta que la instrucción `dec BC` no modifica los flags, por lo que si quieres comprobar que el valor resultante de la operación es 0 tendrás que usar otros medios, como hacer *B or C*, que solo será 0 si tanto B como C son cero.

Si lo has hecho todo bien, si ejecutas el archivo ensamblado y abres la ventana de Tile Data deberías poder ver la fuente en memoria como en la figura 3.7.

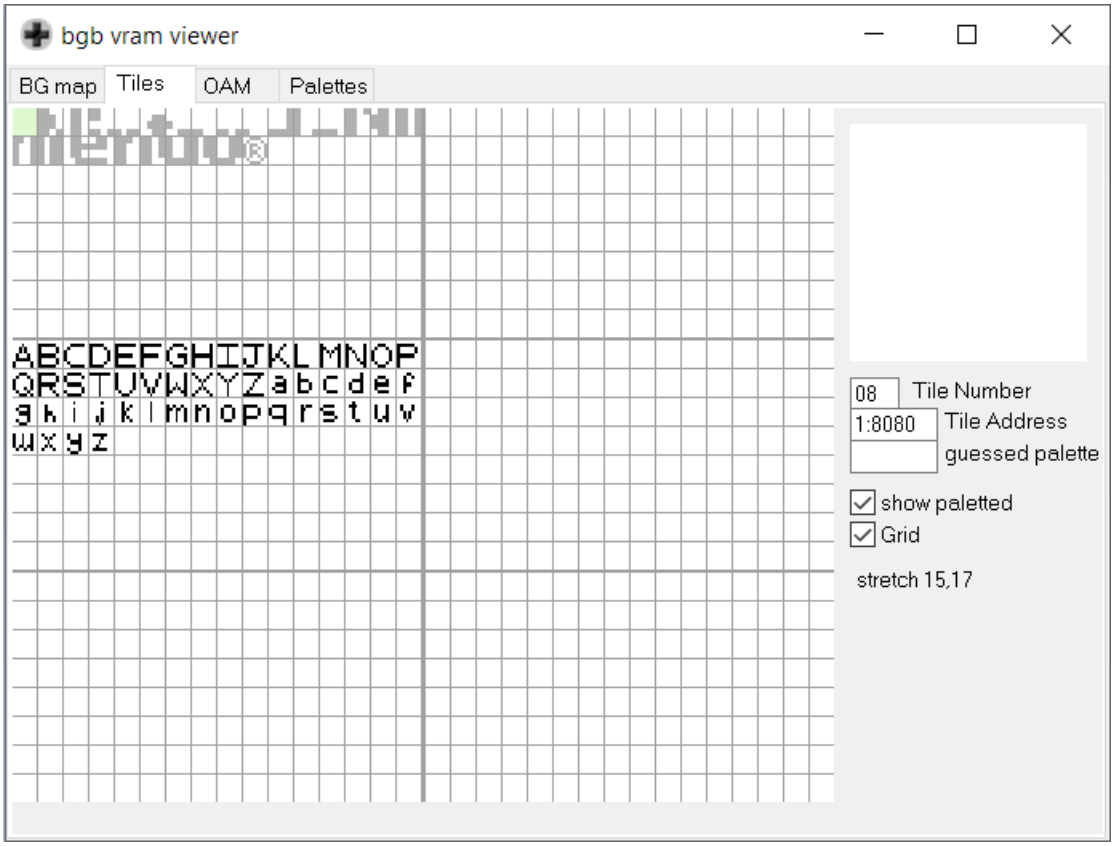



**Figura 3.7:** Fuente cargada en Tile Data

## 3.3 Guarda una línea en la ROM

Vamos a crear un nuevo archivo dónde estará el texto que escribiremos en la pantalla. Crea un archivo en la carpeta *src/res* y llámalo *texto.asm*. Empieza escribiendo en el archivo `SECTION "Texto", ROM0`. Esta directiva le dice al ensamblador en qué lugar de la ROM debe poner lo que va a continuación. Si quieres saber más acerca de las secciones en rgbasm, consulta el apéndice B.2.

Para guardar texto, podemos usar la directiva `DB` que hemos visto anteriormente, pero en vez de seguirla con bytes en hexadecimal, podemos escribir una cadena de texto entre comillas. Antes de ensamblar, esta cadena de texto se sustituirá por una serie de valores que dependen del charmap en uso.

Un charmap es una asociación que relaciona caracteres o cadenas de ellos con valores de 8 bits. El charmap por defecto se llama *main* y es equivalente a la codificación UTF-8 (las teras que ocupan más de un byte en esta codificación se expanden en varios bytes). Con este charmap, la directiva `DB "Hola mundo!"` Se converiría en `DB, 72, 111, 108, 98, 32, 109, 117, 110, 100, 111, 33`. Esto supone un problema porque nuestras letras no están en tiles que coincidan con estos números, por ejemplo, hemos puesto la H mayúscula en 135, no 72. Es por esto que necesitamos definir un charmap que se adecúe a nuestra disposición de la fuente en Tile Data.

Crea un nuevo charmap que sea una copia del charmap por defecto. Escribe al principio del archivo `NEWCHARMAP texto, main`. Esta directiva, además de crear el charmap *texto* como copia de *main*, también

cambia a él. Ahora tenemos que redefinir los caracteres. Debajo de la directiva que acabamos de escribir, escribe `CHARMAP "A", 128`, y sigue con el resto de letras, mayúsculas y minúsculas, una en cada línea. Escribe también una entrada para el carácter de espacio, al que le puedes dar el valor posterior a la z minúscula, que debería estar vacío. En este punto podrías volver a los diseños de tiles y añadir algunos caracteres más, como por ejemplo el símbolo de exclamación, u otros signos de puntuación, y añadir una línea para asignar cada uno de ellos al tile en el que se encontrarían.

Ahora puedes escribir dentro de la sección que has creado antes el siguiente código (excluye el signo de exclamación si no lo ahs creado):

```
linea::
DB "Hola mundo!", 0
```

Este fragmento se convertirá según el charmap que hemos definido, y podremos acceder a él desde otros archivos dónde estará el código que lo escriba gracias a la etiqueta que lo precede. El cero que sigue al texto servirá para poder identificar dónde termina la línea.

## 3.4 Escribe una línea

Ya podemos escribir nuestra primera línea de texto, simplemente hay que cargar la fuente en Tile Data como ya hemos hecho, y copiar los bytes que hemos definido en el fichero de texto sobre posiciones consecutivas de la memoria de vídeo, parando cuando lleguemos a un byte con valor de 0, que es el que hemos usado para marcar el final de la línea. El algoritmo quedaría así:

```
cargar fuente

ld hl, texto
ld de, memoria de vídeo
call escribeTexto
```

```
escribeTexto:
esperar a memoria accesible
carga byte en registro
si es 0, ret
si no, cópialo
aumenta punteros
repetir
```

Tras programar y ejecutar este código, obtenemos el resultado de la figura 3.8.

Podemos hacer un añadido con una nueva función específica para escribir diálogos, ya que podemos suponer que los diálogos siempre se escribirán en la misma posición, igual que lo hacen los juegos mostrados en los ejemplos 3.1b y 3.1a. Para esto sólo tenemos que mover la línea que carga la memoria de vídeo y la llamada a la función de escribir texto dentro de una nueva función, y sustituir por una llamada a esta las líneas eliminadas.

```
cargar fuente
```

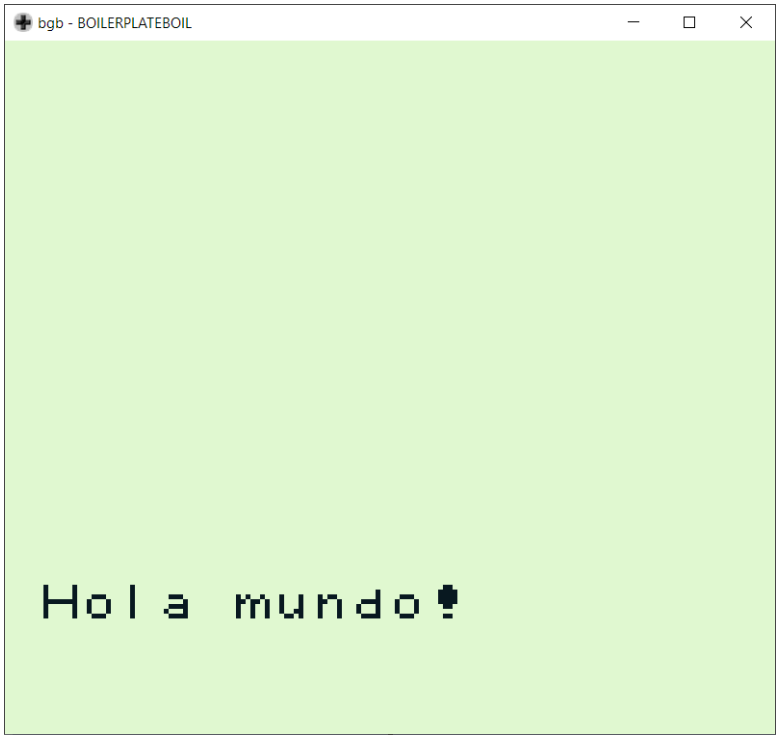



**Figura 3.8:** Línea de texto en la Game Boy

```
ld hl, texto
call escribeDialogo
```

```
escribeDialogo:
ld de, memoria de vídeo
call escribeTexto
ret
```

De esta forma seguimos pudiendo escribir texto en cualquier parte de la pantalla con la misma función, pero no hace falta que cada vez que queramos escribir un diálogo en el mismo sitio tengamos que volver a cargar la dirección de memoria de inicio.

## 3.5 Organicemos el código

A estas alturas tendrás muchas funciones definidas en el archivo *header.asm*, y muchas llamadas dentro del inicio de ejecución. Vamos a organizar las cosas para que sea más fácil encontrar cada función y añadir nuevo código.

Puedes empezar moviendo las funciones generales a un archivo a parte, que puedes llamar *utils.asm*. Encabeza el archivo con `SECTION "Utils", ROM0` y asegúrate que las funciones que defines aquí están encabezadas por una etiqueta con dos puntos dobles. Borrar el logo de Nintendo, las funciones de escribir en regiones de memoria, la que usamos para esperar a que la memoria de vídeo esté libre, y la de copiar datos de HL a DE, puedes ponerlas aquí, así como cualquier otra función de uso general que consideres, o que escribas más adelante. No olvides incluir el archivo *hardware.inc* en cualquier otro en el que uses las constantes definidas en él,

o el proceso de ensamblar fallará al haber símbolos desconocidos.

Este sería también un buen momento para crear una función de configuración inicial (llamada setup de ahora en adelante) desde dónde hacer todas las llamadas a otras funciones que se tienen que hacer siempre al principio de la ejecución. Por ahora esta función solo contendrá la llamada a borrar el logo de Nintendo, pero le añadiremos más cosas en el siguiente apartado. Deja esta función en el archivo inicial.

Por último, vamos a echar un vistazo a los llamados ”números mágicos”. Es muy probable que durante las secciones anteriores hayas escrito números directamente en el código, copiando valores a registros. Por ejemplo, probablemente a la hora de guardar un valor en BC para usarlo como contador de bytes que copiar al cargar la fuente, habrás hecho algo como:

```
ld bc, 864
```

El problema con esto es que a simple vista no se entiende de dónde viene este número, por qué ese y no otro, es un número que parece que sale de la nada, de ahí ”número mágico”. Cuando más adelante veas ese código, u otro en el que hayas hecho cosas similares, no sabrás por qué pusiste ese número en concreto ahí. Por eso el archivo *hardware.inc* tiene tantas constantes definidas, si las usaras todas ellas como su número equivalente no quedaría claro qué se está haciendo, pero si pones `_VRAM` en vez de `8000` sabes que estás escribiendo en memoria de vídeo.

Para arreglar esto, puedes crear tus propias constantes y usarlas a lo largo del código, y lo que es mejor, puedes operar con ellas, porque antes de ensamblar el preprocesador resuelve las operaciones. Por ejemplo, puedes cambiar el ejemplo anterior de la cantidad de bytes que copiar por lo siguiente:

```
DEF BYTES_PER_TILE EQU 16
DEF TILES_TOTAL EQU 54

ld bc, TILES_TOTAL*BYTES_PER_TILE
```

El código anterior define dos constantes. Si quieres saber más sobre cómo definir constantes y sus tipos, puedes ver el anexo B.1.2.

Ahora queda mucho más claro qué significa el número elegido. Puedes mover todas las definiciones de constantes al principio del archivo para que no molesten en medio del código, o incluso moverlas a un archivo distinto en incluirlas del mismo modo que *hardware.inc*.

Con esto el fichero inicial debería estar más organizado. De ahora en adelante procura seguir estas pautas, haz un repaso de lo que has escrito de vez en cuando para que el código no llegue a estar muy enrevesado.

## 3.6 Escribir más despacio

Ya podemos escribir cualquier línea formada por los caracteres que hemos diseñado, pero que aparezcan las líneas enteras instantáneamente una detrás de otra puede ser abrumador para algunos jugadores. Vamos a ver cómo esperar varios frames para ir escribiendo los caracteres poco a poco. Podríamos hacer esto con la función

que utilizamos para esperar a un momento adecuado para dibujar, pero comprobando si nos encontramos específicamente en un VBlank, el problema de esto es que hacer que la CPU esté haciendo cálculos tanto tiempo para simplemente esperar consume mucha energía que se puede ahorrar, lo cual debería ser una prioridad en una consola que funciona con pilas. Para esperar al final de un frame sin usar este método, vamos a ver cómo funcionan las interrupciones.

Una interrupción es una señal que puede recibir la CPU que le indica que debe interrumpir la ejecución del programa y ejecutar otro código. Una característica de la CPU de la Game Boy es la instrucción `halt`, que detiene la CPU hasta que se detecta una interrupción. Esta instrucción es justo lo que necesitamos para ahorrar energía mientras esperamos a que pasen un número concreto de frames.

Para atender a los distintos tipos de interrupciones, la consola dispone de los siguientes elementos:

- IME - Interrupt master enable flag: Este es un bit de solo escritura, no se puede acceder a él a través de ninguna dirección, es un flag propio de la CPU que indica si se deben atender las interrupciones.
- IE - Interrupt enable: Este es un byte accesible en la dirección $FFFF que controla qué tipo de interrupciones pueden generarse.
- IF - Interrupt flag: Este byte se encuentra en la dirección $FF0F y es modificado por el sistema según suceden peticiones de interrupción.

Las únicas formas de modificar IME son las instrucciones `ei`, que lo activa, `di`, que lo desactiva, y `reti`, que lo activa y ejecuta también un `ret`.

Existen 5 tipos de interrupciones que pueden habilitarse en IE, y como es obvio, se pueden recibir peticiones de los mismos 5 tipos en IF. El algoritmo que decide cuando se generan y atienden interrupciones es el siguiente:

- Se compara IE e IF, si un tipo de interrupción está habilitada en IE y se recibe una petición del mismo tipo en IF, se genera ese tipo de interrupción.
- Si IME está activado, y se genera una interrupción, se atiende este tipo de interrupción.
- Atender una interrupción implica poner a 0 en IF el flag del tipo de interrupción que se está atendiendo, desactivar IME, y se hace una llamada al controlador de la interrupción.

Este algoritmo está esquematizado en la figura 3.9.

Para nuestro objetivo, la interrupción que nos interesa es VBlank, que se activa cuando la PPU entra en el modo 1, y corresponde al bit 0 de IE e IF. Como ahora mismo sólo vamos a usar este tipo de interrupción no necesitaríamos complicar mucho el código para esperar al final de un frame, pero como puede que en el futuro queramos usar otros tipos es más conveniente escribir el algoritmo como si ya fuera así, en vez de tener que cambiarlo más adelante.

Vamos a empezar por crear una función que llamaremos `interruptSetup`, que invocaremos desde la función de configuración inicial.

```
interruptSetup:
ld a, %00000001
ld [rEI]   ;Habilita la interrupción VBlank y deshabilita el resto
reti       ;Retorna y activa IME
```

Ahora vamos a programar la rutina que se ejecutará al activarse la interrupción. En el caso de la interrup-

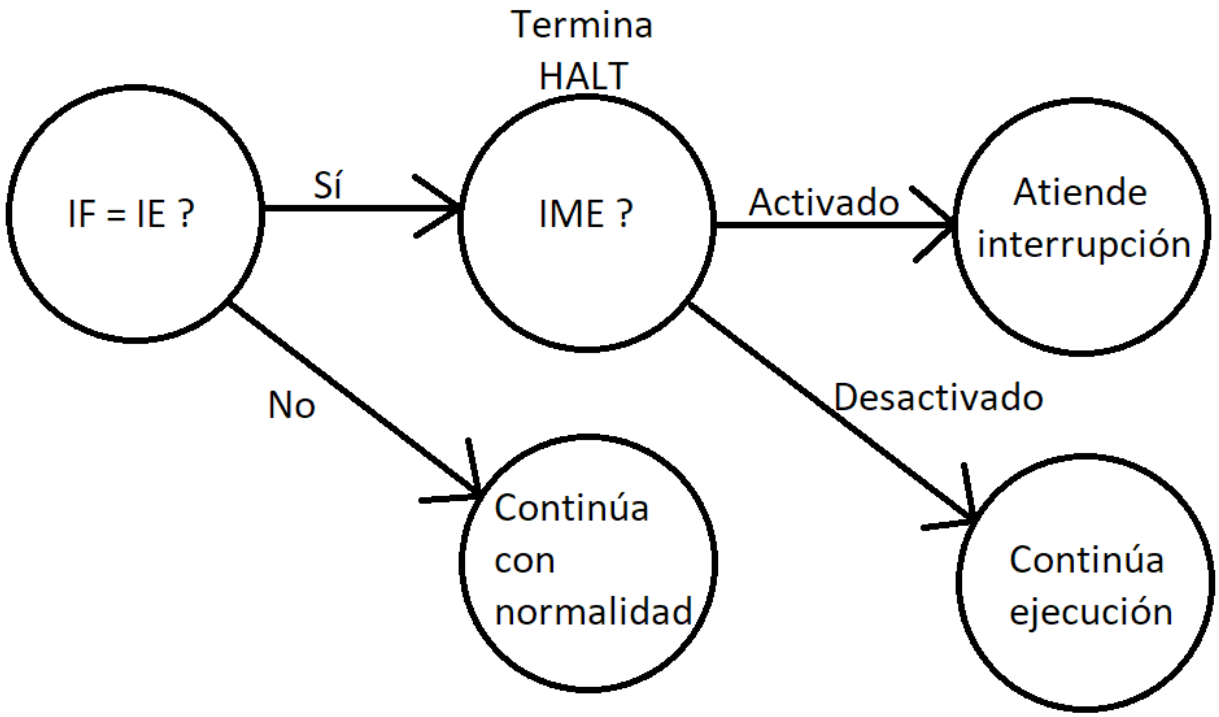



**Figura 3.9:** Algoritmo que procesa las interrupciones

ción VBlank, la dirección a la que se llama es $0040. Para asegurar que una sección de código se escribe en una región de memoria en concreto podemos aprovechar la directiva `SECTION` que precede a cada sección de código. Escribiendo `SECTION "Interrupts", ROM0[$40]` aseguramos que el código que se escriba a entre esta declaración de sección y la siguiente empiece en la dirección $0040. Puedes poner la directiva anterior encima de la de cabecera, ya que esta última va en la dirección $0100 que es posterior, y así el código queda en el mismo orden que en el que estará en el cartucho. Una cosa que debemos tener en cuenta, es que solo disponemos de 8 bytes de espacio para cada controlador de interrupción, si queremos ejecutar algo que no quepa en 8 bytes, deberemos escribir el código en otra parte de la memoria y saltar a ella, que es lo que vamos a tener que hacer, con lo que acabamos con la siguiente sección:

```
SECTION "Interrupts", ROM0[$40]
;$40 VBlank
jp vblankHandler
ds 5, 0   ;Deja 5 bytes de espacio vacíos
```

Además vamos a necesitar un byte de espacio que modificaremos desde el controlador de la interrupción para que cuando la CPU salga del estado HALT se pueda diferenciar qué tipo de interrupción es la que se ha lanzado. Para esto necesitamos definir un byte de RAM para este uso. Crea una nueva sección separada del resto e introduce lo siguiente:

```
SECTION "Variables", WRAM0
vblankFlag:
ds 1
```

`WRAM0` hace que la sección se asigne a la región de memoria "Work RAM", `ds 1` reservaün byte de espacio, de forma que si debajo de esta directiva definiera otra etiqueta, tendría el valor de `vblankFlag` más uno. Ten

en cuenta que como esta sección se corresponderá con RAM, no es posible escribir código aquí, ni emitir bytes con `DB`.

Ahora que tenemos un byte destinado a comprobar si la interrupción activada ha sido VBlank, ya podemos escribir su controlador. En primer lugar tenemos que escribir un valor distinto de 0 en este byte, el siguiente código es suficiente para ello:

```
   vblankHandler:
      ld hl, vblankFlag
      ld a, 1
      ld [hl], a
```

Pero debemos tener en cuenta que la interrupción puede haberse llamado en cualquier momento, desde cualquier parte del código, por lo que los registros podrían contener datos importantes. Para no perderlos, usaremos las instrucciones `push` y `pop`. Recodarás como la instrucción `call` copia en la pila el valor de PC, y `ret` lo recupera, en este caso se diría que `call` hace push de PC a la pila, y `ret` hace pop. Estas instrucciones permiten hacer lo mismo con el resto de registros de 16 bits, así que para guardar los registros HL y AF que se modifican en esta función, incluiremos lo siguiente:

```
   vblankHandler:
  push hl
  push af
  ld ld hl, vblankFlag
  ld a, 1
  ld [hl], a
  pop af
  pop hl
    reti
```

Por último `reti` regresa al punto de ejecución desde dónde se llamó a la interrupción y vuelve a activarlas.

Como notarás, el orden en el que se ejecutan las instrucciones pop es el inverso al orden en el que se ejecutan las push, ya que se quita primero de la pila lo último que se añadió. Es esencial emparejar cada push con un pop, a no ser que tengamos un motivo especial, pues si nos olvidamos de algún pop o añadimos alguno de más, cuando se ejecute el siguiente `ret` no se cogerán los dos bytes correspondientes al `call` correspondiente, sino otros dos distintos, lo cual provocará que el programa ya no siga ejecutando lo que pretendíamos. Este tipo de error obliga a reiniciar la consola, ya que a partir de ese momento la ejecución del programa escapa de nuestro control.

Lo último que nos queda por hacer es la función que espera hasta un VBlank.

```
waitVBlank:
   ld  hl, vblankFlag    ; hl = pointer to vblankFlag
   xor a                 ; a = 0
   ld  [hl], a           ; pone vblankFlag a 0
.wait
   halt
   cp  a, [hl]           ; si vblankFlag es 0
   jr  z, .wait          ; vuelve a esperar
```

```
ret
```

Si incluimos una llamada la función que acabamos de definir, las letras irán apareciendo una cada frame en vez de todas de golpe. Ten en cuenta los posibles registros que pueden verse modificados por cada función y que son importantes, e incluye instrucciones `push` y `pop` antes y después de las llamadas para no perder sus contenidos. Si incluyes comentarios antes de cada rutina indicando qué registros se modifican, será mucho más fácil comprobarlo.

Para finalizar, puedes crear una función que espere una cantidad de frames arbitraria, que recibiría en algún registro, y usarla para variar la velocidad a la que se escriben los textos.

## 3.7 Escribe múltiples líneas

Ya podemos escribir líneas en cualquier parte de la pantalla, eso sí, debemos tener cuidado de que la longitud de las líneas no superen el ancho de la pantalla, o el texto se saldrá por el borde e incluso aparecerá en la siguiente fila, como en la imagen 3.10.

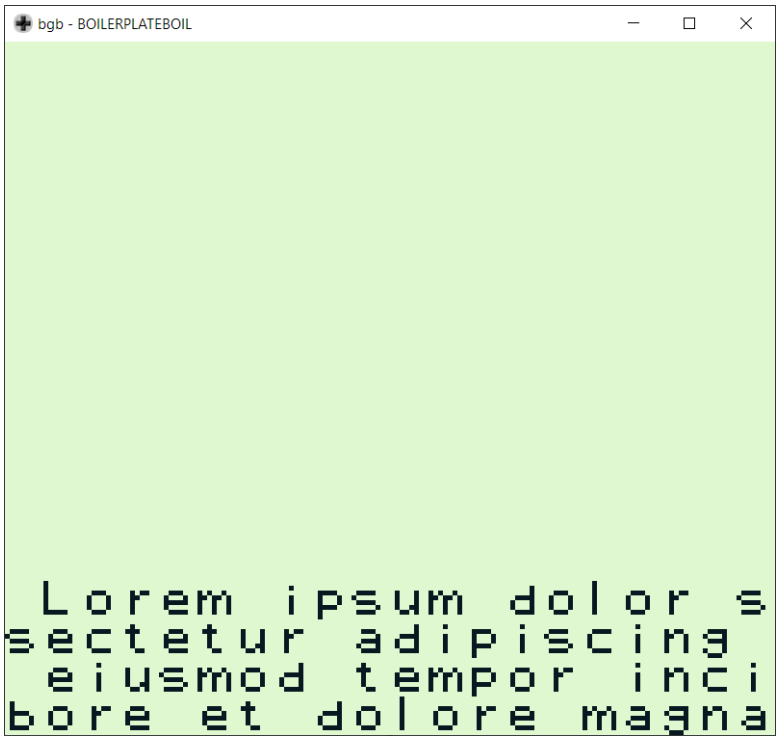



**Figura 3.10:** Texto saliéndose de los bordes

Si queremos escribir textos más largos en varias líneas sin tener que hacer varias llamadas a la misma función, vamos a tener que modificar las funciones que ya tenemos y el modo en que se codifica el texto.

Para empezar, seguiremos indicando el final del texto con un 0, pero marcaremos con un 10 el final de cada línea del mismo texto. El diálogo inicial con Oak en Pokémon rojo y azul se guardaría de la siguiente forma:

```
TextoPokemon::
```

```
DB "¡Hola a todos!", 10
DB "¡Bienvenidos al", 10
DB "mundo de POKEMON!", 10
DB "¡Me llamo OAK!", 10
DB "¡Pero la gente me", 10
DB "llama el PROFESOR", 10
DB "POKEMON!", 0
```

Si ahora añadimos una comprobación en la función de escribir texto para que llame a otra función que prepare la escritura de la siguiente línea, podremos escribir textos tan largos como queramos. La nueva función debería borrar la línea actual de texto, volver a cargar el puntero a la dirección de memoria de vídeo dónde empieza la línea, y cualquier otro detalle que consideres necesario. El resultado en la pantalla tras dibujar todas las líneas quedaría parecido a la imagen 3.11.




**Figura 3.11:** Última línea de un texto.

Igual te has dado cuenta de que no he creado ninguna letra con acento todavía, eres libre de añadirlas si quieres y te parece que quedaría mejor.

Ya casi tenemos un módulo que escriba diálogos de forma similar a los que aparecen en juegos como Pokémon o Zelda, lo único que faltaría (además de poder controlar el avance de las líneas con los botones y que no sea automático) es hacer que el texto se vaya desplazando, al menos hasta ver dos líneas, y también que se borre al final. Esto podemos hacerlo usando dos direcciones de inicio de líneas, usando la primera en la llamada original a la función, y usando la función de configuración de nueva línea para cambiar a la segunda. Además en esta misma función podemos incluir la rutina que haga el desplazamiento del texto. Para desplazar el texto basta con copiar tres líneas de abajo a arriba de la forma que muestra la figura 3.12. Como la posición de la segunda línea depende de la primera, tenemos dos opciones: calcular la dirección de inicio de la segunda línea

si queremos que la función pueda usarse para escribir diálogos en cualquier parte de la pantalla, o suponer que todos los diálogos que mostremos van a estar siempre en la misma posición y poner la dirección de la segunda línea directamente. Puedes elegir la opción que quieras.




**Figura 3.12:** Desplazamiento de texto.

Este desplazamiento sólo debe realizarse a partir de la segunda línea, si lo hacemos cuando sólo se ha escrito la primera la haremos desaparecer y dará la impresión de que hay una línea en blanco en medio del texto. Para controlar si es la primera línea o no, puedes usar un byte de RAM al igual que hicimos para las interrupciones, o usar algún registro si tienes alguno libre (o puedes guardarlo en la pila). La idea sería guardar un valor en el registro o byte antes de llamar a la función para escribir un diálogo, y comprobar ese valor cuando se configure la siguiente línea, si es el inicial habría que cambiarlo y no hacer el desplazamiento, si es distinto se desplazaría el texto.

Para hacer que el texto se borre, basta con introducir una función que podríamos llamar cleanup a la que

llamar después de escribir el texto completo. Tras todo esto, podemos mover todo el código a un archivo distinto para no saturar el archivo inicial, si lo llenamos de demasiadas funciones no relacionadas entre sí será difícil encontrar lo que queramos más adelante. Ya que esta sección consiste en crear un sistema de diálogos, podemos crear una nueva carpeta *sys* dentro de *scr* en la que guardar el código de todos los sistemas, incluido este. En windows tendrás que crear carpetas del mismo nombre en los directorios *dep* y *obj*.

## 3.8 Control por botones

El paso final para tener un módulo de diálogos funcional, es poder controlar el avance de estos con los botones, que el jugador pueda decidir cuando avanzar el texto. En esta sección aprenderemos a leer y guardar el estado de los botones.

Los botones están dispuestos en una matriz 2x4, y su lectura se hace a través del byte $FF00, cuyos bits cumplen las siguientes funciones:

- Bit 7: Sin uso.
- Bit 6: Sin uso.
- Bit 5: Seleccionar botones. (0 = seleccionados; 1 = sin seleccionar)
- Bit 4: Seleccionar direcciones. (0 = seleccionados; 1 = sin seleccionar)
- Bit 3: Start/Abajo. (0 = pulsado; 1 = no pulsado)
- Bit 2: Select/Arriba. (0 = pulsado; 1 = no pulsado)
- Bit 1: B/Izquierda. (0 = pulsado; 1 = no pulsado)
- Bit 0: A/Derecha. (0 = pulsado; 1 = no pulsado)

Si no se selecciona ni los botones ni las direcciones (los bits 4 y 5 son ambos 1), entonces todos los bits inferiores tendrán el valor de no pulsados. Si se seleccionan ambos a la vez, cada bit se activará si cualquiera de los botones asignados a él está pulsado. El nibble bajo de este byte (los bits que indican qué botones están pulsados) es de solo lectura, no puede modificarse manualmente por el código.

Además de guardar el estado de los botones, también queremos saber cuáles han pasado de no estar pulsados a estarlo en el frame que hacemos la lectura, ya que en algunos juegos, como uno de plataformas, hay acciones que querríamos que sucedieran sólo si el jugador pulsa el botón, y no si lo mantiene, como por ejemplo saltar cuando estás tocando el suelo. Si un jugador pulsa el botón de saltar estando en el suelo, y lo mantiene hasta que termina el salto, no querríamos que volviera a efectuar un salto inmediatamente, sino que debería soltar el botón y volver a pulsarlo. Para obtener qué botones han cambiado de no estar pulsados a estarlo, necesitamos conocer el su estado actual y el anterior. Suponiendo que los hemos almacenado en bytes de forma que un 1 indique que el botón está pulsado, y un 0 que no, podemos usar la función 3.1 para obtener qué botones han pasado a estar pulsados, también llamado flanco ascendente.

$$Flanco = (AnteriorXORActual)ANDActual \tag{3.1}$$

El siguiente código es una función típica de lectura de los botones:

```
;Registros modificados: A, F, B
leerBotones::
    ld a, $20
    ldh [rP1], a  ;Selecciona direcciones
    ldh a, [rP1]  ;
    ldh a, [rP1]  ;Espera algunos ciclos
    ldh a, [rP1]  ;Guarda direcciones en a
    cpl           ;Invierte bits (1 = seleccionado; 0 = no seleccionado)
    and $0F       ;Borra los bits que no indican el estado de los botones
    swap a        ;Mueve los bits al nibble superior
    ld b, a       ;Guarda el resultado

    ld a, $10     ;Repite para los botones
    ldh [rP1], a
    ldh a, [rP1]
    ldh a, [rP1]
    ldh a, [rP1]
    cpl
    and $0F

    or b          ;Recupera el estado de las direcciones
                  ;En este punto A contiene el estado de los botones
                  ;A = [D|U|L|R|St|Se|B|A]
    ld b, a
    ldh a, [estadoBotones] ;Recupera el estado en el frame anterior
    xor b
    and b                ;Obtiene el flanco ascendente en A
    ldh [flancoAscendente], a
    ld a, b
    ldh [estadoBotones], a

    ld a, $30
    ldh [rP1], a  ;Deselecciona todos los botones
ret
```

La instrucción `ldh [d16], a` realiza la misma operación que `ld [d16], a`, con la diferencia de que ocupa un byte menos y tarda un ciclo menos en ejecutarse. Su único inconveniente es que la dirección `d16` debe ser una dirección mayor que $FF00. Como HRAM empieza en $FF80, las etiquetas dónde se guarda el estado de los botones se definen de la siguiente manera:

```
SECTION "Input variables", HRAM

estadoBotones::
ds 1
flancoAscendente::
ds 1
```

Dado que podríamos necesitar el estado de los botones en cualquier frame, podemos incluir una llamada a esta función a la rutina de interrupción VBlank, que sucede una vez por frame. En caso de necesitar leer los botones en alguna situación en la que no sucedan interrupciones, podemos hacer llamadas a la función en aquellas partes que sea necesario. Ya podemos saber el estado de los botones leyendo de `estadoBotones` y `flancoAscendente`.

Lo que necesitamos hacer ahora para que la función de escribir diálogos espere a que se pulse un botón (por ejemplo el A) antes de avanzar el texto es incluir el siguiente bucle en la función de configuración de nueva línea:

```
waitApressed::
    ldh a, [flancoAscendente]
    cp PADF_A
    ret z
    halt
jr waitApressed
```

También hará falta hacer una llamada a esta función antes de borrar el texto.

## 3.9 Otras mejoras

Ya tienes un sistema de escritura perfectamente funcional, y podrías avanzar al siguiente capítulo para proseguir con otro apartado del desarrollo, pero aún hay cosas que se pueden mejorar en este apartado. Eres libre de avanzar a otra sección y volver más tarde para revisitar estas opciones que se te presentan. Entre los aspectos a mejorar se encuentran:

- Encuadrar el texto con un marco para que quede más bonito. No sería muy complicado.
- Hacer la escritura sobre la ventana para no escribir sobre el fondo. Es un cambio sencillo, que solucionaría un problema que tiene nuestro algoritmo actual, pero que aborda varios conceptos nuevos.
- No cargar la fuente entera en Tile Data para ahorrar memoria. Haría que el espacio necesario de Tile Data no dependiera de la cantidad de caracteres distintos en nuestra fuente, sino que sería constante, sería necesario alterar la función de desplazamiento de texto.
- Añadir más funciones a los botones. Además de hacer avanzar el texto cuando ha terminado de escribirse una línea, se puede hacer que pulsar el botón A mientras se está escribiendo una línea haga que termine de escribirse inmediatamente, o que mantener pulsado el botón B haga avanzar el texto sin necesidad de pulsar A.

## 3.10 Usando la ventana

De las opciones anteriores, mover los textos a la ventana es probablemente la más útil. Ahora mismo, si escribiéramos el texto sobre un fondo de algún escenario, tras terminar de escribirlo tendríamos que volver a dibujar el fondo. Usando la ventana, podemos mostrar el texto sin modificar el fondo.

Hasta ahora has estado dibujando todos los gráficos en una región de memoria entre $9800 y $9BFF. Esta región es un Tile Map, un conjunto de datos que representan tiles para formar una imagen más grande, pero no es el único del que dispone la Game Boy, en la región de $9C00 a $9FFF hay otro Tile Map más. Podemos seleccionar cualquiera de los dos Tile Maps para representar el fondo, y podemos usar el que no se asigne a este para representar la ventana, un objeto gráfico que se puede superponer al fondo.

Usando esta segunda región, podemos escribir ahí los textos, y usar la ventana para superponerlos al fondo sin borrar este. Una vez se termine de escribir, solo tenemos que desactivar la ventana y el fondo volverá a verse.

Para activar la ventana, tenemos que escribir un 1 en el bit 5 del registro de hardware LCDC, para desactivarla, basta con escribir un 0. Si aún no has visto nada del capítulo siguiente, solo debes saber que LCDC se encuentra en la dirección $FF40. Una vez activada, la ventana se mostrará según indiquen los registros WY y WX, en las direcciones $FF4A y $FF4B respectivamente. WY indica el píxel verical de la pantalla en el que empieza la ventana, y WX el horizontal más 7. El píxel más arriba a la izquierda de la pantalla corresponde a unos valores de WY y WX de 0 y 7. La región del Tile Map asociado a la ventana que se muestra corresponde al rectángulo que empieza en el píxel superior izquierdo del Tile Map y que acaba en el píxel más abajo a la derecha que quepa en la pantalla. La imagen 3.13 muestra un ejemplo de la ventana. WY y WX tienen ambos el valor 50, la ventana contiene un tile distinto al fondo.

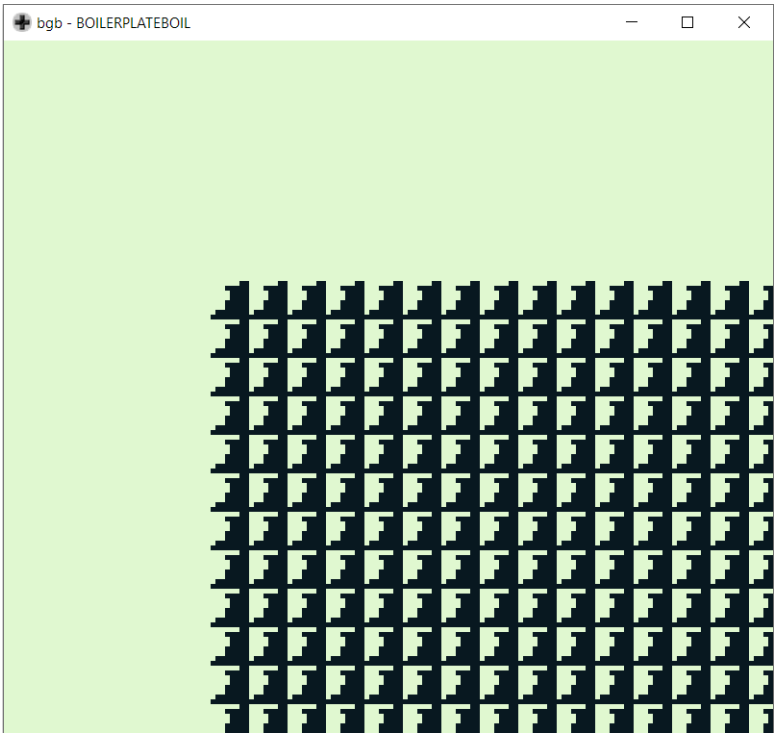



**Figura 3.13:** Ventana sobre fondo

Antes de dibujar la ventana, tenemos que asignarle el segundo Tile Map, poniendo a 1 el bit 6 de LCDC. Entonces solo tenemos que activar la ventana al iniciar la escritura del texto, poniendo los valores de WY y WX para que coincidan con la posición anterior del texto, cambiar las direcciones donde se escriben las líneas a las del segundo Tile Map, que deberían ser $9C21 para la primera línea y $9C61 para la segunda, y desactivar la ventana al terminar.

# Capítulo 4 Minijuego

Es el momento de crear tu primer juego. Al final de este capítulo tendrás un juego sencillo en el que el jugador deberá controlar a un personaje para que huya de un enemigo que se moverá por la pantalla. Esto servirá como ejercicio para crear un primer motor de juego sencillo sobre el que construiremos el juego final.

## 4.1 Diseñar personajes

Empecemos diseñando dos personajes en GBTD, uno para el personaje del jugador y otro para la entidad enemiga. Vamos a hacer diseños más grandes que para las letras para que tengan más detalle. Abre el programa y selecciona un tamaño más grande haciendo clic en *View*, luego en *Tile size* y por último en *16 x 16*, como se muestra en la imagen 4.1.

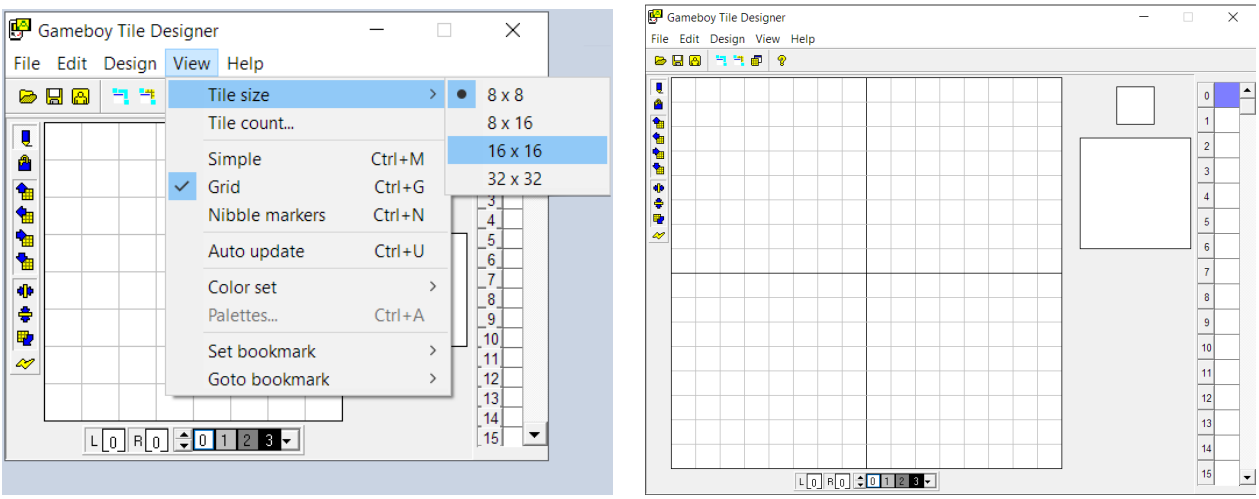



**Figura 4.1:** Crear lienzo de 16 por 16 píxeles

Puedes diseñar el personaje como quieras usando el espacio completo, pero debes tener en cuenta que, por motivos que se explican más adelante, el color 0 de la paleta (debajo del lienzo) será transparente, y en vez de verse ese color se verá el fondo. Puedes cambiar cualquiera de los colores de la paleta seleccionándolo con clic izquierdo y después haciendo clic en las flechas para elegir entre los 4 colores diferentes. Si recuerdas de secciones anteriores, cada píxel se representa por dos bits que indican el color, pero este color no representa un tono concreto de gris de los 4 disponibles. Al pintar en GBTD, estás asignando un número del 0 al 3 a cada píxel, según el color que tengas seleccionado, por eso si cambias el color asociado a un número, cambian todos los píxeles de ese número. Como dibujar con números no es práctico, GBTD te deja configurar una paleta para ver los diseños más claramente, pero cuando exportas los tiles, esa paleta no se exporta, sólo se exportan los números. Tú debes tomar nota de cómo es la paleta para configurarla desde el código. Por ejemplo, en los ejemplos de la figura 4.2 puede verse el mismo diseño con dos paletas distintas. Aunque en el editor se vea con colores distintos, cuando se exporten ambos archivos serán exactamente iguales, es nuestra responsabilidad crear y asociar las paletas desde el código.

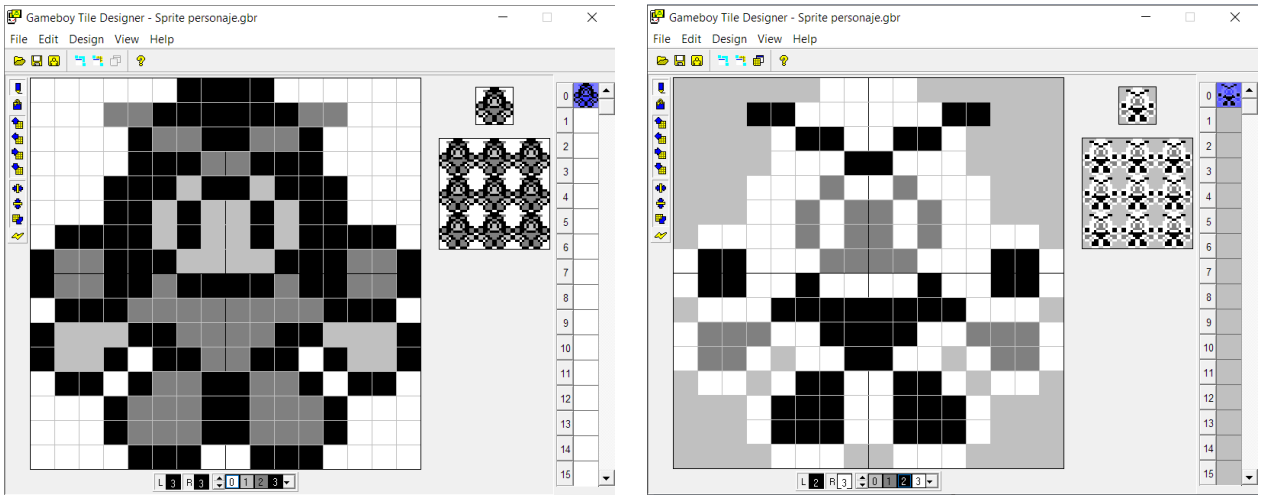



**Figura 4.2:** Mismo diseño con dos paletas distintas.

Cuando tengas el diseño del personaje terminado, guárdalo y expórtalo como ya hicimos con la fuente. Si abres el archivo generado, verás que hay 8 filas de 8 bytes definidas, porque nuestro diseño es tan grande como 4 tiles de 8x8 píxeles. El tile se exporta siguiendo el orden que muestra la figura 4.3.

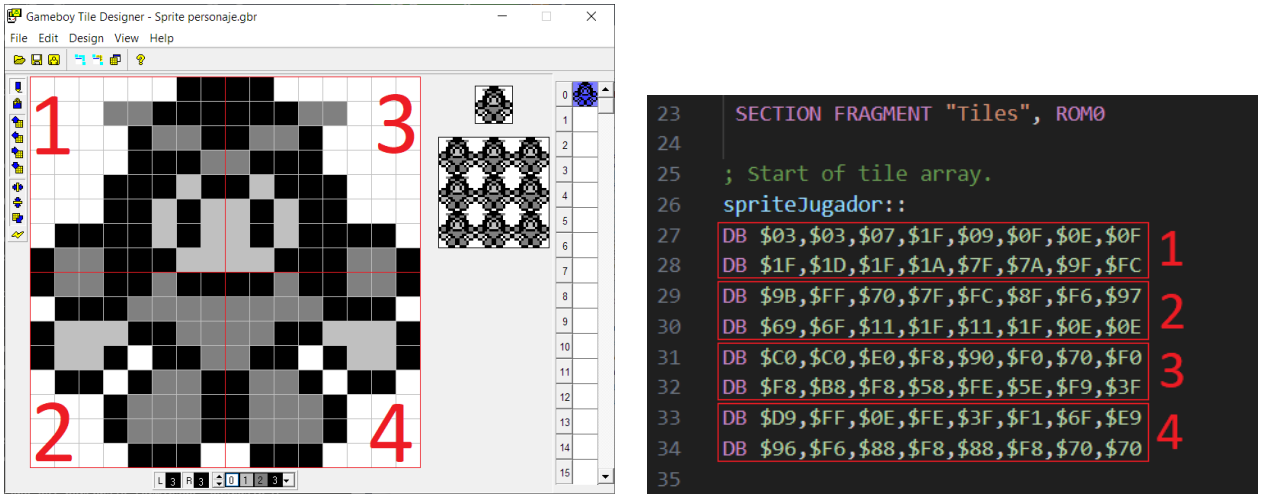



**Figura 4.3:** Asociación de bytes y tiles.

Es decir, las dos primeras filas corresponden al tile de arriba a la izquierda, las siguientes dos al de abajo a la izquierda, las siguientes al de arriba a la derecha, y las últimas al de abajo a la derecha. Esto tiene ciertas ventajas al dibujar sprites, pero otras desventajas al tratar con otro objeto llamado metatiles. Veremos ambos casos más adelante.

Exporta también el diseño del enemigo y añade ambos archivos al proyecto, recuerda cambiar la extensión a *asm* para que ensamblen. Ahora puedes intentar ensamblar un nuevo programa, y si lo haces deberías obtener un error del linker, diciendo que la sección ”Tiles”tiene múltiples definiciones. RGBASM no nos deja definir dos secciones distintas con el mismo nombre, y la sección por defecto que usa GBTD al exportar se llama ”Tiles”. Tenemos varias formas de solucionar este error, la primera es cambiar el nombre por defecto cada vez que exportemos, de forma que cada grupo de diseños esté en una sección, pero esto hará que los datos de los tiles puedan estar separados en la ROM, y ya que son datos del mismo estilo nos gustaría que estuvieran juntos para poder copiarlos en Tile Data seguidos. La segunda opción es combinar los archivos y copiar las definiciones de

los tiles a un mismo archivo para que estén en una sola sección, de esta forma nos aseguramos de que estén juntos, y podemos agrupar los tiles según tipo si lo combinamos con la solución anterior de renombrar secciones, separando la fuente de los tiles de personajes. La tercera opción es usar fragmentos de secciones. RGBASM permite definir una sección como fragmento añadiendo **FRAGMENT** después de **SECCTION**. Esto hace que todas las secciones definidas con el mismo nombre y sean fragmentos se posicionen juntas, pero no puede haber una sección con ese nombre y no ser un fragmento. Puedes optar por la opción que prefieras, en mi caso he elegido la segunda, separando en dos secciones distintas la fuente y los personajes.

## 4.2 Dibujar el primer sprite

Por fin puedes dibujar el primer sprite, la ventaja de los sprites respecto al fondo es que puedes ponerlos en cualquier píxel, sin tener que estar alineados de 8 en 8, y pueden ser movidos independientemente del fondo.

Empieza por cargar el sprite del jugador en la dirección $8000, correspondiente al tile de índice 0. Carga los 4 tiles que lo forman, no solo el primero. Si lo haces correctamente, todo el fondo debería haber cambiado a ser el primer tile, como en la imagen 4.4, ya que por defecto esta sección de VRAM contiene el valor 0 en todos sus bytes.

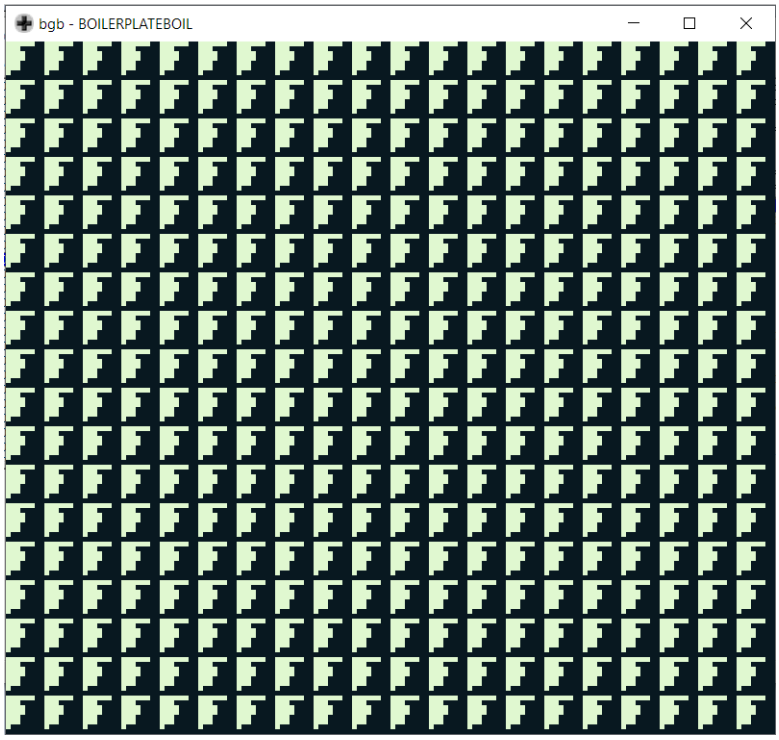



**Figura 4.4:** Fondo con nuevo diseño.

Vamos a arreglar esto cambiando el fondo de la pantalla. El tile de fondo pasará de ser el $00 al $7F, y de ahora en adelante ese tile se quedará vacío. Debes escribir $7F en todas las direcciones de memoria desde $9800 a $9FFF, un total de 2048 bytes. Si no tienes ninguna función que pueda copiar un mismo valor tantas veces, créala ahora. El cambio del fondo debería hacerse en el setup inicial, además esto deja obsoleta la función de borrar el logo de Nintendo, ya que tienes que escribir sobre esa región de memoria también. Como sabrás, toda esa región pertenece a la memoria de vídeo, por lo que no puede escribirse sobre ella en cualquier momento si la

unidad de procesado de imagen está activa, pero podemos desactivarla mientras dibujamos y volver a activarla después.

La Game Boy tiene un registro de control de la pantalla, llamado LCDC (LCD Control), direccionado en $FF40. Con él se pueden controlar varios aspectos del dibujado, pero por ahora sólo nos interesan la activación de la pantalla y los objetos. El bit 7 del registro sirve para activar y desactivar la pantalla, si se guarda un 0 en este bit, la PPU deja de dibujar y la pantalla se queda en blanco. MUCHO CUIDADO, cambiar este bit de 1 a 0 fuera del periodo VBlank (modo 1 de la PPU) puede provocar daños irreparables a la pantalla de la consola, si vas a probar la ROM en una consola de verdad, y no en un emulador asegúrate de que cualquier instrucción que haga esto se vaya a ejecutar siempre durante un VBlank. Mientras la pantalla está apagada, como la PPU no está dibujando no es necesario comprobar si la memoria de vídeo es accesible.

Espera a un VBlank (como es una sola vez puedes usar el método de esperar a la línea 144) y empieza poniendo un 0 en el bit 7 sin modificar los demás, puedes usar una máscara de bits, o usar la instrucción `res`. Después escribe $7F en toda la región de memoria del fondo, y vuelve a activar la pantalla escribiendo un 1 en el bit 7. Ahora la pantalla debería seguir apareciendo vacía incluso después de cargar el sprite del jugador en el índice 0.

Vamos a dibujar nuestro primer sprite. Los sprites son la representación gráfica de objetos, un elemento aparte del fondo, y que se puede posicionar en la pantalla independiente de este. Para dibujar un sprite necesitamos dos cosas, la primera es activar el dibujado de objetos en LCDC, poniendo un 1 en el bit 1, y modificando la región de memoria OAM con los datos necesarios para el sprite que queremos dibujar.

OAM es una región de memoria de 160 bytes, ubicada en las direcciones $FE00 a $FE9F, y puede representar hasta un máximo de 40 objetos distintos, usando 4 bytes cada uno. Estos 4 bytes representan la información siguiente para cada objeto:

- Byte 0 - Posición Y: Representa la posición vertical del píxel superior izquierdo del sprite. La posición en la pantalla del sprite será el valor de este byte menos 16, si este byte vale 16, el sprite estará alineado con el borde superior. Es posible ver únicamente una parte del sprite si el valor es inferior a 16, por ejemplo si valiera 15, la primera línea del sprite estaría oculta por el borde superior de la pantalla. Puedes asegurar que un objeto no se dibujará escribiendo un 0 en este byte. Si guardas un valor de 160, el objeto estará oculto por la parte inferior de la pantalla.
- Byte 1 - Posición X: Representa la posición horizontal del píxel superior izquierdo del sprite. La posición en pantalla del sprite será el valor de este byte menos 8, si este byte vale 8, el sprite estará alineado con el borde izquierdo de la pantalla, si vale 7, la columna de píxeles izquierda del sprite estará oculta por el borde de la pantalla. Puedes asegurar que un sprite no se vea escribiendo un 0 en este byte, sin embargo es mejor ocultar los sprites verticalmente, usando el byte 0, dedicado a la coordenada Y por motivos que se explicarán más adelante.
- Byte 2 - Índice del tile: Este byte selecciona qué tile de Tile Data se asocia al objeto y se dibujará en la posición indicada por los dos bytes anteriores. El valor de este byte se correlaciona con uno de los tiles entre $8000 y $8FFF. No hay forma de usar un tile guardado en el último tercio de Tile Data ($9000 a $97FF) como sprite.
- Byte 3 - Atributos: Este byte indica una serie de características asociadas al sprite. Estas son:

- Bit 7 - Prioridad: 0 - El sprite se pinta sobre el fondo. 1 - Los colores 1 a 3 del fondo se pintan sobre el sprite.
- Bit 6 - Orientación Y: 0 - Normal; 1 - El sprite se refleja verticalmente.
- Bit 5 - Orientación X: 0 - Normal; 1 - El sprite se refleja horizontalmente.
- Bit 4 - Paleta DMG, sólo para Game Boy original o Game Boy Color en modo GB: 0 - OBP0; 1 - OBP1.
- Bit 3 - Banco, sólo para Game Boy Color: 0 - Banco 0 de VRAM; 1 - Banco 1 de VRAM.
- Bits 2 a 0 - PAleta CGB, sólo para Game Boy Color: Elige entre OBP0 a OBP7.

Prueba a dibujar el tile superior izquierdo del personaje en la esquina superior izquierda de la pantalla, separado un píxel de arriba y de la izquierda del borde. OAM forma parte de la memoria que puede no ser accesible debido a la PPU, por lo que tendrás que esperar a que esté libre. El resultado debería ser el de la figura 4.5.

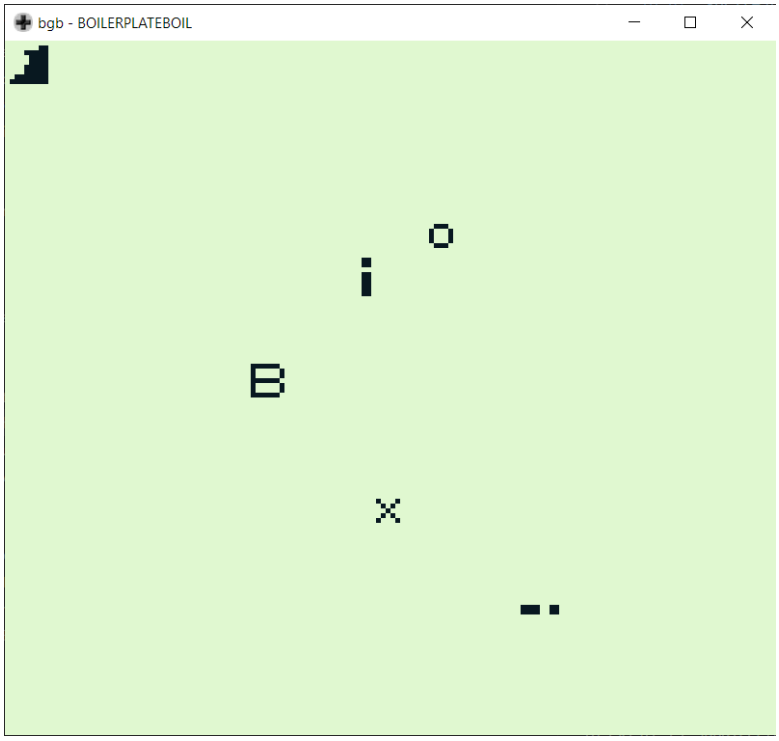



**Figura 4.5:** Sprites dibujados sobre el fondo

Mirando la figura, y el resultado que obtengas tú, notarás dos cosas. La primera es que hay varios otros sprites dibujados sobre la pantalla, y la segunda es que nuestro sprite no tiene los colores que esperábamos. El primer problema puede arreglarse añadiendo un borrado de OAM a la función en la que cambiábamos la pantalla, pero poniendo todo a 0. De esta forma inicialmente todos los sprites estarán fuera de la pantalla en el borde superior izquierdo. Hazlo ahora y comprueba que tras ejecutar ese programa sólo queda nuestro sprite en la pantalla.

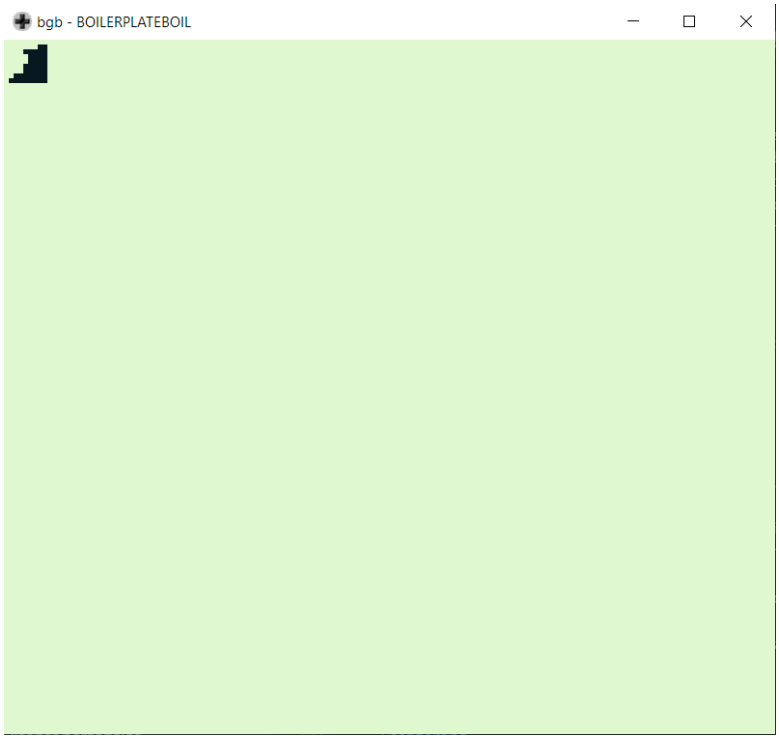



**Figura 4.6:** Único sprite en la esquina superior izquierda.

## 4.3 Paletas y sprites más grandes

Como hemos mencionado anteriormente, el sprite que has dibujado se ve completamente negro, sin los colores que pusiste en GBTD, y como puede que recuerdes, los colores que asocies a cada píxel no es lo único responsable del que muestren una vez dibujados, también está la paleta de colores, que asocia a cada número del 0 al 3 un color de los cuatro disponibles. Al diseñar en GBTD, eliges qué número va en cada píxel, pero no el color, estos sólo están ahí para ayudar al diseñar los gráficos. En los bytes de atributos de OAM, habrás visto que hay algunos bits dedicados a la paleta. Como estamos usando una Game Boy clásica, sólo nos interesa el bit 4, los bits 2 a 0 no tienen utilidad. Este bit selecciona entre la paleta OBP0 si vale 0, y la paleta OBP1 si vale 1. Estas paletas están en los registros $FF48 y $FF49 respectivamente. Cada sprite puede usar cualquiera de estas paletas independientemente del resto de sprites. Como por defecto escribimos ceros en toda la región OAM, todos los sprites usan la paleta 0 por defecto, pero no deberíamos suponer que va a ser así siempre, por lo que además de introducir las coordenadas y el identificador del tile, escribe siempre en el byte de atributos aquellos que necesites, que en este caso será todo 0. Además de las dos paletas para los objetos, también hay una para el fondo que no hemos llegado a ver, y se encuentra en el registro $FF47.

Las paletas funcionan del siguiente modo:

- Bits 7 y 6: Color para número 3.
- Bits 5 y 4: Color para número 2.
- Bits 3 y 2: Color para número 1.
- Bits 1 y 0: Color para número 0.

Cada par de bits elige un color que dar a los números que se eligieron al diseñar. Para cada dos bits, el valor a guardar según el color es:

- 00: Blanco.

- 01: Gris claro.
- 10: Gris oscuro.
- 11: Negro.

La única excepción, es que en los sprites, el color para el número 0 es irrelevante, pues será siempre transparente. Si no lo fuera, el contorno de todos los objetos sería rectangular, dejando un color para transparencias se deja ver el fondo y se pueden dar formas a las entidades.

Guarda la paleta, que en nuestro ejemplo correspondería con `%11100100` en OBP0. Haz esto justo antes de la activación de objetos en LCDC, puedes poner ambas acciones en una función de configuración de sprites. Si lo has hecho todo bien, ahora el sprite tendrá colores distintos para cada zona según lo diseñaste, como en la figura 4.7.

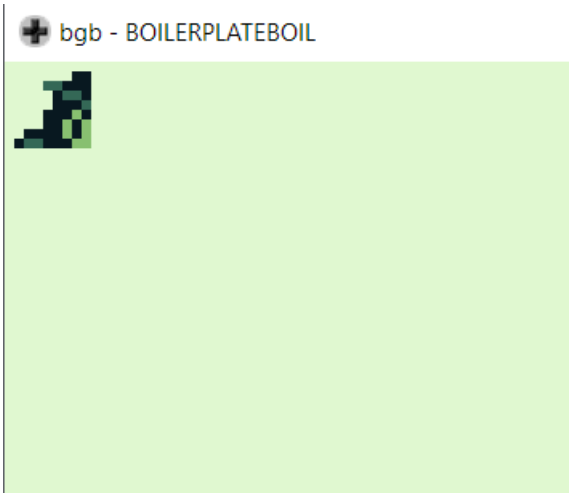



**Figura 4.7:** Único sprite en la esquina superior izquierda, con paleta correcta.

Ya tenemos un sprite de 8x8 píxeles dibujado, pero nuestro personaje ocupa un total de 16x16 píxeles. Para terminar de pintarlo podríamos dibujar los otros 3 tiles, pero esto ocuparía 4 de los 40 espacios para objetos de OAM, y podemos hacer que ocupe la mitad, suponiendo que todos los elementos serán de más de 8 píxeles de altura, esto permitirá tener más elementos en pantalla.

El bit 2 de LCDC controla el tamaño de los objetos, cuando está a 0, estos son de 8x8 píxeles, pero si está a 1, entonces los objetos serán de 8x16 píxeles, y se usará el tile siguiente al indicado en el byte 2 de OAM para dibujar la mitad inferior del objeto. Prueba a poner este bit a 1 en la función de configuración de sprites, y comprueba el resultado. Debería ser similar al de la figura **??**.

Al poner un identificador en el byte 2 de OAM en el modo de sprites grandes, siempre se ignorará el último bit, o lo que es lo mismo, los números impares se interpretarán como el anterior. Esto quiere decir que si ponemos un 1 como identificador en este modo, el tile superior será el 0, y el inferior el 1, al igual que en nuestro ejemplo, en el que estamos escribiendo un 0. Puedes probar esto cambiando el valor y comprobando que en modo de sprites pequeños cambia al tile 1, pero en el modo de sprites grandes no cambian los tiles dibujados. También puedes probar con otros valores, como el 2 y el 3, que tendrán ambos el mismo resultado.

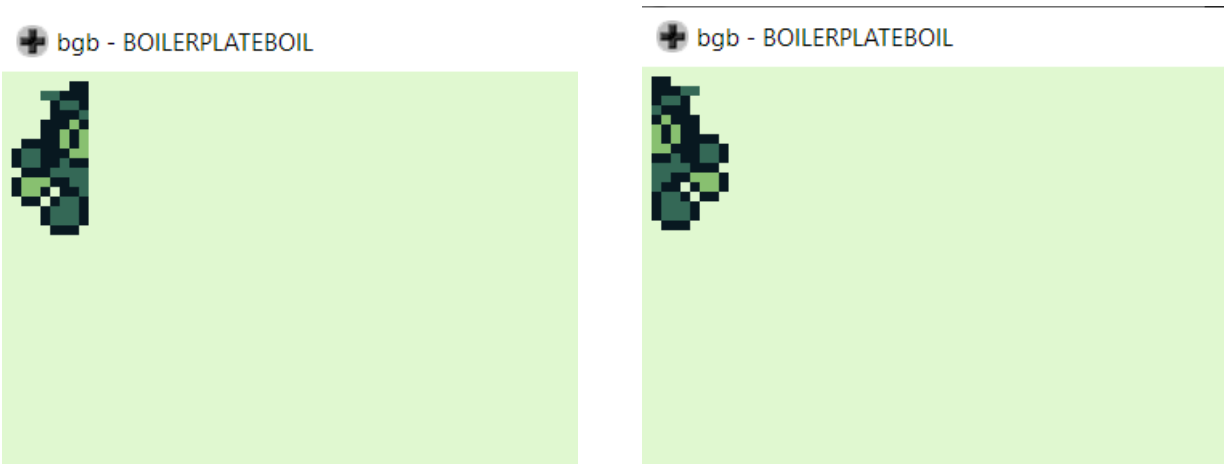



**Figura 4.8:** Sprites de 8x16 píxeles en la esquina superior izquierda.

## 4.4 OAM DMA

Ya sabes como dibujar sprites usando OAM, pero si queremos tener múltiples objetos que se muevan, estar escribiendo sobre esta región de memoria para actualizarlos cada frame, teniendo que esperar a los momentos en los que se puede escribir, necesitamos una forma más cómoda y rápida de hacerlo. La solución a esto es una transferencia de acceso directo a memoria (DMA por sus siglas en inglés). La Game Boy dispone de un mecanismo que permite copiar 160 bytes desde cualquier parte de memoria a OAM. Para iniciar esta transferencia se debe escribir un byte con valor entre $00 y $DF sobre la dirección $FF46, el valor escrito se multiplicará por $100 y se usará como la dirección de memoria de origen. La transferencia sucede de las direcciones $XX00 a $XX9F sobre $FE00 a $FE9F, y dura 160 milisegundos, o 160 ciclos máquina, mucho menos que si se tuviera hacer la transferencia a través de la CPU con el código.

Los principales inconvenientes de este método son que iniciar la transferencia a mitad de dibujar la pantalla puede dar a errores gráficos, por lo que se recomienda realizarla durante el modo 1 de la PPU o directamente desde el controlador de interrupción de VBlank, y que durante la transferencia la CPU sólo tiene acceso a HRAM, una región de memoria entre $FF80 y $FFFE. Esto implica que si se inicia la transferencia desde ROM, durante 160 ciclos máquina la ejecución será impredecible y al recuperar la CPU el acceso a la memoria, la ejecución no seguirá desde el punto en el que se llamó a la transferencia. Esto nos deja con la única opción de hacer la llamada desde HRAM y usar una rutina para esperar hasta el final de la transferencia antes de continuar con el programa.

Ejecutar código desde RAM no es diferente a hacerlo desde ROM, al fin y al cabo el código no son más que una serie de bytes en memoria, si se copian esos mismos bytes sobre RAM y se cambia PC para que lea desde ahí, ejecutará el mismo código. Necesitamos escribir una rutina que inicie la transferencia DMA y espere 160 ciclos, después tenemos que definir una región en HRAM del tamaño de la rutina y entonces copiarla ahí. Para iniciar una transferencia se usará la instrucción `call` usando la dirección de HRAM donde empieza la rutina. También necesitamos definir una región de WRAM en la que crearemos una copia de OAM, que será la que se transfiera, y de ahora en adelante toda la información de los objetos se copiará en esa región como si fuera OAM sin necesidad de preocuparse por si la memoria es accesible.

Primero definimos la región de WRAM, que debe estar alineada con los 8 bits inferiores, para que la

dirección en la que empiece sea $XX00. Podemos asegurar esto con la opción de sección **ALIGN**:

```
   SECTION "Copia OAM", WRAM0, ALIGN[8]

   copiaOAM::
   DS OAM_COUNT*sizeof_OAM_ATTRS
```

Después escribimos la rutina que inicia la transferencia DMA:

```
   rutinaDMA:
   ld a, HIGH(copiaOAM) ; Obtiene el byte alto de la dirección
   ldh [rDMA], a       ; Inicia una transferencia DMA inmediatamente tras la instrucción
   ld a, 40        ; Espera un total de 40x4 = 160 ciclos
.espera
   dec a           ; 1 ciclo
   jr nz, .espera  ; 3 ciclos
   ret
.fin
```

Ahora definimos la región de HRAM donde irá esta rutina:

```
   SECTION "OAM DMA", HRAM

   OAMDMA::
   DS rutinaDMA.fin - rutinaDMA
```

Y por último escribimos la función que copia esta rutina sobre HRAM:

```
   copiaRutinaDMA::
   ld hl, rutinaDMA            ;Origen de datos
   ld b, rutinaDMA.fin - rutinaDMA ;Cantidad de bytes a copiar
   ld c, LOW(OAMDMA)         ;Byte bajo de la dirección de destino
.loop
   ld a, [hl+]
   ld [c], a
   inc c
   dec b
   jr nz, .loop
ret
```

Una última consideración a tener en cuenta, es que las interrupciones deben estar deshabilitadas durante una transferencia, pues provocan que PC cambie a una dirección de ROM. Debes asegurarte de que siempre se deshabiliten las interrupciones antes de iniciar una transferencia, o iniciarla desde un controlador de interrupciones.

Cambia tu código anterior para que el sprite se escriba sobre `copiaOAM` en vez de OAM, elimina cualquier rutina que utilizaras para esperar a que la memoria de vídeo fuera accesible, e incluye una llamada a OAMDMA en el controlador de VBlank. Por último incluye una llamada a la función de copia de la rutina en la configuración inicial. Si lo has hecho todo correctamente deberías seguir viendo el sprite como en la figura 4.9 a pesar de no escribir directamente sobre OAM.

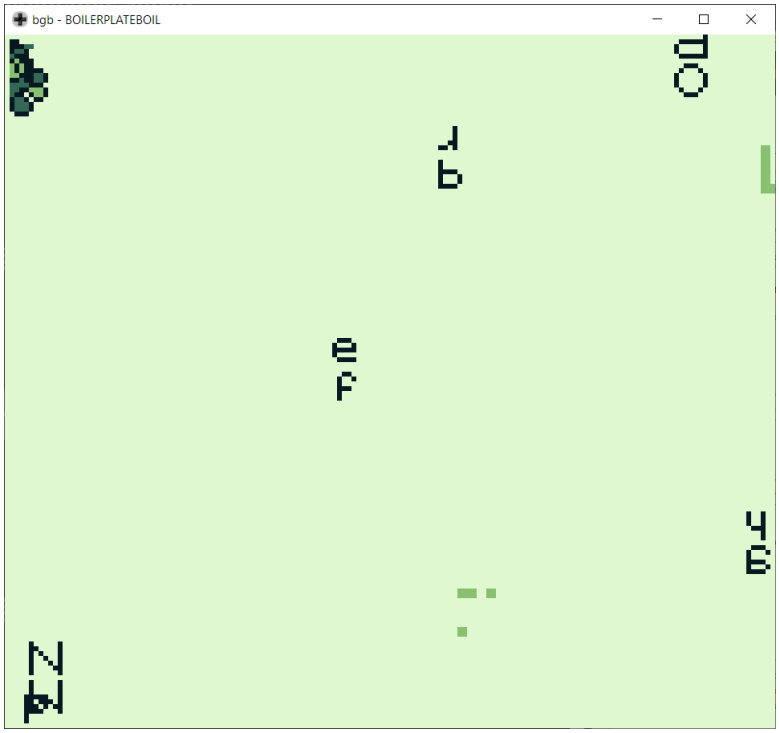



**Figura 4.9:** Sprite de la otra mitad del personaje, copiado con DMA.

Ahora bien, no queremos que salgan todos esos otros sprites y es que aunque inicialmente borramos la región de OAM, no hemos borrado la región de la copia. En la configuración de la pantalla cambia la rutina que borra OAM para que borre la copia al iniciar el programa y volverá a quedar como antes. Si añades en los siguientes 4 bytes la información para la otra mitad del personaje, poniéndolo justo al lado, podrás dibujarlo entero.

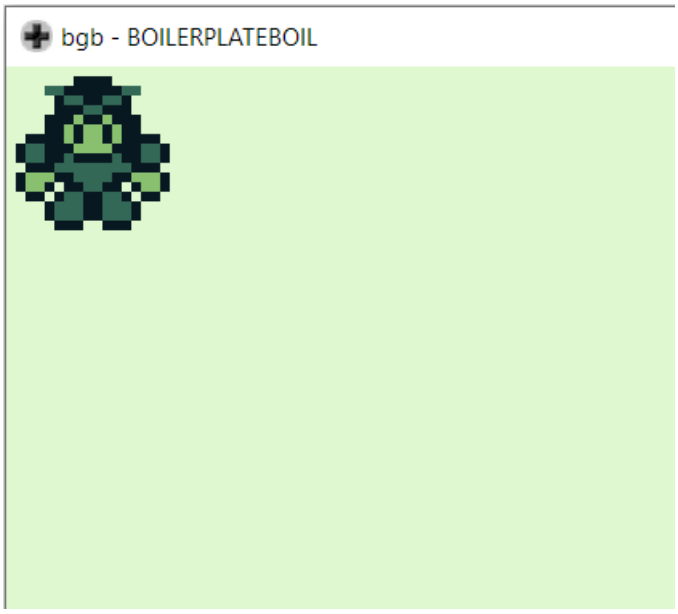



**Figura 4.10:** Sprite completo del personaje.

## 4.5 Crear estructuras

Como has visto, dibujar sprites en sí mismo es muy sencillo, simplemente hay que escribir 4 bytes en una región de memoria, pero no podemos estar escribiéndolos a mano, y además querremos modificarlos en el futuro, actualizando su posición para hacer que se muevan.

Para poder modificar las entidades sin tener que modificar explícitamente cada uno de los sprites que la forman a mano, necesitamos que sus datos estén en RAM. Tendremos por un lado en ROM la información mínima que representa a la entidad, que por ahora serán los sprites que la forman, en RAM tendremos además de la información de los sprites, otros datos esenciales como la posición de la entidad, pero no la posición individual de cada sprite que la forma, sino una global. Y después tendremos una función que se encargará de poner en OAM, o más bien su copia, ambos sprites de la entidad a partir de los datos en RAM. Por ahora supondremos que todas las entidades estarán formadas por dos sprites uno al lado de otro.

Vamos a crear una estructura en ROM que represente los datos de la entidad Jugador. Como vamos a hacer un gestor, o mánager de entidades, pondremos los archivos relacionados en una carpeta que llamaremos *man*, dentro de *src*. También definiremos el espacio en RAM en el que irá nuestra entidad.

```
SECTION "Entity data", ROM0

playerData::
DB PLAYER_SPRITE_ID1, PLAYER_SPRITE_ID2

SECTION "Entity array", WRAM0

RSRESET
DEF ENTITY_POSY RB 1
DEF ENTITY_POSX RB 1
DEF ENTITY_SPRITE_1 RB 1
DEF ENTITY_SPRITE_2 RB 1
DEF ENTITY_SIZE RB 0

entityArray::
DS ENTITY_SIZE
```

En `playerData` definimos los datos de la entidad del jugador, que tiene dos sprites. Estos se han definido como 0 y 2 en una sección de constantes. En `entityArray` se define un espacio del tamaño de una entidad en RAM, que contiene sus posiciones X e Y, y los identificadores de ambos sprites. En el bloque anterior a esta etiqueta se definen una serie de constantes que se usarán como índices al acceder a la entidad. Se han definido usando constantes relativas. Puedes leer sobre como funcionan estas constantes en el anexo B.1.2.3. El primer byte de la estructura representará la posición vertical del objeto, y el segundo su posición horizontal.

Ahora debemos copiar los datos de `playerData` en el lugar adecuado de `entityArray`. `playerData + 0` irá en `entityArray + ENTITY_SPRITE_1` y `playerData + 1` irá en `entityArray + ENTITY_SPRITE_2`. Puedes hacer que la función sirva para cargar otras entidades si haces que la dirección de los datos de la entidad

sea una variable. Además de copiar los valores de los sprites, pon también valores iniciales para la posición. Elige los que quieras, cuando termines esta sección podrás comprobar si la entidad se ha dibujado donde debería según los valores que hayas elegido, y hacer que se dibuje en otros cambiándolos.

Para finalizar, escribe una función que copie los datos de `entityArray` en OAM tal y como se definen las estructuras. La función debería recibir la dirección de la entidad como un parámetro. Pon esta función en otro archivo distinto en la carpeta de sistemas, ya que pertenece al sistema de dibujado. La figura 4.11 muestra una entidad con posición Y de 10 y posición X de 100. Como Y el menor a 16, las 6 filas superiores de los sprites se salen de la pantalla.

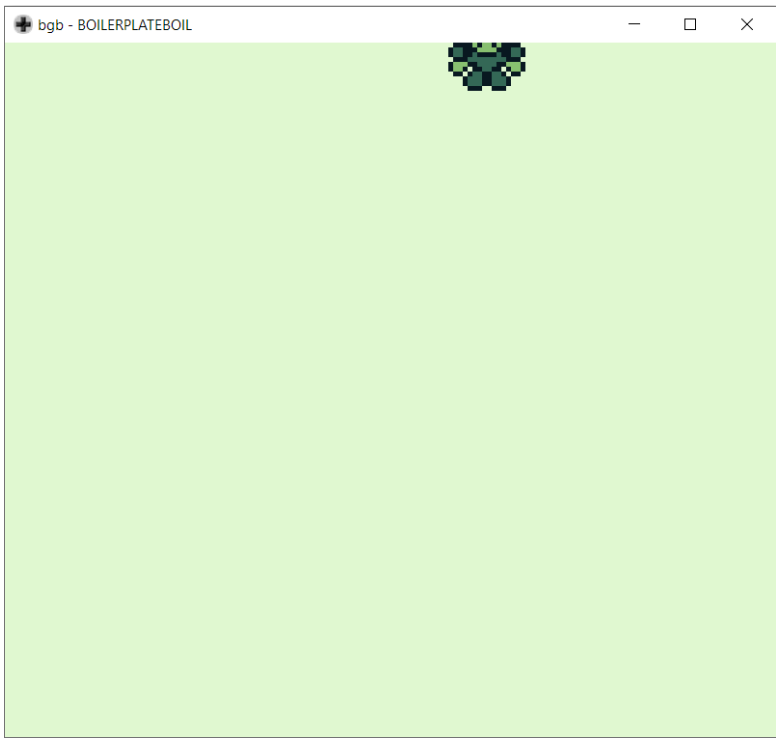



**Figura 4.11:** Entidad sobresaliendo por el borde superior.

## 4.6 Moviendo sprites

Es hora de ver como actualizar la posición de la entidad, es hora de añadir físicas al juego. Lo que haremos por ahora será únicamente añadir campos de velocidad X y velocidad Y a la estructura del jugador, y les daremos un valor por defecto (elige uno entre 1 y 5 para ambas). Añadir estos campos es tan fácil como añadir dos líneas a la sección en la que se definen las constantes de índice de la entidad:

```
RSRESET
DEF ENTITY_POSY   RB 1
DEF ENTITY_POSX   RB 1
DEF ENTITY_VX     RB 1
DEF ENTITY_VY     RB 1
DEF ENTITY_SPRITE_1 RB 1
DEF ENTITY_SPRITE_2 RB 1
DEF ENTITY_SIZE   RB 0
```

La constante `ENTITY_SIZE` se actualizará automáticamente, y también lo hará el tamaño de la zona definida en WRAM. En la inicialización da valores a las velocidades y asegúrate de que la rutina de dibujado sigue funcionando. Tendrás que modificar todas las funciones que usen atributos de una entidad siempre que modifiques la estructura de la entidad, a no ser que las funciones usen las constantes de índice para obtener las direcciones. Si estás usando instrucciones `inc` o sumando arbitrariamente, deberías cambiarlo ahora para que modificar la estructura de la entidad (va a suceder más veces) no suponga tener que reescribir partes de varias funciones.

Crea un nuevo archivo en la carpeta *sys*, en este irá todo lo relacionado con el sistema de físicas, que por ahora consistirá en actualizar las posiciones en función de la velocidad de una entidad. En él crea una función que reciba la dirección de una entidad como parámetro y sume el valor de sus atributos de velocidad a sus posiciones. Al igual que al dibujar la entidad, deberías recorrerla usando las constantes de índices, y como ahora las necesitan varios archivos, deberían estar en otro que se incluya, o en el mismo que el resto de constantes.

Por último crea un bucle en el que se actualice la posición de la entidad con la función que acabas de crear y se dibuje. Añade una espera de un frame entre cada iteración del bucle o la posición se actualizará demasiadas veces y no se podrá ver bien a la entidad moviéndose. Si lo has hecho todo correctamente, deberías ver los sprites moviéndose en la dirección indicada por las velocidades que elegiste. Puedes poner otras velocidades, incluso negativas, y el sprite debería moverse en otra dirección y sentido.

## 4.7 Desacopla el gestor y los sistemas

Ahora mismo deberías tener dos sistemas, uno de físicas y otro de dibujado que interactúan directamente con la entidad, saben en qué dirección está. Pongámonos en una situación en el futuro, en la que tenemos múltiples entidades. Estos sistemas, tal y como está programada su interacción con el gestor de entidades, requieren saber su implementación, dónde se encuentra cada entidad. Si se modificara el gestor de entidades sería necesario modificar cada uno de los sistemas para que funcionen con la nueva implementación. Esto se convierte en un gran problema cuando tenemos muchos sistemas y posiblemente varios tipos de entidades. La imagen 4.12 muestra el acoplamiento actual entre los sistemas y el gestor de entidades.

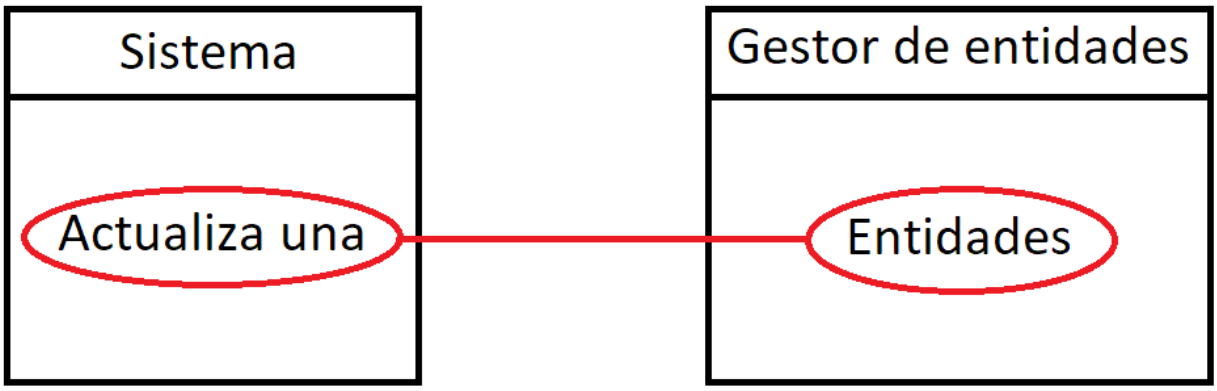



**Figura 4.12:** Acomplamiento entre sistemas y entidades.

Necesitamos crear una función que haga de interfaz con la que interactúen los sistemas y sea la encargada de obtener las direcciones de las entidades. En cada sistema habría una función que se llamaría para actualizar todas las entidades usando ese sistema, en nuestro caso por ahora, entrarían el de físicas y el de dibujado. Esta función general llamaría a la función de interfaz del gestor de entidades pasándole como parámetro la función que se usa para actualizar una única entidad en ese sistema. El gestor de entidades hará una llamada a la función que ha recibido como parámetro por cada entidad, pasándole una. De este modo se desacopla la implementación de las entidades de los sistemas. El resultado sería un esquema como el de la imagen 4.13.

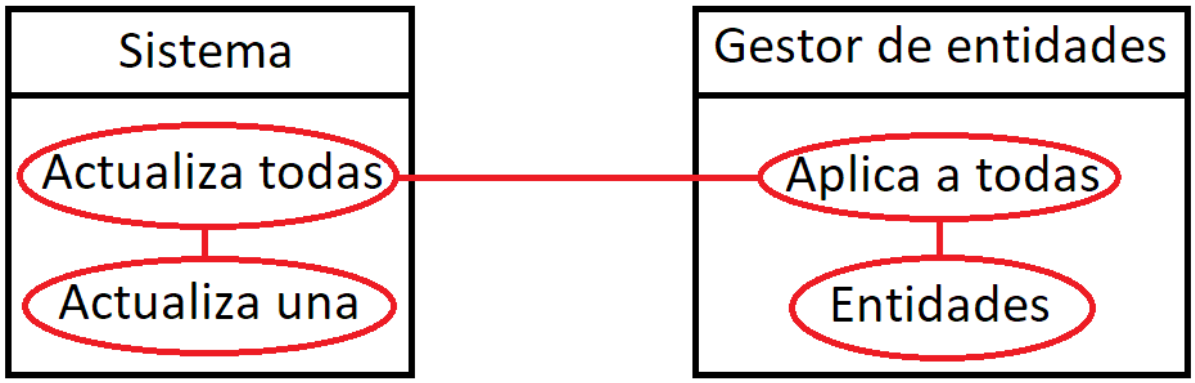



**Figura 4.13:** Sistemas y entidades no acoplados.

Como puedes ver, con esta estructura aunque cambie la implementación de las entidades no es necesario modificar cada uno de los sistemas, como mucho habrá que modificar la función del gestor que aplica una función a todas las entidades.

Vamos a implementar esta estructura, empieza creando una función vacía en el gestor de entidades, esta será la encargada de llamar a las funciones específicas de cada sistema con cada entidad. En cada uno de los sistemas crea un función que llame a la anterior pasando como parámetro la rutina que actualiza una única entidad teniendo su dirección. Esta función será la que se llamará a partir de ahora cuando se quieran actualizar las entidades.

```
sys_dibujaTodas::
    ld hl, sys_dibujaUna
    call man_entidades_aplicaFuncionTodas
ret
```

Ahora necesitamos implementar la función que haga el salto a otra arbitraria, que recibe como parámetro. Como por ahora solo tenemos una entidad, esto no será muy complicado. Empieza poniendo la dirección de la entidad dónde debería recibirla la función del sistema, ten en cuenta que dadas las limitaciones de esta función, todas las funciones de sistema deben recibir la dirección de la entidad en el mismo registro, el gestor no va a diferenciar qué sistema está llamando y cambiar el registro en el que se pasa la entidad, o al menos no lo hará en nuestra implementación. Hay varias formas de hacer este salto, la más sencilla de ellas siendo usar la instrucción `jp hl`, que salta a la dirección indicada en el registro HL. Antes de hacer el salto habría que poner en la pila la dirección de retorno, o al retornar de la función del sistema no volverá al mismo lugar. El único inconveniente

de este método es que no podemos usar HL para pasar ningún parámetro a la función a la que estamos saltando, al estar este registro ocupado por la dirección del salto, pero esto puede arreglarse cambiando el registro en el que se recibe el parámetro.

En caso de no haber registros libres y requerir de HL, otra opción es hacer código modificable. Igual que para la rutina DMA copiamos código en RAM, podemos hacer lo mismo aquí, copiando el código de una instrucción `jp d16` en RAM, y modificando los dos bytes que indican la dirección del salto desde nuestra función del gestor de entidades. Entonces sólo habría que hacer una llamada o un salto a la dirección de RAM en la que se encuentra este código.

Por último, deberíamos quitar la dirección en OAM del sistema de dibujado. Ahora mismo la dirección sobre la que se escriben los datos de los sprites está puesta a mano, pero en el futuro será una distinta para cada entidad. Deja espacio para una variable de 2 bytes en RAM, que contendrá la dirección de OAM donde se escribirán los datos de la entidad, y cambia la función de dibujado para que obtenga esta dirección en vez de introducirla directamente en el código. También tendrás que inicializar esta variable en la configuración.

## 4.8 Crea entidades

Vamos a sacarle partido ahora a tener los sistemas separados del gestor, vamos a aumentar el espacio dedicado a entidades y crear varias. Necesitamos cambiar la definición del espacio en el que estábamos copiando la plantilla de la entidad.

```
SECTION "Entity array", WRAM0

entityArray:
DS ENTITY_SIZE*MAX_ENTITIES
```

Además, para tener entidades con posiciones y velocidades distintas, en esta sección incluiremos estos datos en la plantilla de la entidad.

```
playerData::
DB 50  ;POSY
DB 50  ;POSX
DB -1  ;VY
DB  1  ;VX
DB PLAYER_SPRITE_ID1, PLAYER_SPRITE_ID2
```

Crea otras 4 entidades más con valores de posición y velocidad distintas, y dales una etiqueta a cada una, pero mantén los sprites iguales. Al final de esta sección verás cinco copias del personaje moviéndose por la pantalla.

Ahora vamos a cambiar la función que carga la entidad. Para empezar, tiene que obtener los datos de posición y velocidad a partir de la estructura que recibe como parámetro. Pero el mayor problema está en decidir en qué dirección se escriben estos datos, ya que no pueden estar las cinco entidades en la misma dirección. Necesitamos dos bytes en RAM que contendrán la dirección del primer hueco libre de la lista de entidades. En

estos dos bytes se guardará `entityArray` durante la configuración inicial, y se aumentará su valor en el tamaño de una entidad tras crear una nueva. Cuando se cree una entidad, esta se copiará sobre la dirección de memoria indicada por estos dos bytes.

Ahora solo debes llamar a la función que crea entidades pasándole como parámetro cada una de las plantillas, si haces todo esto, desde el emulador deberías poder ver como se crean las 5 copias del personaje en la memoria, pero por ahora solo verás una, porque la función a través de la cual se invoca la función de dibujado solo se llama una vez. Como prueba para ver si se han creado todas las entidades correctamente, puedes cambiar la dirección que toma la función de dibujado para que se llame con alguna de las otras entidades sumándole un múltiplo del tamaño de una entidad.

Un detalle más a tener en cuenta, es que no debería crearse ninguna entidad nueva si se llama a la función cuando no queda hueco en el espacio que hemos designado. Para controlar esto necesitaremos otro byte más que cuente el número de entidades creadas. Antes de crear una entidad nueva se comprobará este byte, y si su valor es igual al número máximo de entidades, se retornará de la función sin crear nada.

Ahora tendrás que modificar la función del gestor de entidades para que aplique la función que recibe por parámetro a todas las entidades, y no solo a una. Por último, tenemos que modificar la función de dibujado para que escriba los datos de cada entidad en un hueco distinto de OAM. Esto lo haremos incrementando la variable que hemos creado anteriormente en el tamaño que ocupan dos sprites en OAM tras dibujar cada entidad. Siempre tendremos que inicializar esta variable antes de dibujar ninguna entidad para que vuelva a apuntar al principio de la OAM. Una vez esté terminada, puedes comprobar que todo está funcionando ejecutando el programa, deberías ver las cinco copias del personaje moviéndose según los parámetros que introdujiste.

## 4.9 Borra entidades

Si solo podemos crear entidades nuevas, llegará un punto en el que nos quedaremos sin espacio, como ya hemos hecho en la sección anterior, por lo que necesitamos una forma de eliminar entidades que ya no sean necesarias.

Actualmente nuestro motor considera que las entidades existentes están todas seguidas, lo cual también hace más sencilla la implementación de la función que actúa sobre todas las entidades. Además, comprobar cuál es la siguiente posición en la que debería ir la siguiente entidad sería mucho más complicado si hay huecos en la lista de entidades. Lo que haremos entonces al borrar una entidad es reemplazarla por la última. La figura 4.14 muestra un ejemplo de borrado.

Adicionalmente, como ahora se dibujan varias entidades, si no eliminamos de OAM los datos del último hueco, se quedarán dibujados aún cuando la entidad se ha borrado, por lo que debemos acceder al último hueco en el que hay entidades dibujadas usando el contador de entidades, y poner ceros en los atributos de posición Y para esconder los dos sprites de la entidad.

Para comprobar que la función que hayas creado para borrar entidades funciona basta con que crees dos entidades con velocidades distintas y después borres la primera. Debería dibujarse una sola entidad en el primer

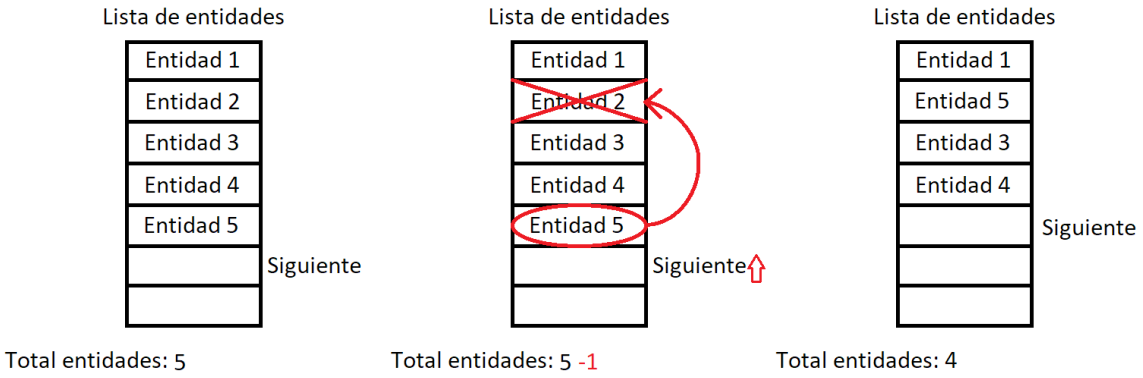



**Figura 4.14:** Proceso de borrado de una entidad.

hueco de OAM con la velocidad de la segunda.

## 4.10 Tipos de entidades y animaciones

Pongamos que ahora quieres controlar una entidad por teclado, o que no quieres que algunas tengan físicas, o que quieras hacer algunas invisibles y solo existan para comprobar colisiones con ellas, por ejemplo para que cuando el jugador pase por encima se active un diálogo. Para evitar que las entidades que queramos hagan uso de algunos sistemas, necesitamos una forma de clasificarlas e identificarlas por tipos. Para conseguir esto podemos añadir un byte a las entidades que denote el tipo, y que sea una máscara de bits. Por ejemplo el bit 0 podría indicar que la entidad tiene físicas y el bit 1 que se debe dibujar, entonces según el valor de este byte podríamos tener los siguientes tipos de entidades:

- 00000000: No se dibuja ni se mueve.
- 00000001: Se mueve pero no se dibuja.
- 00000010: Se dibuja pero no se mueve.
- 00000011: Se mueve y se dibuja.

Más adelante crearemos más tipos que usarán el resto de bits libres.

Añade el byte del tipo a las estructuras, y haz que alguna se dibuje pero no se mueva. Podrías hacer alguna entidad que se moviera pero no fuera visible, pero tendrías que mirar en la memoria con el emulador para comprobar que se está moviendo. Puedes combinar máscaras de bits con los operadores binarios, por ejemplo, el siguiente es un ejemplo de dos estructuras, una con físicas y otra sin, ambas se dibujan:

```
DEF ENT_TIPO_FIS EQU %00000001
DEF ENT_TIPO_DIB EQU %00000010

entityData::
DB ENT_TIPO_FIS | ENT_TIPO_DIB
DB 50  ;POSY
DB 50  ;POSX
DB -1  ;VY
DB  1  ;VX
DB PLAYER_SPRITE_ID1, PLAYER_SPRITE_ID2
```

```
entityData2::
DB ENT_TIPO_DIB
DB 2
DB 95
DB  1
DB  1
DB PLAYER_SPRITE_ID1, PLAYER_SPRITE_ID2
```

Pero no podemos dejar que se llame a las funciones de sistema que actualizan entidades si el tipo no corresponde. Crea una nueva función, usando de base la que ya tienes para aplicar funciones a todas las entidades, y haz que reciba un parámetro que será el tipo. Entonces añade una comprobación para que la llamada solo se haga si la entidad tiene el tipo recibido como parámetro.

Una vez tengas la función, tendrás que llamar a esta desde los sistemas en vez de usar la anterior. Al ejecutar el programa, aquellas entidades que hayas decidido no deberían moverse, y si has hecho que alguna no se dibuje, deberías ver menos entidades de las que hayas creado.

Como podemos tener entidades que no se dibujen, no todas usarán la OAM. Debemos modificar las funciones de creación y borrado de entidades según si se van a dibujar o no. Todo lo que tenemos que hacer es crear una nueva variable en RAM que cuente cuantas entidades que se dibujan existen en memoria en todo momento. Esta variable se inicializaría a 0 y se aumentaría o reduciría cada vez que se creara o destruyera una entidad con el tipo adecuado. Además en la función de borrado, para obtener la dirección de OAM en la que hay que borrar los sprites se deberá usar esta nueva variable en vez del contador total.

Vamos a aprovechar esta clasificación en tipos para crear un nuevo sistema y tipo, las animaciones. Animar una entidad implica tener que cambiar sus sprites asignados, por lo que primero vamos a cambiar como se guardan los sprites de una entidad para hacerlos más fáciles de cambiar.

Actualmente tenemos una estructura que contiene dos identificadores de sprites:

```
RSRESET
DEF ENTITY_TYPE   RB 1
DEF ENTITY_POSY   RB 1
DEF ENTITY_POSX   RB 1
DEF ENTITY_VY     RB 1
DEF ENTITY_VX     RB 1
DEF ENTITY_SPRITE_1 RB 1
DEF ENTITY_SPRITE_2 RB 1
DEF ENTITY_SIZE   RB 0
```

En esta estructura se guardan ambos identificadores de sprites, por lo que tendríamos que cambiar ambos para modificar el dibujo que produce la entidad. En vez de eso podemos cambiarlos por un atributo de dos bytes que haga de puntero.

```
RSRESET
DEF ENTITY_TYPE   RB 1
DEF ENTITY_POSY   RB 1
```

```
DEF ENTITY_POSX   RB 1
DEF ENTITY_VY     RB 1
DEF ENTITY_VX     RB 1
DEF ENTITY_SPRITE_PTR RW 1
DEF ENTITY_SIZE   RB 0
```

Entonces solo tenemos que crear una estructura en ROM que contenga los datos del sprite, y añadir a la plantilla de la entidad la dirección de esa estructura:

```
sprite_jugador_frente::
    DB PLAYER_SPRITE_FRONT
    DB OAMF_PAL0
    DB PLAYER_SPRITE_FRONT
    DB OAMF_PAL0 | OAMF_XFLIP
```

De paso que estamos cambiando como representamos los sprites, vamos utilizar también losflags, que por ahora hemos ignorado. Previamente hemos visto que el cuarto atributo de cada sprite en OAM son una serie de características extra que se pueden activar o desactivar. En la Game Boy clásica, estas son elegir entre dos paletas, invertir el sprite en el eje X o Y, y hacer que se vea por detrás del fondo. Dado que nuestro personaje es simétrico, podemos usar un solo sprite y en el segundo hacer que se invierta en el eje X. De esta forma obtenemos el mismo gráfico ocupando la mitad de espacio en Tile Data.

Cambia el sistema de dibujado para que obtenga los identificadores y las flags del sprite a partir del atributo de la dirección, y cambia los sprites de todas las entidades por un puntero a la estructura que acabas de crear. Visualmente no debería haber ninguna diferencia.

Como nuestro personaje es simétrico, los dos tiles de la derecha no los necesitamos, por lo que podemos abrir GBTD y cambiar el tamaño del diseño de 16x16 a 8x16 píxeles, como en la figura 4.15. Aprovecha también para hacer algunos diseños del personaje caminando.

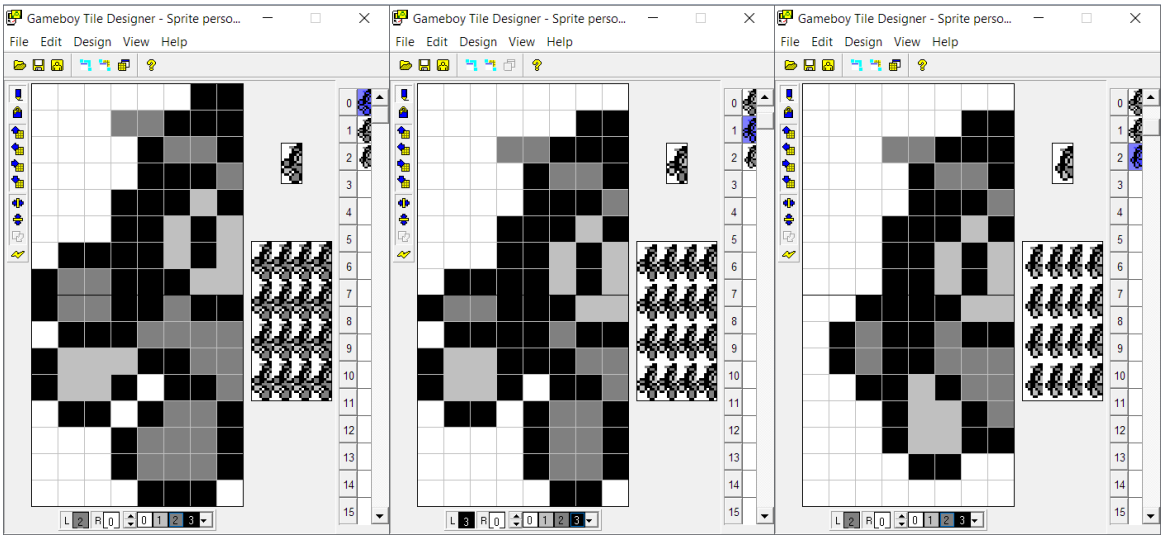



**Figura 4.15:** Diseño del personaje en 8 por 16 píxeles.

Crea dos estructuras de sprites como la que hemos hecho antes, pero esta vez para representar al personaje caminando, tanto con el pie derecho como con el izquierdo delante.

```
sprite_jugador_frente_camina1::
DB PLAYER_SPRITE_FRONT_WALK1
DB OAMF_PAL0
DB PLAYER_SPRITE_FRONT_WALK2
DB OAMF_PAL0 | OAMF_XFLIP

sprite_jugador_frente_camina2::
DB PLAYER_SPRITE_FRONT_WALK2
DB OAMF_PAL0
DB PLAYER_SPRITE_FRONT_WALK1
DB OAMF_PAL0 | OAMF_XFLIP
```

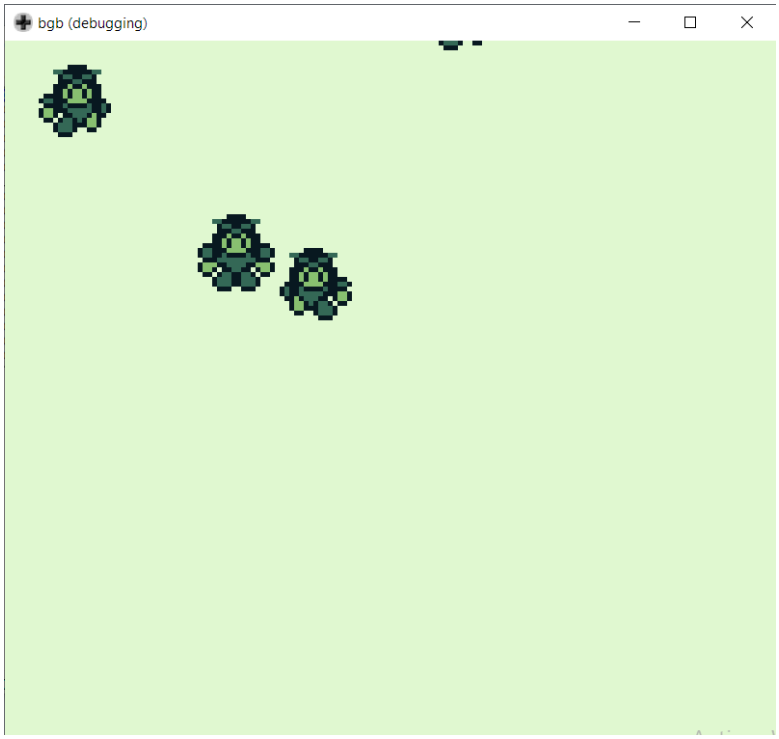



**Figura 4.16:** Varios sprites diferentes

Para poder aplicar una animación a una entidad, tenemos que definirla primero, y no es más que un conjunto de imágenes que se suceden. Crea una estructura de datos formada por una secuencia de sprites. La animación se hará cambiando el puntero a sprite de la entidad por uno de los de la animación de forma consecutiva.

```
anim_jugador_camina_frente::
DW sprite_jugador_frente
DW sprite_jugador_frente_camina1
DW sprite_jugador_frente
DW sprite_jugador_frente_camina2
```

Esta animación debería repetirse, por lo que necesitamos incluir un puntero al principio de la animación.

```
anim_jugador_camina_frente::
DW sprite_jugador_frente
```

```
DW sprite_jugador_frente_camina1
DW sprite_jugador_frente
DW sprite_jugador_frente_camina2
DW anim_jugador_camina_frente
```

Pero este puntero no es igual al resto. Mientras que los 4 primeros son punteros a sprites, el último es un puntero a la animación, si cambiamos el puntero a sprite de la entidad por él, no volverá a aparecer el primer sprite. Este último puntero debe usarse para recuperar el primer sprite del resultado, y el sistema de animación necesitará una forma de diferenciar cuando los siguientes dos bytes de la estructura pertenecen a un sprite o a una animación. Una forma de hacer esto es incluir un identificador antes del puntero a la animación para que el sistema sepa que lo que hay a continuación no es otro sprite, pero debemos elegir este identificador con cuidado, pues no podemos elegir ningún valor que pudiera tomar un puntero a sprite.

Para asegurarnos de que el valor elegido no es el de un sprite, podemos elegir cualquier dirección de memoria de vídeo ($8000 a $9FFF), pero personalmente he elegido usar la dirección de entrada de la ROM ($0100).

```
DEF ANIMATION_END EQU $0100

anim_jugador_camina_frente::
DW sprite_jugador_frente
DW sprite_jugador_frente_camina1
DW sprite_jugador_frente
DW sprite_jugador_frente_camina2
DW ANIMATION_END
DW anim_jugador_camina_frente
```

Ahora necesitamos crear un sistema y un tipo de animaciones. Empieza creando el tipo y añadiendo un atributo a las entidades para punteros a animaciones.

```
 RSRESET
 DEF ENTITY_TYPE  RB 1
 DEF ENTITY_POSY  RB 1
 DEF ENTITY_POSX  RB 1
 DEF ENTITY_VY    RB 1
 DEF ENTITY_VX    RB 1
 DEF ENTITY_SPRITE_PTR RW 1
 DEF ENTITY_ANIM_PTR RW 1
 DEF ENTITY_SIZE  RB 0
```

Elige alguna de las entidades, dale el tipo animada y dibujada, y en su atributo de animación ponle la que acabamos de crear. Si tienes otras entidades que no vayan a ser animadas, puedes rellenar el hueco de su atributo con cualquier valor, como por ejemplo ceros.

```
playerData::
DB ENT_TIPO_DIB | ENT_TIPO_ANI
DB 50  ;POSY
DB 50  ;POSX
DB -1  ;VY
```

```
DB 1   ;VX
DW sprite_jugador_frente
DW anim_jugador_camina_frente
```

Ahora tenemos que crear la función del sistema que modifica el sprite de la entidad. Esta función debe usar el puntero a animación para obtener el puntero del siguiente sprite, copiarlo en el atributo correspondiente, y aumentar el valor del puntero a la animación para que apunte al siguiente sprite. La imagen 4.17 muestra como hay que modificar la entidad.

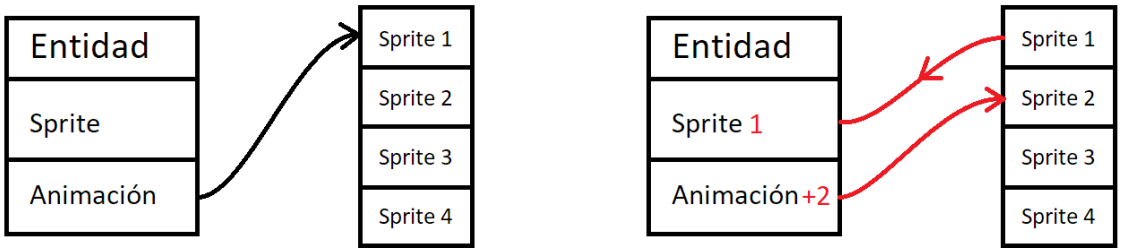



**Figura 4.17:** Sistema de animación

Con el sistema programado, puedes incluir una llamada a este en el bucle principal y ver como el personaje cambia de sprite. Te habrás dado cuenta de que la animación sucede demasiado rápido, el sprite está cambiando 60 veces por segundo, y no se puede ver bien la animación. Si queremos controlar el tiempo que debe haber entre un sprite y el siguiente de la animación, necesitamos incluir en la estructura de la animación la cantidad de frames que debe haber entre cada sprite, y un atributo en la entidad para este contador.

```
anim_jugador_camina_frente::
DW sprite_jugador_frente
DB 10
DW sprite_jugador_frente_camina1
DB 20
DW sprite_jugador_frente
DB 10
DW sprite_jugador_frente_camina2
DB 20
DW ANIMATION_END
DW anim_jugador_camina_frente
```

```
RSRESET
 DEF ENTITY_TYPE  RB 1
 DEF ENTITY_POSY  RB 1
 DEF ENTITY_POSX  RB 1
 DEF ENTITY_VY    RB 1
 DEF ENTITY_VX    RB 1
 DEF ENTITY_SPRITE_PTR RW 1
 DEF ENTITY_ANIM_PTR RW 1
```

```
DEF ENTITY_TEMPLATE_SIZE RB 0
DEF ENTITY_ANIM_COUNT RB 1
DEF ENTITY_SIZE  RB 0
```

El contador de animación no forma parte de la plantilla de la entidad, por lo que debe inicializarse al crearla. Para completar nuestra animación tenemos que cambiar la función de animación de entidades para que antes de cambiar el sprite lea el contador, lo reduzca en uno, y actualice el sprite solo si el resultado es cero. Además, tras actualizar el sprite deberá copiar el contador de la animación en el contador de la entidad. El contador de las entidades debe inicializarse a 1 para que en la primera llamada al sistema de animaciones se actualice el sprite.

Con todo esto tendrás una animación sencilla de 4 sprites que muestra a un personaje caminando. Más adelante crearemos algunas animaciones más y veremos como cambiarlas.

## 4.11 Usando macros y moviendo la pila

En las últimas secciones hemos estado añadiendo atributos a las entidades poco a poco. Cada uno de estos cambios ha supuesto tener que actualizar cada una de las plantillas que tenemos para entidades distintas, lo cual es probable que haya resultado en un error durante la ejecución por olvidar añadir algún atributo a una de las entidades, y encontrar el origen del fallo puede haber sido tedioso. Vamos a ver como usar macros para evitar que estos fallos lleguen al código final del juego y se detecten antes de empezar a ensamblar.

Una macro (macroinstrucción) es un símbolo que representa un bloque de código. Con las macros podemos definir fragmentos de código e invocarlos simplemente escribiendo el nombre de la macro. Al ensamblarse el código, el nombre de la macro se sustituirá por el código equivalente. Las macros pueden tener parámetros, que se pueden usar para cambiar el código por el que se sustituirán. Vamos a definir una macro para crear plantillas de entidades, parte de la macro consistirá en comprobar que el número de argumentos que ha recibido al ser invocada coincide con la cantidad de atributos que debería tener una plantilla de entidad, si no fuera así, se lanzaría un error y no se ensamblaría el programa.

Para definir una macro se usa la directiva `MACRO` seguida del nombre que queramos darle. Para indicar dónde termina una macro, hay que usar la directiva `ENDM`.

```
MACRO entityTemplate
    ASSERT _NARG == ENTITY_TEMPLATE_NUM_ATTRS
ENDM
```

La macro anterior comprueba que el número de argumentos que recibe es igual a `ENTITY_TEMPLATE_NUM_ATTRS`. Al ser invocada, en caso de no cumplir la condición, el ensamblador dará un error, indicando la línea desde donde se ha invocado la macro, seguida de la línea de la propia macro que ha provocado el fallo.

Para usar argumentos en el código de una macro, hay que usar las expresiones **\1** a **\9** para los primeros 9 parámetros. Para usar más parámetros las expresiones tienen la forma **\<n>**, donde **n** es el número de argumento, por ejemplo el décimo se escribiría como **\<10>**.

Si quieres ver más información y usos de las macros, consulta el apéndice B.4.

Haz una macro con la estructura que tienen las plantillas de entidades.

```
MACRO entityTemplate
    ASSERT _NARG == ENTITY_TEMPLATE_NUM_ATTRS
    \1::       ;Nombre
    DB \2      ;Tipo
    DB \3      ;POSY
    DB \4      ;POSX
    DB \5      ;VY
    DB \6      ;VX
    DW \7      ;Sprite
    DW \8      ;Animación
ENDM
```

Y ahora cambia las definiciones de entidades por invocaciones a la macro.

```
SECTION "Entity data", ROM0

    DEF ENT_TIPO_JUG EQU ENT_TIPO_FIS | ENT_TIPO_DIB | ENT_TIPO_ANI
    DEF ENT_TIPO_MOV EQU ENT_TIPO_DIB | ENT_TIPO_FIS

    EntityTemplate playerData, ENT_TIPO_JUG, 50, 50, -1, 1, sprite_jugador_frente, anim_
        jugador_camina_frente

    EntityTemplate playerData2, ENT_TIPO_DIB, 2, 95, 1, 1, sprite_jugador_frente_camina1, 0

    EntityTemplate playerData3, ENT_TIPO_MOV, 88, 71, -2, 1, sprite_jugador_frente, 0

    EntityTemplate playerData4, ENT_TIPO_MOV, 62, 66, 2, 1, sprite_jugador_frente_camina2, 0

    EntityTemplate playerData5, ENT_TIPO_DIB, 20, 15, 3, 2, sprite_jugador_frente_camina1, 0
```

Todos los parámetros de la llamada deben estar en la misma línea, separados por comas, y el nombre de la macro en la invocación debe estar indentado (pulsa tabulador para desplazarlo a la derecha). La próxima vez que tengas que modificar las entidades añadiendo o eliminando parámetros y se te olvide alguna, descubrirás el error antes de siguiera ejecutar el programa, ahorrándote tener que depurarlo para encontrarlo.

Ahora vamos a tocar otro detalle que es fácil pasar por alto. Como ya sabes la pila es una estructura de datos que va guardando las direcciones de memoria desde las que se llama a funciones para poder retornar a ellas, o puedes también guardar registros para recuperar sus contenidos después de modificarlos. Esto supone un problema y es que hace a la pila crecer cuantas más llamadas anidadas hagas y más registros guardes en la pila, aunque se vacía al retornar de las funciones y recuperar los registros.

Como la pila comparte región de memoria con otros datos, es posible que la pila aumente hasta sobrescribir variables o incluso código, como muestra la figura 4.18. Inicialmente la pila está en HRAM, empezando en $FFFE, y va creciendo desde ahí a direcciones más bajas de memoria. Dada la utilidad de HRAM para almacenar variables, al haber una instrucción específica que puede acceder a esta región de memoria sin usar registros usando un solo byte para indicar la dirección, querremos aprovecharla al máximo sin preocuparnos por si la pila

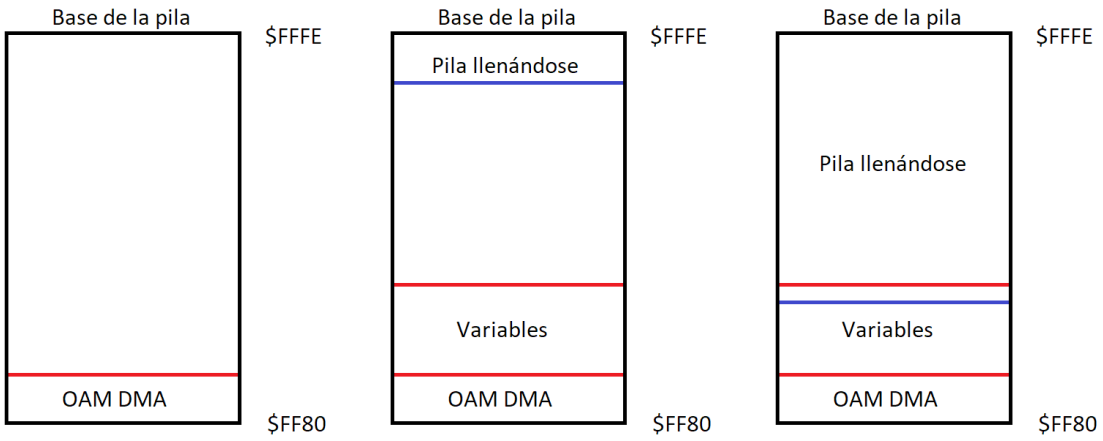



**Figura 4.18:** Crecimiento de la pila en HRAM.

puede llegar a crecer lo suficiente como para modificar nuestros datos o código. Además, solo disponemos de 126 bytes de HRAM, por lo que dejar que la pila ocupe este espacio es un desperdicio.

Vamos a mover la base de la pila al final de WRAM, a la dirección $DFFF. Por ahora todas nuestras variables en WRAM se han definido en el banco 0, que va de las direcciones $C000 a $CFFF, por lo que por el momento no supondrá un problema, y aunque en el futuro quisiéramos usar el banco 1 de WRAM, con direcciones entre $D000 y $DFFF, RGBASM intenta poner todo lo que definamos en la zona más baja de memoria posible, por lo que la región cerca de $DFFF sería lo último en ocuparse.

Para mover la pila de sitio no tenemos más que modificar el registro SP.

```
ld sp, $DFFF
```

Ten en cuenta que esta instrucción debe ponerse fuera de cualquier llamada a una función, en caso contrario, al intentar retornar de la función leerá dos bytes de $DFFF en vez de donde estaba antes la pila por lo que no tendrán los valores correctos. Si aún así quieres poner esta modificación dentro de una función, podrías hacer pop de la pila dentro de la función, después modificar el registro SP, y entonces hacer push con los valores que has extraído antes. De esta forma se conserva la dirección de retorno de la función, pero del mismo modo si esta era una función anidada seguirá habiendo problemas, pues no podrá retornar correctamente de la siguiente función. Para ahorrarnos estos problemas, simplemente haz que esa sea la primera instrucción en ejecutarse al iniciar el programa en `EntryPoint`, antes de cualquier función.

Estos cambios no deberían haber modificado el programa, puedes probar a ensamblar y comprobar que todo funciona igual que antes. Lo que sí puedes hacer es comprobar en el emulador como cambia la dirección de la pila.

## 4.12 Mover entidades con los botones y cambiar animaciones

Como las entidades se pueden mover con sus atributos de velocidad, vamos a hacer que sea el jugador el que decida hacia donde se mueve usando los botones de dirección. Tendrás que crear una función que compruebe el estado de los botones y cambie la velocidad de la entidad acorde a los botones pulsados, y entonces tendrás que llamar a esta función dentro del bucle principal. Como en un principio la única entidad que usará el sistema de input será la del jugador, no hace falta que crees un tipo para él, basta con llamar a la función pasándole como parámetro la dirección de la primera entidad.

Ahora vamos a hacer que la entidad del jugador tenga animaciones de caminar en 4 direcciones distintas, y que esta cambie en función de hacia donde se esté moviendo. Empieza por crear los diseños de los sprites y las estructuras de las 3 animaciones de caminar restantes. Puedes probar cada animación individualmente sustituyéndolas en la plantilla del jugador.

Con todas las animaciones creadas, ahora tenemos que añadir al sistema la capacidad de cambiar la animación en función del estado en el que se encuentra la entidad. Por ahora tendremos los estados ”quieto”, .ªrriba”, .ªbajo”, ”derecha”, e ïzquierda”. Queremos relacionar cada uno de los estados direccionales con una animación distinta, y en el caso de estar quieto, simplemente no se actualizará la animación. Además, en el caso de que el estado coincida con el del frame anterior, no se deberá cambiar la animación. Para hacer todo esto, necesitamos añadirle a la entidad un byte para controlar su estado, dos bytes para incluir una función que calculará su estado según los otros atributos, y un puntero a un array que relacionará estados con animaciones. Para las entidades que tengan una sola animación, crearemos una función que siempre devuelva 1 para obtener el estado, y les daremos un estado por defecto de 1. De esta forma nunca actualizarán su animación por defecto. No olvides actualizar la constante del número de atributos para que la macro siga funcionando como debe.

Ahora tenemos que cambiar la función del sistema para que primero obtenga el estado actual de la entidad usando su función asociada, entonces si su valor es 0 (quieto), obtiene el anterior estado y reinicia la animación para dejar al personaje en la pose quieta, si el valor coincide con el estado anterior, pasa a actualizar el sprite como hasta ahora, y si no coincide, primero guarda el nuevo valor de estado, obtiene el puntero a la animación correspondiente del array asociado a la entidad y lo copia sobre el atributo de animación, cambia el contador de animación a 1, y entonces actualiza el sprite como de normal.

Con los estados que hemos elegido, el array de animaciones para el jugador es el siguiente:

```
anim_jugador_array::
    DW anim_jugador_camina_espalda
    DW anim_jugador_camina_frente
    DW anim_jugador_camina_derecha
    DW anim_jugador_camina_izquierda
```

La función de estado de la entidad jugador debe devolver un 0 si está quieto, un 1 si tiene velocidad hacia arriba, un 2 si tiene velocidad hacia abajo, un 3 si se mueve a la derecha, y un 4 si se mueve hacia la izquierda. Si a este número le restas uno, lo multiplicas por dos y el resultado lo sumas a `anim_jugador_array`, obtendrás el puntero a la animación correspondiente a ese estado, como se muestra en la figura 4.19.

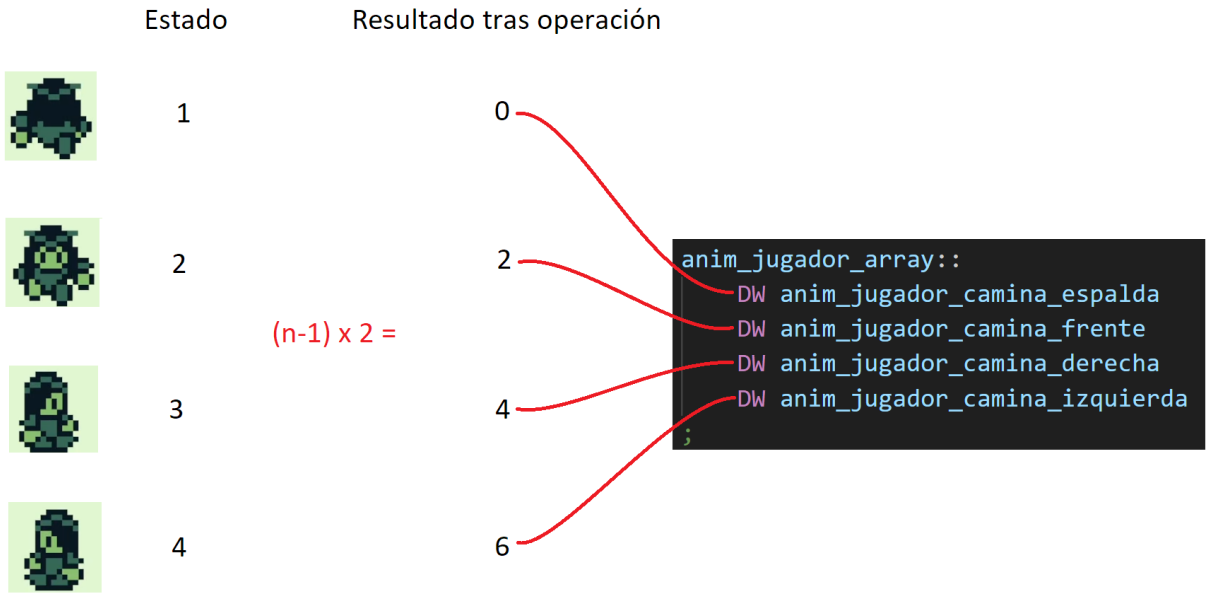



**Figura 4.19:** Relación entre estado e índices de animaciones.

Ya tienes un personaje que puedes mover son los botones y el cual cambia la dirección en la que mira según hacia donde se mueva.

## 4.13 Enemigos y sistema de borrado de entidades

Vamos a añadir una plantilla de entidad enemiga. A partir del diseño de enemigo que hiciste al principio de este capítulo, crea una pequeña animación para esta entidad. Sólo tendrá una animación, así que su función de estado será la genérica.

En el juego que tendremos al terminar este capítulo, habrá entidades enemigas que aparecerán y desaparecerán, por lo que necesitamos darles funciones de comportamiento que les digan cuando ser borradas, pero no podemos dejar que sea el propio sistema de comportamiento el que invoque la función de borrado. Si se borra una entidad en mitad del proceso de actualizarlas con un sistema, como la última entidad se sobrescribe en la que se está borrando, no se actualizará. La imagen 4.20 muestra este problema. El sistema puede que igualmente actualice la copia de la entidad en su posición anterior, porque tomó la cantidad de instrucciones antes del borrado, pero cualquier sistema posterior considerará ese hueco como vacío.

Para solventar este problema, lo que necesitamos es una forma de marcar las entidades que queremos que sean destruidas, y borrarlas fuera de los sistemas, al final del bucle principal. Podemos hacerlo creando un nuevo tipo de entidad, que marcará las entidades que deben ser borradas. Llamaremos a este nuevo tipo ”muertas”, y crearemos una función del gestor de entidades que le añadirá este tipo a una entidad que reciba por parámetro.

```
DEF ENT_TIPO_MUE EQU %10000000

;Input - DE: Entidad a marcar
man_entidades_prepararBorrado::
```

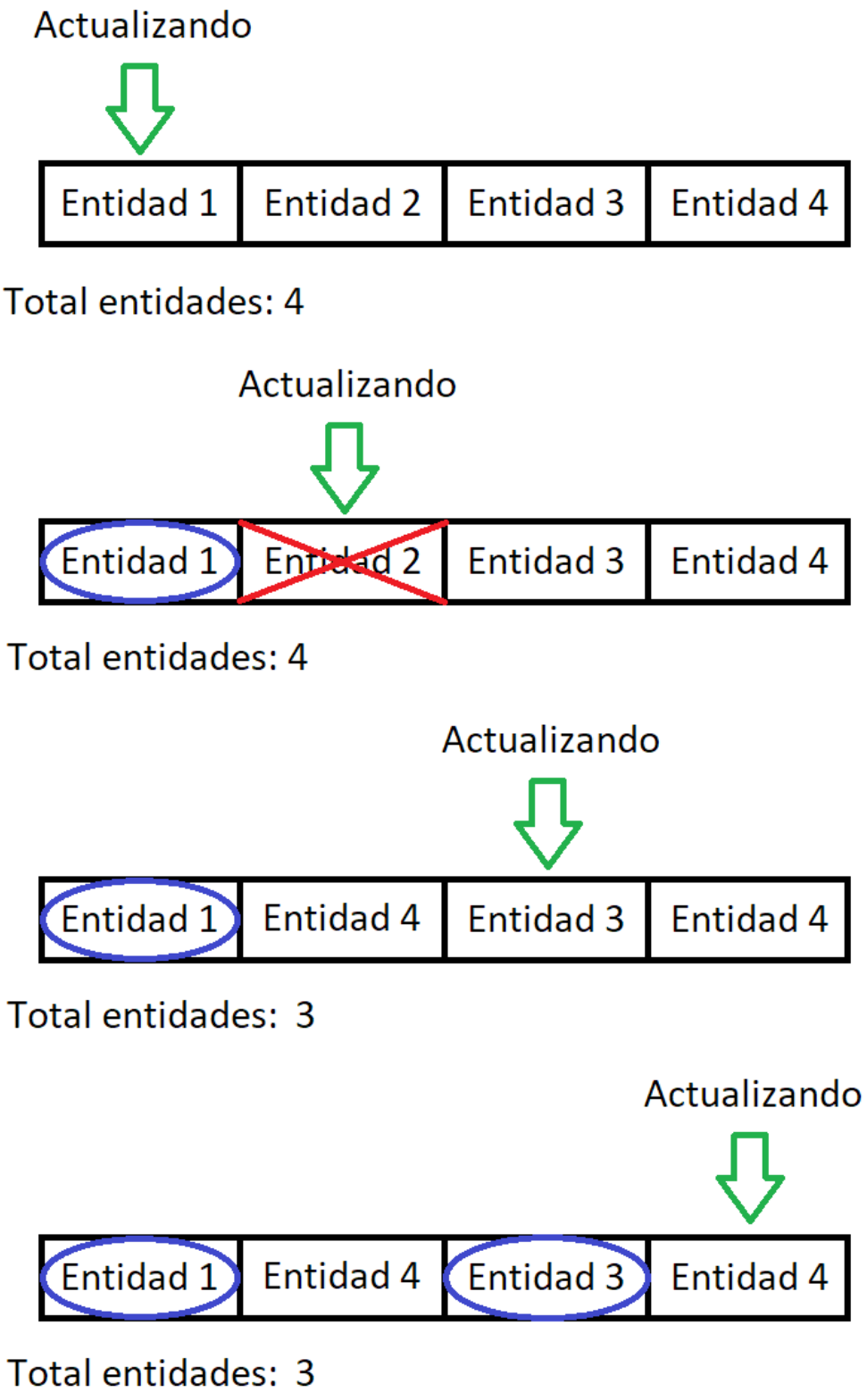



**Figura 4.20:** Borrado de entidades durante actualización (en azul las actualizadas).

```
    ld a, [de]    ;A = tipo de la entidad
    or ENT_TIPO_MUE ;Añade el tipo "muerta" a la entidad
    ld [de], a
ret
```

Esta función debe pertenecer al gestor de entidades, que es el que sabe cómo se representan. Cuando un sistema quiera que una entidad se borre, deberá usar esta función.

Ahora que tenemos diferenciadas las entidades a borrar, podemos crear una función similar a la que recorre todas las entidades actualizándolas, pero con la función de borrado. La diferencia es que al borrar una entidad, no se deberá avanzar el puntero al array, y el final del bucle se tendrá que controlar con la cantidad total de entidades en cada momento, no con la inicial.

Puedes comprobar que la rutina funciona creando varias entidades y dándole a algunas de ellas el tipo ”muertas”por defecto. Deberían borrarse todas ellas y el resto permanecer perfectamente.

## 4.14 Sistema de IA

Tenemos una entidad que hemos nombrado como enemigo, pero si simplemente se queda quieta, o no responde a nada, no tiene mucho de enemigo. Vamos a darle un sistema de comportamiento, o inteligencia artificial. Las entidades que usen este sistema tendrán como atributo una función de comportamiento, que será la que actualice sus velocidades. Además, esta función también será la encargada de marcar las entidades como para ser eliminadas.

En un principio nuestras entidades enemigas tendrás una posición inicial y velocidades definidas en su plantilla, no vamos a modificarlas en su función de comportamiento, solo vamos a eliminarlas si se salen de la pantalla, para que no reaparezcan por el otro lado. Tendrás que marcar las entidades como muertas si su posición X o Y se sale de la pantalla, para la posición X serían valores entre 168 y 248, y para Y cualquier valor mayor que 160.

Aunque pudiera parecer que el personaje del jugador no debería tener IA, vamos a darle también una función de comportamiento. Esta función debe comprobar si dada su velocidad actual en algún eje, el jugador se saldría por el borde de la pantalla, y de ser así debe cambiar esa velocidad a 0. Para que este cambio tenga sentido, el sistema de IA debe llamarse antes de las físicas. Esto sería más fácil implementarlo en el sistema de físicas, que ya calcula la suma de velocidad y posición, pero provocaría que ninguna entidad pudiera salirse por los bordes, y queremos que los enemigos lo hagan.

## 4.15 Gestor de partida

Ya tenemos casi todos los elementos que necesitamos para crear nuestro primer juego, pero vamos a necesitar un gestor que se encargue de iniciar las partidas y controlar cuando han terminado. La función principal del gestor debería llamar a otra de configuración para inicializar cualquier variable que necesite la partida, como un contador de tiempo o puntuación, y por ahora un bucle principal. Este bucle contendrá las llamadas a los sistemas que antes teníamos en la cabecera. Añade también una función que cree una entidad cada cierto número de frames, como por ejemplo 60. Por ahora no te preocupes de que el enemigo se mueva en direcciones

diferentes, solo asegúrate de que salen continuamente.

Al empezar el programa haz que aparezca un texto diciendo algo como *Pulsa A para empezar*. De esta forma la partida empezará cuando el jugador quiera.

### 4.15.1 Aleatoriedad

Queremos que los enemigos aparezcan en posiciones aleatorias de la pantalla, así que nos hará falta una forma de generar números aleatorios, o al menos, que den la sensación de serlo. El método de generación de números aleatorios que usaremos será un registro de desplazamiento con retroalimentación lineal, LFSR por sus siglas en inglés. Un algoritmo LFSR funciona desplazando todos los bits en una dirección y generando el valor del nuevo con una función que usa el estado anterior como parámetro. Dado que hay un número limitado de estados según los bits, este algoritmo entrará inevitablemente en un bucle, por lo que para dar la mayor sensación de aleatoriedad querremos usar una función para generar nuevos bits que pase por la mayor cantidad de estados posibles. Una de estas funciones para un registro de 16 bits consiste en sumar los bits 15, 13, 12 y 10, sumar a esto un uno, y guardar el resultado en el bit 0 tras el desplazamiento hacia la izquierda. La imagen 4.21 muestra un ejemplo de este proceso.

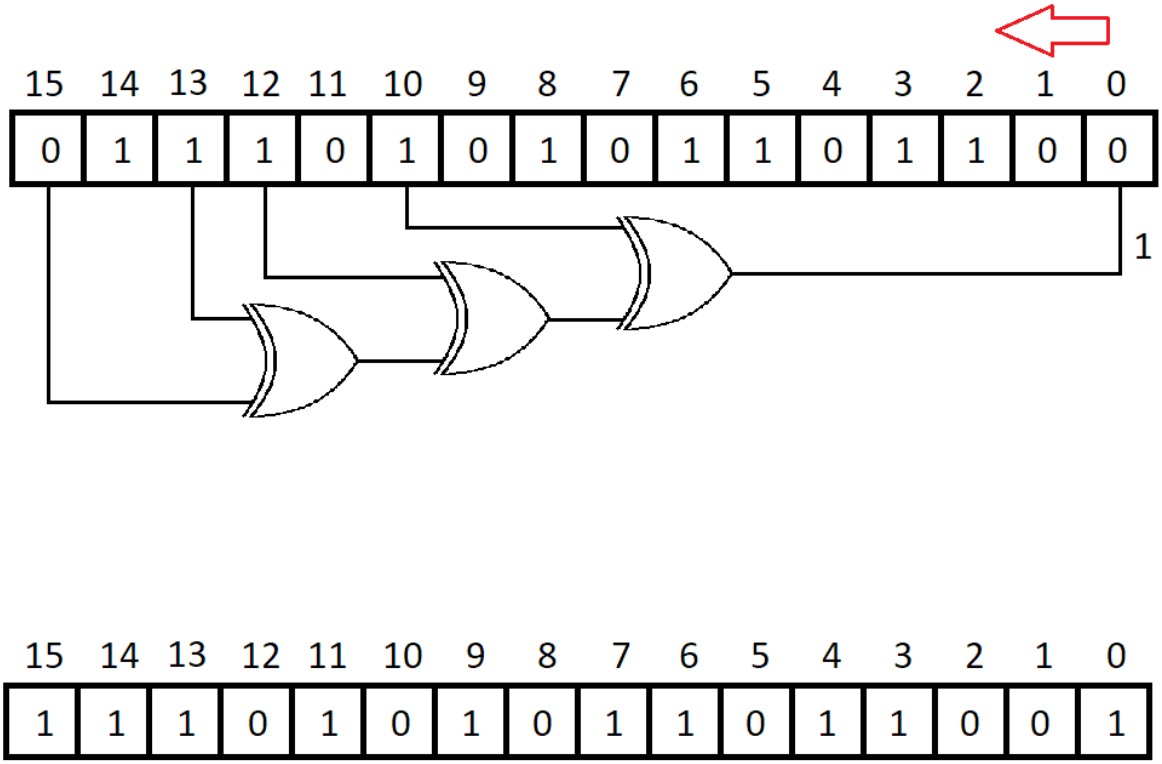



**Figura 4.21:** Generador de números aleatorios LFSR.

Este algoritmo es rápido y sencillo de aplicar gracias a ser reducible a varias operaciones XOR y de des-

plazamiento, que una CPU puede hacer fácilmente.[1]

Implementa este algoritmo en el juego. Cuando quieras obtener un número aleatorio sólo tienes que aplicar el algoritmo de rotación una vez y obtener el byte bajo. Un último detalle a tener en cuenta es que si el valor inicial de los 2 bytes es 0, nunca cambiará de valor, por lo que tenemos que inicializarlo con otro distinto. Puedes crear una variable que haga de contador global y aumente en uno a cada frame, al iniciar una partida, puedes hacer que tanto el byte alto como el bajo tengan el valor del contador, excepto si el contador en ese momento vale 0, que tendrás que elegir uno distinto. Ten en cuenta que esto no es un generador de números aleatorios real, una vez elegida la semilla, si se conoce, es posible predecir todos los número que generará. Los ordenadores no son capaces de producir aleatoriedad con algoritmos, por lo que a los número generados de este modo se les llama pseudoaleatorios, para el usuario dan la sensación de ser aleatorios, pero no lo son.

Ahora tenemos que hacer que cuando creemos una entidad en la partida, se cambien sus valores de posición en función del resultado, pero evitando que puedan aparecer de la anda en medio de la pantalla, pues no sería justo para el jugador. Esto deja como puntos en los que aparecer los cuatro bordes de la pantalla, por lo que podemos separar a los enemigos en 4 grupos de entidades, como muestra la figura 4.22.

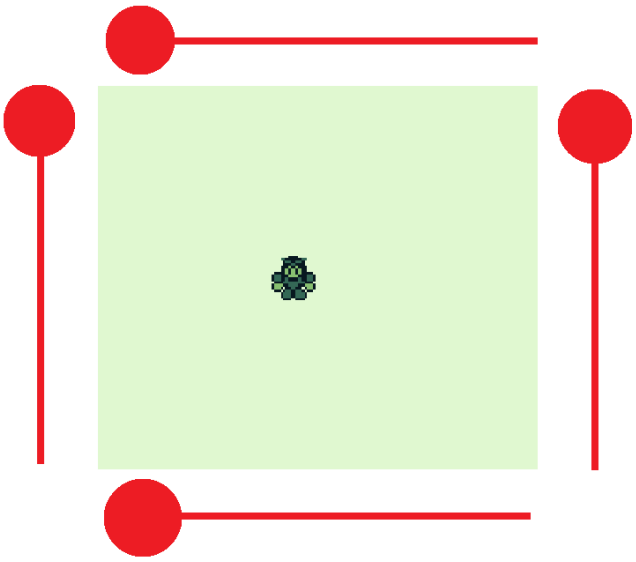


**Figura 4.22:** Puntos de aparición de los enemigos.

Podemos usar los dos bits más bajos del número aleatorio generado para decidir entre los 4 tipos de entidades, y después sumar el byte entero a la posición vertical u horizontal en la que empieza la entidad según en qué eje esté. Cada grupo de entidades debería tener solo velocidad horizontal o vertical, y en el sentido adecuado para que crucen por la pantalla.

### 4.15.2 Puntuación

Ahora vamos a hacer que los enemigos aparezcan con más frecuencia cuanto más tiempo pase. Al principio aparece uno cada 60 frames, y podemos hacer que el siguiente aparezca después de 59, el siguiente con 58, e ir

reduciendo los frames en uno cada vez, hasta llegar a un límite de 10. Ten en cuenta que si no has puesto una capacidad de entidades lo suficientemente alta, si aumenta mucho el ritmo de aparición, habrá veces en las que no se genere ninguna.

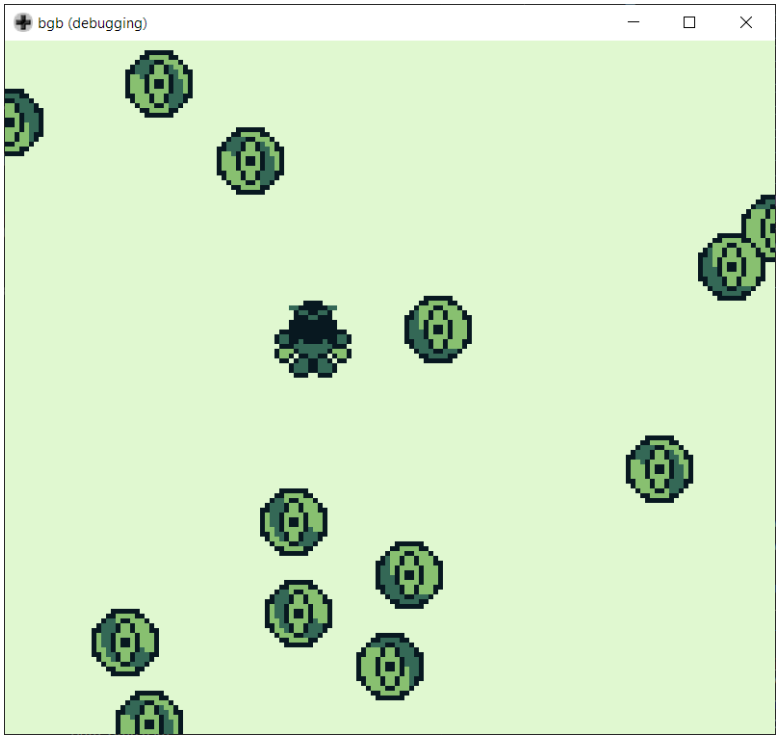



**Figura 4.23:** Partida con muchos enemigos

Añade un contador de puntuación que aumente periódicamente cada cierto número de frames. Por ahora no lo mostraremos en pantalla, pero la puntuación se seguirá contando internamente.

## 4.16 Colisiones

Para finalizar nuestro pequeño juego, tenemos que añadir colisiones para que los enemigos puedan matar al jugador. Los elementos que necesitamos añadir son:

- Un tipo de entidad colisionable (el jugador).
- Un tipo de entidad colisionadora (los enemigos).
- Un sistema de colisiones.
- Una función de colisión, para cuando se produce una.

Los dos tipos no son difíciles de añadir, aún nos quedan bits libres en le byte de tipos, elige dos de ellos.

El sistema de colisiones es algo distinto al resto de sistemas, ya que se deben comparar entidades dos a dos. En nuestro caso necesitamos comparar todas las entidades colisionables con todas las colisionadoras, por lo que una sola llamada al gestor de entidades no será suficiente. Si tratamos de reconstruir el proceso hacia atrás, las funciones que necesitaremos son:

- Comprueba colisiones: Recibe dos entidades como parámetros. Debe ser llamada con cada par de entidades.
- Aplica comprobar colisiones a todas las entidades de tipo colisionador: Se usa el gestor de entidades para llamar a la función anterior con todas las entidades de tipo colisionador. Se requiere haber recibido como

parámetro una entidad de tipo colisionable.

- Aplica función anterior a todas las entidades de tipo colisionable.

Como ves, necesitamos tres funciones para implementar el sistema, en vez de dos como en el resto. Cada una de las funciones intermedias selecciona un tipo distinto para que la final tenga dos entidades. Si no tienes suficientes registros para guardar ambas entidades, siempre puedes usar espacio en RAM. Por ahora crea la estructura aunque la función final no haga nada, y comprueba que se llama con cada par de entidades jugador-enemigo.

Ahora tenemos que comprobar si dos entidades colisionan. La figura 4.24 muestra las condiciones para determinar si dos entidades colisionan.

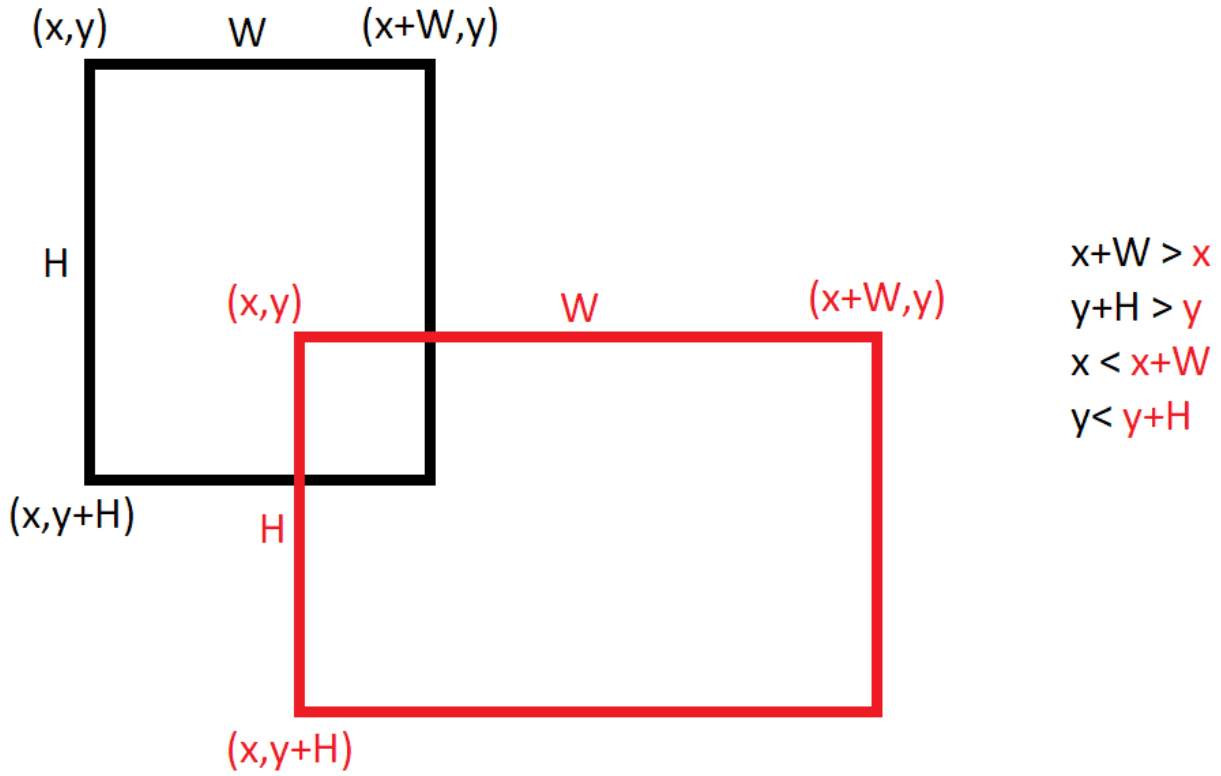



**Figura 4.24:** Condiciones de colisión entre dos entidades.

Los rectángulos negro y rojo representan dos entidades, x e y son sus coordenadas, coloreadas igual que los rectángulos a los que pertenecen, W es la anchura y H es la altura. Dos entidades colisionan si y solo si se cumplen todas las condiciones de la derecha. Suponiendo una anchura de 16 píxeles, implementa esta comprobación, y si se cumple haz que el programa se quede en un bucle infinito (`jr @`). Comprueba que el algoritmo funciona intentando colisionar con los enemigos, el programa debería quedarse atascado cuando suceda.

### 4.16.1 Reduce el rango de golpeo

Si has conseguido implementar las colisiones como se ha descrito, habrás notado que la colisión se registra antes de que las dos entidades parezcan colisionar. Esto se debe a que hemos supuesto que las entidades son cuadrados perfectos de 16 píxeles de lado, cuando las esquinas de nuestras entidades están vacías, por lo que se

detectan colisiones aún cuando solo se han tocado dos esquinas vacías.

Para solucionar esto tenemos que reducir el tamaño de las cajas colisionables, tenemos que cambiar la anchura y altura de las entidades. Cada entidad tendrá además unos valores de anchura y altura distintos, pues no tienen la misma forma, así que estos atributos habrá que añadirlos a las entidades. Elige unos valores de anchura y altura que se ajusten a los sprites de cada entidad. Por último, las cajas no deberían empezar en la esquina superior izquierda, porque está vacía también, por lo que a las posiciones X e Y de cada entidad habrá que sumarles un desplazamiento (offset) para que en el cálculo de colisiones se encuentren dentro de la parte no transparente del sprite. Estos desplazamientos, tanto en Y como en X también deben ser atributos de la entidad.

## 4.17 Final de la partida

Enhorabuena, ya tienen un juego sencillo terminado. En este juego el jugador debe esquivar a los enemigos para no perder, y su puntuación aumenta cuanto más tiempo aguante sin que le golpeen. Ahora podrías añadir más tipos de enemigos, con otros sprites y animaciones, o que se muevan en otras direcciones o con patrones distintos, pero que el final de la partida conlleve que la ejecución se atasque da un mal acabado al juego. Lo que debería suceder es que el juego vuelva al principio y se pueda empezar una nueva partida.

Vamos a usar un byte de RAM para representar el estado de la partida. Este estado puede ser cualquier cosa, desde un simple ”jugandoz ”fin de la partida.[a] contar las vidas del personaje, para que la partida no acabe con un solo golpe. En cualquier caso, se debe inicializar al empezar la partida. Entonces cuando haya una colisión entre el jugador y un enemigo, se modificará este valor según el uso que se le de, si es un contador de vidas se reducirá en 1, poniendo un límite de 0 por si da la casualidad de que el jugador colisiona con múltiples entidades evitar que llegue a valer 255, y si solo registra el estado de la partida, cambiarlo al valor de ”fin de la partida”. Esto se puede hacer directamente en la función que comprueba colisiones porque solo tenemos un tipo de colisión, si hubiera más tipos, como el jugador recogiendo monedas, o los enemigos con balas, necesitaríamos añadir una función de colisiones a las entidades que deberían ejecutar cuando se produjera una.

Una vez en el bucle principal de la partida, podemos usar una función que compruebe si esta ha terminado (0 vidas o ”fin de la partida”). Si es así, en vez de volver al principio del bucle, habría que salir de él y empezar una nueva partida, borrando primero todas las entidades. De esta forma se podrían jugar una nueva partida sin tener que reiniciar el juego.

Para dar los últimos detalles al juego, solo nos queda mostrar la puntuación obtenida al terminar una partida. Podemos hacer esto usando el sistema de textos. Sin embargo no podemos simplemente guardar un texto en ROM y mostrarlo al terminar la partida, porque la puntuación será distinta cada vez, lo que sí podemos hacer es guardar un texto como plantilla para el final, dejando hueco para la puntuación, y copiarlo en RAM. Al final de la partida se copiará la puntuación sobre los huecos que se han dejado, y entonces se mostrará el mensaje.

```
TextoFinPartida::
DB "Puntos obtenidos:", NEW_LINE_CHAR
DB "  ", END_LINE_CHAR ;Hueco libre de dos espacios para la puntuación.
```

```
TextoFinPartidaRAM::
    DS 18
.puntuacion
    DS 3
```

Ahora solo queda copiar el valor de la puntuación convertido a caracteres sobre los dos huecos libres y llamar a la función de escribir textos.

# Capítulo 5 Sonido

Nuestro juego de ejemplo ya está terminado, al menos la parte de las mecánicas, pero un juego sin efectos de sonido ni música queda muy pobre. Solamente poner efectos de sonido y una melodía básica le da al juego un acabado mucho mejor, y a ojos del jugador tendrá mucha más calidad.

En este capítulo crearemos un controlador de sonido, y un gestor que permita reproducir melodías. El controlador de sonido recibirá los datos y escribirá sobre los registros responsables del sonido, y el gestor será el encargado de pasarle los datos al controlador desde las estructuras de datos que formen las canciones.

## 5.1 Jugando con el sonido

La consola cuenta con un total de cuatro canales de audio. Todos ellos pueden estar activados simultáneamente, y se mezclan antes de enviar el sonido a los altavoces o auriculares. Para controlar el audio se usan una serie de registros de hardware, asociados a direcciones de memoria. Además de su dirección, a cada registro se le suele llamar por un nombre, teniendo este la forma **NRxy**, donde **x** indica el canal al que pertenece el registro, y toma valores del 1 al 4, mientras que **y** puede tomar valores entre 0 y 4, dependiendo del canal concreto. Además de los registros específicos de cada canal, existen tres registros que controlan todos los canales independientemente de sus registros específicos. Estos registros se llaman **NR50**, **NR51** y **NR52**.

- $FF24 - **NR50**: En caso de que la consola tenga auriculares conectados, el sonido se puede modificar para que se oiga más por un lado que por otro. Este registro controla esta función para todos los canales.
  - Bit 7: Canal 4 izquierda.
  - Bit 6: Canal 3 izquierda.
  - Bit 5: Canal 2 izquierda.
  - Bit 4: Canal 1 izquierda.
  - Bit 3: Canal 4 derecha.
  - Bit 2: Canal 3 derecha.
  - Bit 1: Canal 2 derecha.
  - Bit 0: Canal 1 derecha.

  Un 1 permite que el canal indicado pueda emitir sonido por el auricular correspondiente.

- $FF25 - **NR51**: Este registro controla el volumen global de cada auricular, independientemente del volumen indicado por cada auricular.
  - Bit 7: Un 1 permite la salida de audio por el auricular izquierdo.
  - Bits 6-4: Volumen izquierdo.
  - Bit 3: Un 1 permite la salida de audio por el auricular derecho.
  - Bits 2-0: Volumen derecho.

- $FF26 - **NR52**: Este registro tiene la función de indicar qué canales de audio están activos, y la capacidad

de activar o desactivar por completo el sonido.

- Bit 7: Escribiendo un 1 en este bit se activa el sonido. Escribiendo un 0 se desactiva.
- Bit 3: Indica si el canal 4 está activado (solo lectura).
- Bit 2: Indica si el canal 3 está activado (solo lectura).
- Bit 1: Indica si el canal 2 está activado (solo lectura).
- Bit 0: Indica si el canal 1 está activado (solo lectura).

Empezaremos creando una función que inicialice el sistema de sonido. Esta función debe primero borrar el contenido de los registros de todos los canales para evitar que se oiga cualquier sonido configurado anteriormente. Escribiremos un 0 sobre todas las direcciones de memoria entre $FF10 y $FF23. Después esta función escribirá $FF sobre **NR50** y **NR51** para permitir la salida de sonido por auriculares, y para terminar escribirá un 1 sobre el bit 7 de **NR52**. Con esto podremos empezar a reproducir sonidos por los canales de audio.

Por ahora vamos a ver únicamente como funciona el canal 2, que es el más simple. Este canal produce una onda cuadrada, que puede tener cuatro formas diferentes según la configuremos. Este canal usa cuatro registros de hardware, que tienen las siguientes funciones:

- $FF16 - **NR21**: Este registro controla la forma de la onda y el temporizador para la desactivación automática del canal. Los cuatro tipos de onda se diferencian en la proporción de tiempo en que la onda se encuentra activa en todo su periodo. A esto también se le llama ciclo de trabajo. Los bits 7 y 6 de este registro son los responsables del tipo de onda:

| Valor bits 7 y 6 | Ciclo de trabajo |
|---|---|
| 00 | 87,5 % |
| 01 | 75 % |
| 10 | 50 % |
| 11 | 25 % |

  No existe ninguna diferencia en el sonido producido por las ondas con ciclos de 75 % y 25 %.

  Los otros 6 bits indican el tiempo que tardará el canal en apagarse automáticamente, con valores más altos provocando antes la desactivación. En caso de activarse el temporizador del canal desde el registro **NR24**, el valor en estos 6 bits aumentará en uno a un ritmo de 256 hercios, y se apagará cuando llegue sobrepase el límite y vuelva a valer 0.

- $FF17 - **NR22**: Este registro sirve para controlar el volumen del canal, las funciones de cada bit son las siguientes:
  - Bits 7 a 4: Volumen inicial. Indica el volumen del canal, cuanto más alto sea este valor, más fuerte sonará. Un valor de 0 silencia el canal.
  - Bit 3: Este bit controla si el volumen aumentará o se reducirá en uno cuando se cumpla la condición indicada por los bits 2 a 0. Si este bit tiene un valor de 0, el volumen se reducirá, y aumentará con un 1.
  - Bits 2 a 0: Ritmo de modificación. Un contador interno dedicado a la modificación del volumen aumenta a un ritmo de 64 hercios. Cuando este contador alcanza el valor de estos tres bits, se aplica la modificación del volumen y se reinicia el contador. Si el ritmo de modificación tiene un valor de 0, se desactiva esta funcionalidad.

Cualquier cambio a este canal no tendrá efecto hasta que se reactive usando el registro **NR24**.

- $FF18 - **NR23**: Este registro contiene los 8 bits bajos, de un total de 11 que representan un valor al que llamaremos periodo. Cuando se habla de ondas, el periodo es el valor inverso de la frecuencia, este valor no representa el periodo, pero está relacionado con él. La frecuencia de la onda producida se puede calcular usando la siguiente fórmula:

$$Frecuencia = \frac{131072}{2048 - periodo} \tag{5.1}$$

  Donde *periodo* es el valor de los 8 bits de este registro junto con 3 bits de **NR24**. Cuanto más alto sea el valor que hemos llamado periodo, mayor será la frecuencia.

- $FF19 - **NR24**: Este registro incluye los bits altos del periodo, y la capacidad de activar el canal y el temporizador.
  - Bit 7: Si este bit está a 1, escribir cualquier valor en este registro activa el canal, esto incluye escribir el 1 sobre este bit.
  - Bit 6: El temporizador del registro **NR21** se activará al escribir un 1 sobre este bit.
  - Bits 2 a 0: Bits altos del valor del periodo.

No existe el registro **NR20**.

Toda esta información puede ser muy confusa vista de golpe, así que vamos a ver cada elemento uno a uno.

### 5.1.1 Frecuencia

Empezaremos por probar a variar la frecuencia del. Crea un programa de prueba que inicie el canal 2 escribiendo un 1 en el bit 7 de **NR24**. Antes de esto tenemos darle volumen al canal con **NR22**, sube el volumen al máximo pero no actives la modificación automática de este (escribe un $F0). Entonces crea un bucle en el que aumentes un registro (como A) cada cierto tiempo, y guardes ese valor en **NR23**. Si ejecutas el programa, deberías escuchar como el sonido se vuelve más agudo poco a poco, hasta que el registro se desborde y vuelva a sonar grave. Si quieres hacer sonar notas específicas puedes buscar sus frecuencias y resolver la ecuación para el valor del periodo. No olvides que los 3 bits altos de este valor se guardan en los 3 bits bajos de **NR24**.

### 5.1.2 Forma de onda

Ahora vamos a probar las cuatro formas de onda de las que dispone el canal. Para cambiar de una a otra simplemente debes escribir los valores adecuados en los bits 7 y 6 de **NR21**. Al cambiar la forma de la onda también cambia el timbre de esta, aunque siendo todas las ondas cuadradas, no hay mucha diferencia. Comprueba que efectivamente las ondas con un 75 % y 25 % de ciclo de trabajo suenan exactamente igual.

### 5.1.3 Temporizador

Vamos a ver ahora el uso del temporizador. Este se puede usar para fijar la longitud de las notas sin tener que volver a escribir sobre los registros para apagar el canal, o puede usarse justamente para apagar el canal. Si has inicializado el audio como se indica al principio de la sección, **NR21** contendrá ceros en los bits 5 a 0, dando el tiempo máximo hasta que el canal se apague. Escribe un 1 en el bit 6 de **NR24** para activar el temporizador y comprueba cuanto tarda en apagarse el canal tras empezar a sonar. Apenas debería llegar a un cuarto de segundo.

Si inicias el temporizador con todos los bits a 1, casi no dará tiempo a escucharse ningún sonido. Puedes usar esta característica en conjunto con bajar el volumen del canal para apagarlos sin causar pops de audio, que son bastante habituales si se desactiva un canal de sonido sin quitarle antes el volumen.

# Capítulo 6 Hagamos un Sokoban

El resto de este libro se enfocará en la creación de un juego concreto, que se usará como vehículo para explicar el resto de funcionalidades de la consola. Aún si este no es el tipo de juego que quieres programar, te recomiendo que te quedes durante el principio de este capítulo, pues aprenderás a dibujar fondos para tus niveles.

## 6.1 Metatiles

Hasta ahora todo lo que hemos llegado a dibujar en los Tile Maps han sido letras para formar textos, ahora haremos fondos de verdad, que representen habitaciones o paisajes. Pero primero tenemos que tratar el problema del espacio. Como sabrás, cada cuadrado de 8 por 8 píxeles se identifica con un byte, por lo que si quisiéramos almacenar toda la información de una pantalla, necesitaríamos 360 bytes (20 cuadrados de ancho por 18 de alto), si quisiéramos tener múltiples niveles nos quedaríamos sin espacio enseguida. De base la consola cuenta con 32 kilobytes de ROM, si usáramos todo el espacio para guardar niveles del tamaño de una pantalla, sólo tendríamos espacio para 88 niveles, y a eso hay que restar el espacio ocupado por el código o los propios gráficos. La mayoría de juegos que cargan pantalla a pantalla tiene al menos 256 niveles distintos, aunque es cierto que suelen disponer de más memoria ROM, pero también tienen más cantidad de gráficos.

Vamos a ver una forma de reducir el espacio que ocupa una pantalla hasta 90 bytes, cuatro veces menos que guardando todos los bytes individuales. Este método consiste en crear unas estructuras llamadas metatiles. Cada metatile representa un conjunto de cuatro tiles, y por tanto ocupa cuatro bytes, pero podemos representar una pantalla completa usando 90 de ellos. Si identificamos cada metatile con un solo byte, podemos almacenar la información de un nivel entero en solo 90 bytes. Es cierto que los metatiles ocupan un espacio añadido, pero este no se acerca a los 360 bytes por nivel respecto a no usarlos, y el coste por nivel decrece cuantos más creemos con los mismos metatiles.

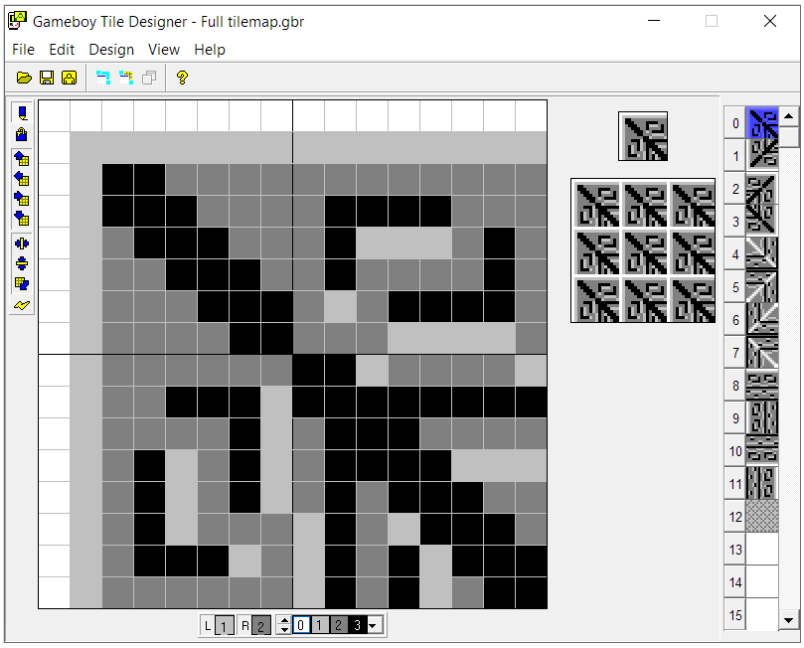



**Figura 6.1:** Conjunto de metatiles.

Para crear metatiles, abre GBTD y elige el tamaño de tile de 16x16. Ahora diseña un conjunto de tiles que formen lo que serán paredes y el suelo. En total deberías tener 13 metatiles, 4 paredes, 8 esquinas (4 hacia dentro y 4 hacia fuera), y un suelo. Puedes crear más metatiles si quieres tener más variedad de escenarios. Una vez hayas terminado de diseñar tiles, usa la opción *Tile count...* de la pestaña *View* e introduce la cantidad de tiles que has diseñado. Esto servirá más adelante para reducir el espacio necesario para identificar un tile. Guarda este archivo y expórtalo. En las opciones de exportación, haz clic en la pestaña *Advanced* y marca la casilla *Convert to metatiles*.

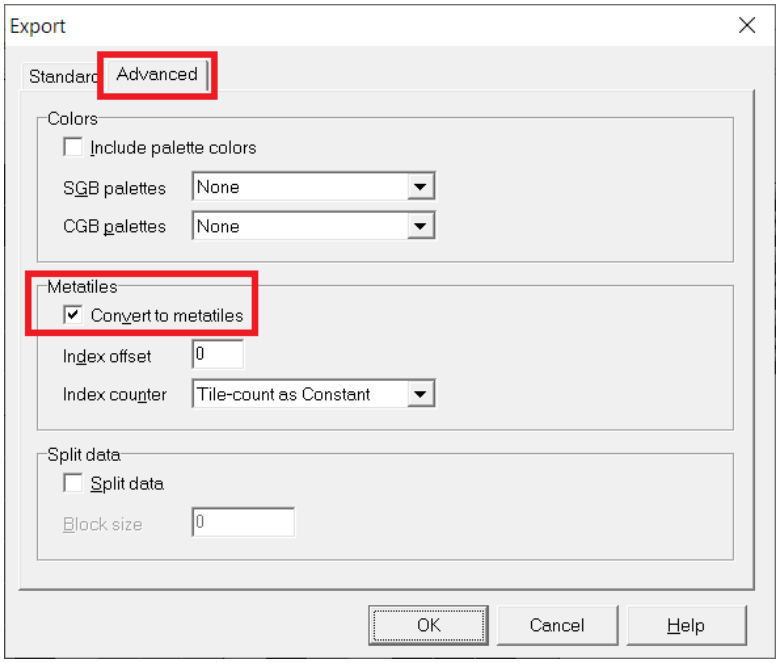



**Figura 6.2:** Opciones de exportación de metatiles.

Si abres el archivo creado, verás que además del conjunto con la información de los tiles, que tendrá la etiqueta que hayas elegido al exportar, hay otro bloque con una etiqueta igual más `Index` al final, como en la figura 6.3.

```
; Metatile index
WallsAndFloorsIndex::
DB $00,$01,$02,$03,$01,$04,$05,$06
DB $02,$07,$08,$09,$0A,$06,$09,$0B
DB $0C,$0D,$0E,$0F,$10,$11,$12,$0E
DB $13,$14,$15,$0D,$16,$13,$10,$17
DB $02,$0D,$02,$0D,$01,$01,$0E,$0E
DB $10,$06,$10,$06,$13,$13,$09,$09
DB $18,$18,$18,$18
```

**Figura 6.3:** Índice de metatiles.

Mientras que el otro conjunto contiene toda la información de los tiles, este contiene la de los metatiles. Los cuatro primeros bytes pertenecen al metatile 0, los siguientes cuatro al 1, y así hasta el último. Además, GBTD detecta automáticamente si entre los metatiles hay tiles iguales, y en vez de duplicar su información en los datos, les asigna el mismo identificador en la definición del metatile. Es por esto que a pesar de haber creado 13 metatiles, que supondrían un total de 52 tiles, viendo el índice de estos, solo hay 27 distintos, del $00 al $18. Si quieres ahorrar más memoria en la creación de metatiles, solo tienes que hacer que los tiles individuales de los que están formados se repitan.

Si miramos la figura 6.5, podrás ver que Tile Data está dividido en tres secciones distintas, a las que llamaremos bancos. El superior es el banco 0, el intermedio el 1, y el inferior el 2. La representación del emulador también está dividida por la mitad verticalmente, y la sección derecha representa un segundo banco de VRAM del que dispone la Game Boy Color, que no trataremos, por lo que podemos ignorarlo.

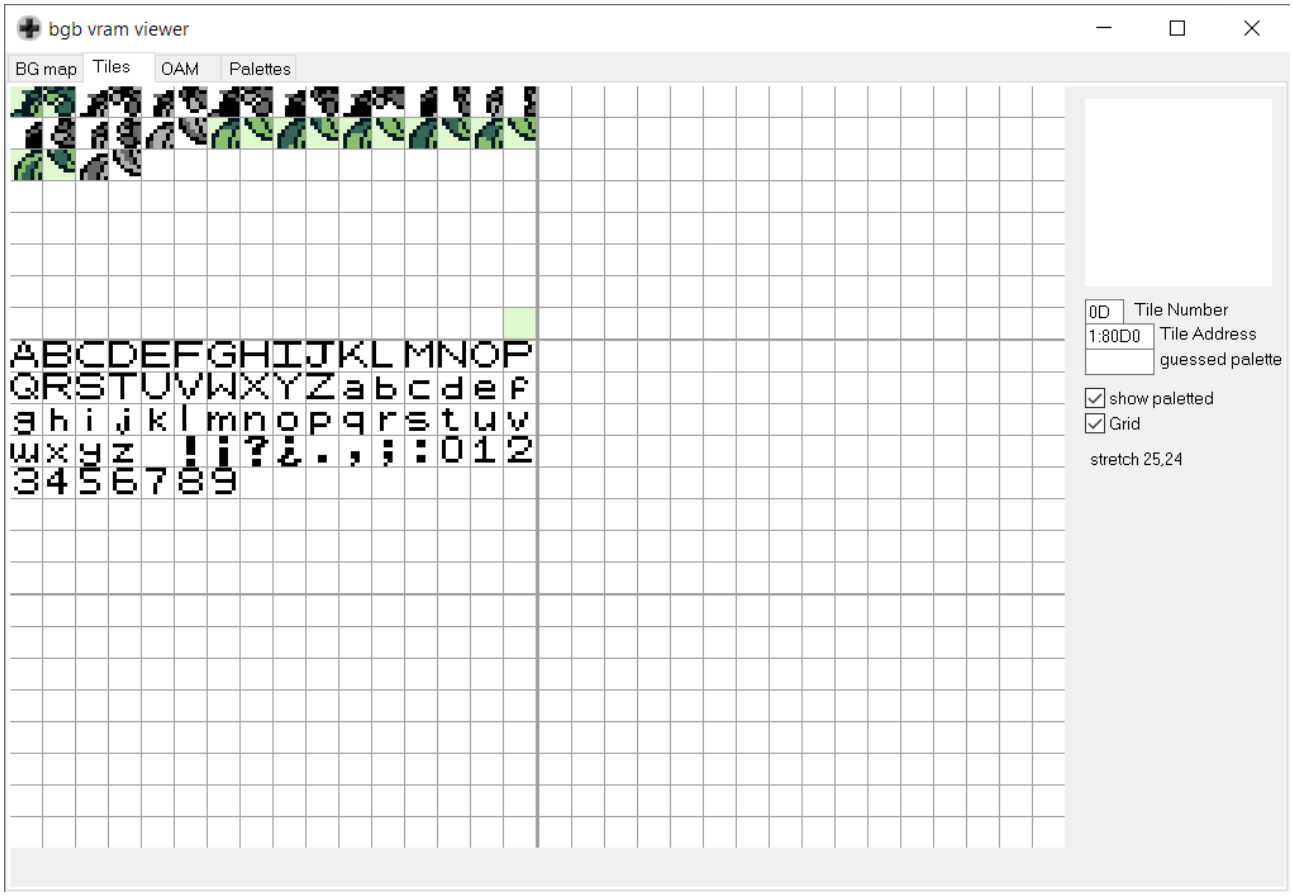



**Figura 6.4:** Uso de Tile Data.

Hasta ahora hemos usado los bancos 0 y 1 de Tile Data, pero no hemos visto como usar el banco 2, a fin de cuentas, los bancos 0 y 1 en conjunto ya suman 256 tiles, la cantidad de identificadores diferentes que se pueden representar usando un byte. Para ver cómo usar el banco 2, vamos a ver en detalle el registro de control de LCD.

- Bit 7 - Activar LCD y PPU: 0 = desactivada; 1 = activada.
- Bit 6 - Tile Map de la Ventana: 0 = $9800-$9BFF; 1 = $9C00-$9FFF.
- Bit 5 - Activar Ventana: 0 = desactivada; 1 = activada.
- Bit 4 - Tile Data del Fondo y Ventana: 0 = $8800-$97FF; 1 = $8000-$8FFF.

- Bit 3 - Tle Map del Fondo: 0 = $9800-$9BFF; 1 = $9C00-$9FFF.
- Bit 2 - Tamaño de sprites: 0 = 8x8; 1 = 8x16.
- Bit 1 - Activar sprites: 0 = desactivados; 1 = activados
- Bit 0 - Activar Fondo y Ventana: Este bit tiene una función distinta en Game Boy Color que en Game Boy original. Cuando está a 0, el fondo y la ventana se vuelven completamente blancos, independientemente del bit 5. Los sprites siguen siendo visibles si estuvieran activados.

Si te fijas en el bit 4, este sirve para seleccionar qué bancos de Tile Data usan el fondo y la ventana. Por defecto este bit está a 1, y se usan los bancos 0 y 1, pero se puede usar el banco 2 en lugar del 0 cambiando este bit a 0. Dado que los sprites solo pueden usar los bancos 0 y 1, vamos a dejar el banco 0 únicamente con sprites, y le asignaremos al fondo y ventana los bancos 1 y 2.

Cambia el banco en la configuración inicial, y carga los tiles en el banco 2 de Tile Data. Puedes comprobar que se han cargado los gráficos correctamente mirando la VRAM en el emulador. Además de estar los tiles de las paredes en la sección inferior, el último tile ahora estará en verde, porque es el que se está usando para dibujar el fondo vacío.

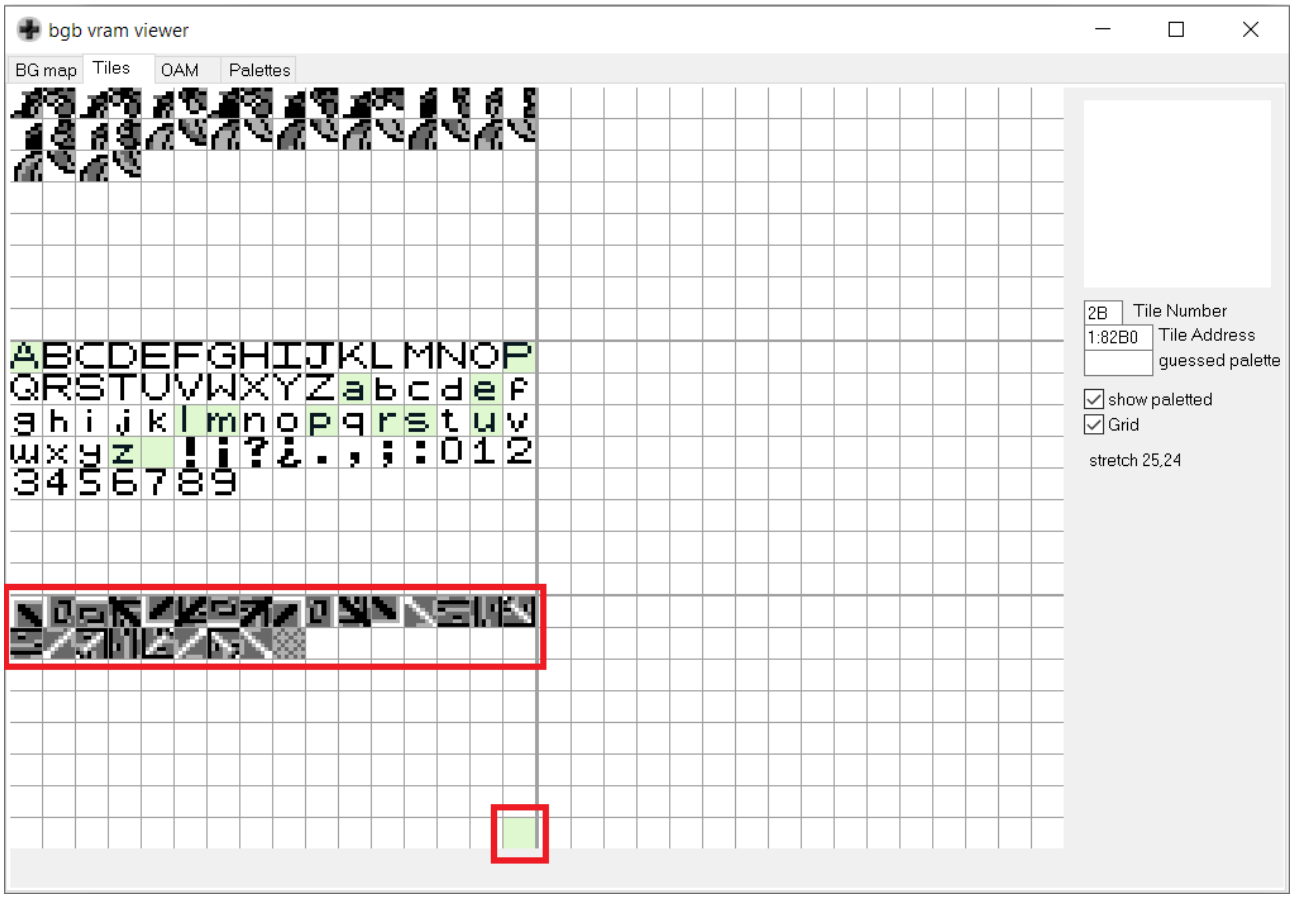



**Figura 6.5:** Tiles de metatiles cargados.

Si te fijas en la ordenación, podrás comprobar como los valores guardados en el índice de los metatiles de la figura 6.3, coinciden con los identificadores de los tiles de cada uno una vez guardados en RAM. Si quisieras guardar los tiles en otra región, por lo que los tiles tendrían identificadores que no empezaran en 0, en las opciones de exportar, indica el valor del primer identificador en *Index offset*.

Solo nos queda un inconveniente que abordar antes de dibujar el primer metatile, y es que cuando se exportan los tiles, el orden en el que se escriben no coincide con el orden secuencial que tendrían en memoria al dibujarse como metatiles. Esto sucede porque el algoritmo de exportación está diseñado pensando en los sprites de tamaño 8x16, que usan esa ordenación. La imagen 6.6 ilustra mejor el problema, en el ejemplo de que quisiéramos dibujar el metatile 0 en la dirección $9800.

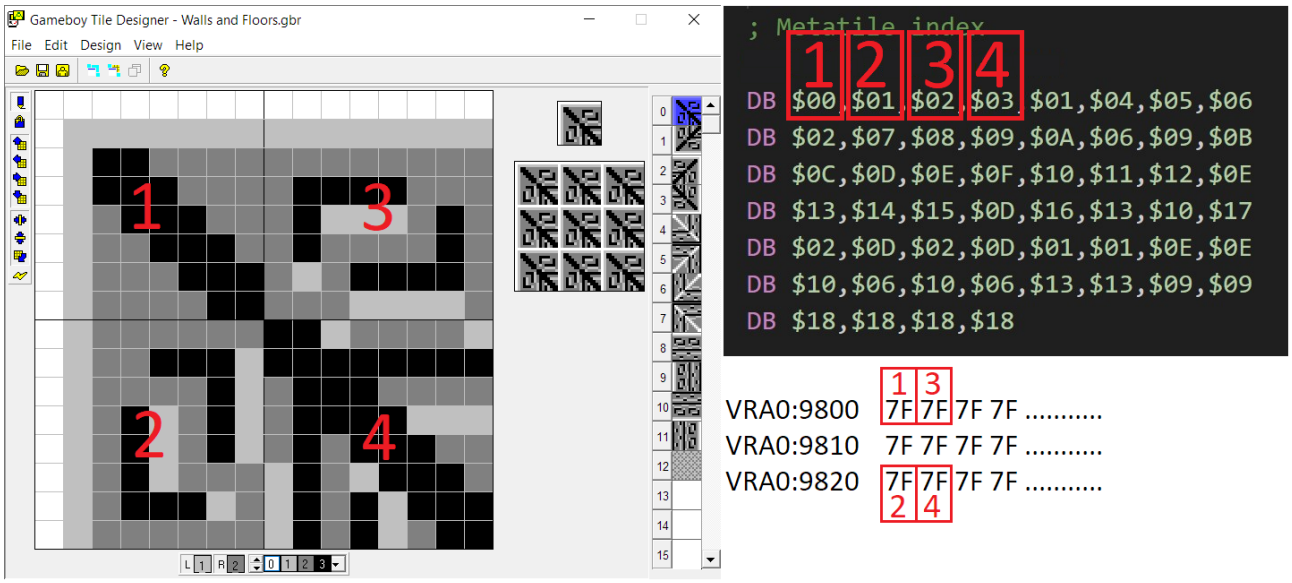



**Figura 6.6:** Equivalencia entre tiles de un metatile, y sus posiciones en VRAM.

Si quisiéramos escribir en VRAM los tiles tal y como están ordenados en memoria, tendríamos que estar saltando adelante y atrás antes de dibujar cada tile. Este inconveniente puede ser mitigado cambiando el orden del segundo y tercer tile de cada metatile, pero hacer esto a mano para cada uno es muy tedioso, y es propenso a errores. Una mejor forma de hacer esto sería creando un programa que reordenara los tiles como quisiéramos, pero podemos hacer lo mismo con una macro, y ahorrarnos complicaciones:

```
MACRO ReordenaMetatiles
    ASSERT _NARG%4 == 0
    REPT _NARG/4
        DB \1, \3, \2, \4
        SHIFT 4
    ENDR
ENDM

WallsAndFloorsIndex::
    ReordenaMetatiles \
    $00,$01,$02,$03,$01,$04,$05,$06 ,\
    $02,$07,$08,$09,$0A,$06,$09,$0B ,\
    $0C,$0D,$0E,$0F,$10,$11,$12,$0E ,\
    $13,$14,$15,$0D,$16,$13,$10,$17 ,\
    $02,$0D,$02,$0D,$01,$01,$0E,$0E ,\
    $10,$06,$10,$06,$13,$13,$09,$09 ,\
    $18,$18,$18,$18
```

Esta macro comprueba que haya una cantidad de datos múltiplo de 4, ya que cada metatile debe estar compuesto de 4 tiles, y los escribe todos intercambiando las posiciones 2 y 3 de cada uno. De esta forma los tiles estarán ordenados para que sea más fácil dibujarlos.

Por último guarda la paleta para el fondo y la ventana del mismo modo que para los objetos, pero en la dirección $FF47, que es la que usan.

Ya lo tenemos todo listo para dibujar el primer metatile, vamos a crear una función que reciba como parámetros la dirección del metatile a dibujar, y la dirección de memoria de vídeo del primer tile de ese metatile. La dirección del metatile es la que apunta al principio de los identificadores de los cuatro tiles que lo forman. En nuestro caso, la dirección del metatile 0 es `WallsAndFloorsIndex`. Vamos a suponer que ningún metatile estará cortado por los bordes de los Tile Maps.

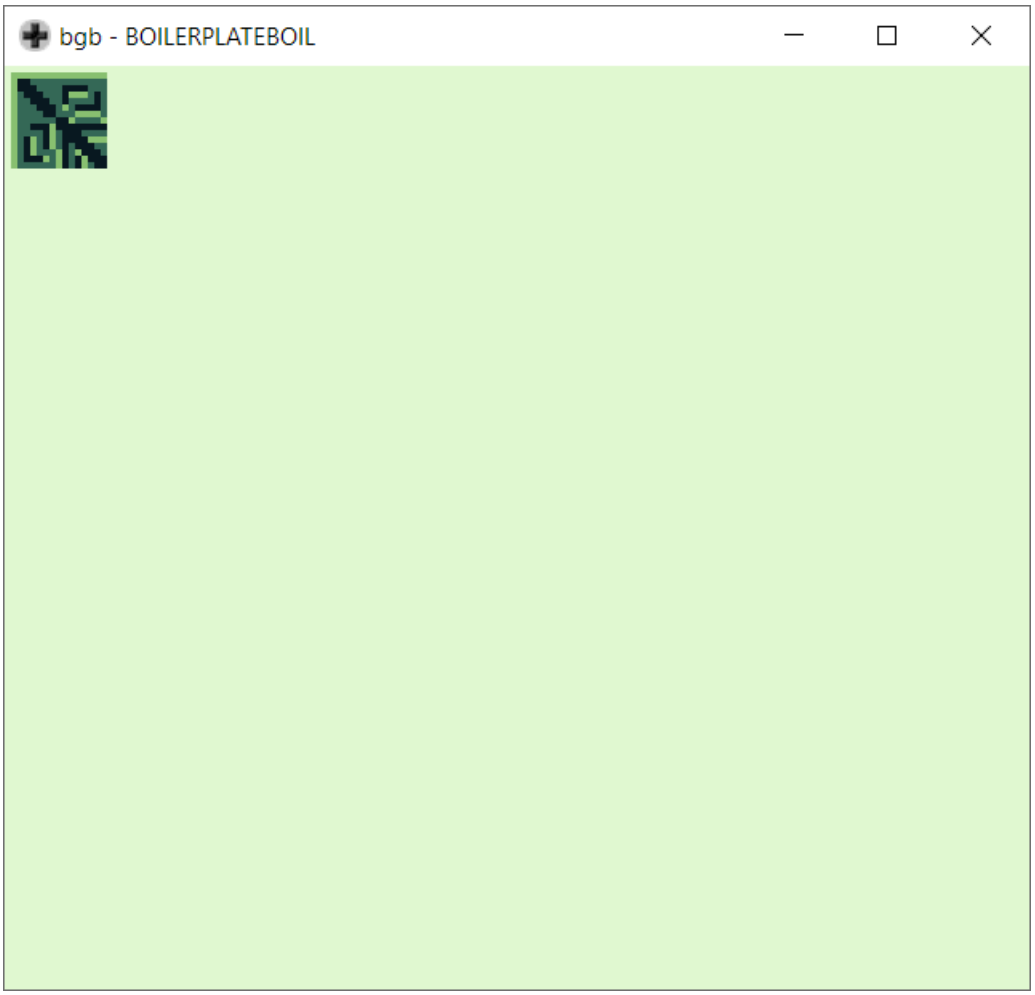



**Figura 6.7:** Metatile dibujado en la esquina superior izquierda.

## 6.2 Tilemaps

Un tilemap (no confundir con los Tile Maps, las regiones de memoria en las que se dibujan el fondo y la ventana), es una estructura que representa un dibujo, que puede ser un nivel, un escenario o cualquier cosa que queramos usar, a través del conjunto de tiles que lo forman. Nuestros tilemaps estarán hechos de los metatiles que hemos creado, y por tanto cada byte que forma el tilemap corresponderá a un metatile.

Para crear un tilemap, abre Game Boy Map Builder (GBMB), un programa que aún no hemos usado, pero que se instaló junto con GBTD. Al inciar el programa, este solo muestra un mensaje que indica como cargar un tileset, el conjunto de metatiles que hemos creado. Para cargarlo sigue las instrucciones haciendo clic en *File* y luego *Map properties...*. Se abrirá una ventana en la que podrás elegir la anchura y altura del tilemap, y el archivo donde están los tiles. El tilemap que vamos a crear ocupará solo el tamaño de la pantalla, que es de 160 píxeles de ancho por 144 de alto, y los tiles son de 16 píxeles de ancho y alto, por lo que el tilemap tendrá 10 tiles de ancho y 9 de alto. La figura 6.8 muestra cómo seleccionar y crear el tilemap inicial.

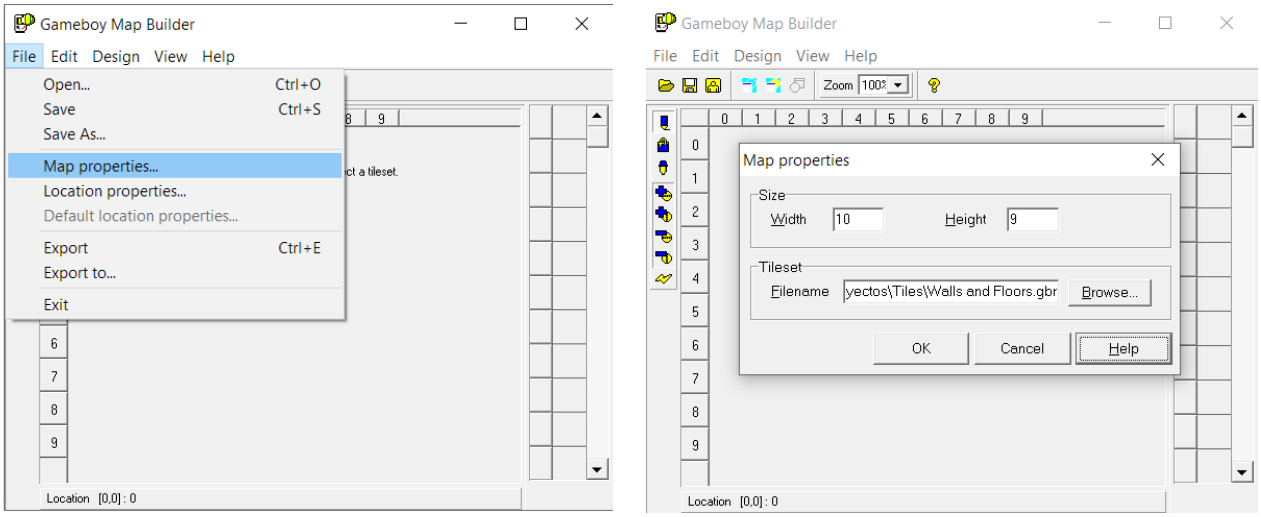



**Figura 6.8:** Cargar un tileset en GBMB.

La interfaz de GBMB es similar a la GBTD, a la izquierda se encuentran las herramientas de dibujado, y en el centro está el lienzo, pero esta vez no hay varios lienzos, y en vez de seleccionar colores en la sección inferior, la sección derecha sirve para seleccionar el tile que dibujar. Los tiles se dibujan con el clic derecho, y el clic izquierdo sirve para seleccionar regiones del tilemap. Diseña el mapa que quieras, puedes hacer que el nivel tenga sentido o no, lo importante es que cuando lo dibujes se vea correctamente, así que intenta usar muchos metatiles distintos.

Ahora vamos a exportar el mapa, haciendo clic en *Export to...* en la pestaña *File*, y rellenamos los campos como se muestra en la figura 6.10.

En la pestaña *Standard* solo tenemos que introducir los campos habituales del nombre de archivo, etiqueta y sección, eligiendo como tipo de exportación RGBDS. En la pestaña de *Location format*, en el desplegable de la izquierda, elige la opción **[Tile Number]**, debería aparecer un 4 a su derecha si no tienes más de 16 metatiles. Si aparece un 5, significa que tienes entre 17 y 32 tiles en el tileset, bien porque decidiste diseñar más, o bien porque no redujiste el contador de tiles cuando los diseñaste. En la sección de la derecha, en la opción *Plane count*, elige **1 plane (8 bits)**.

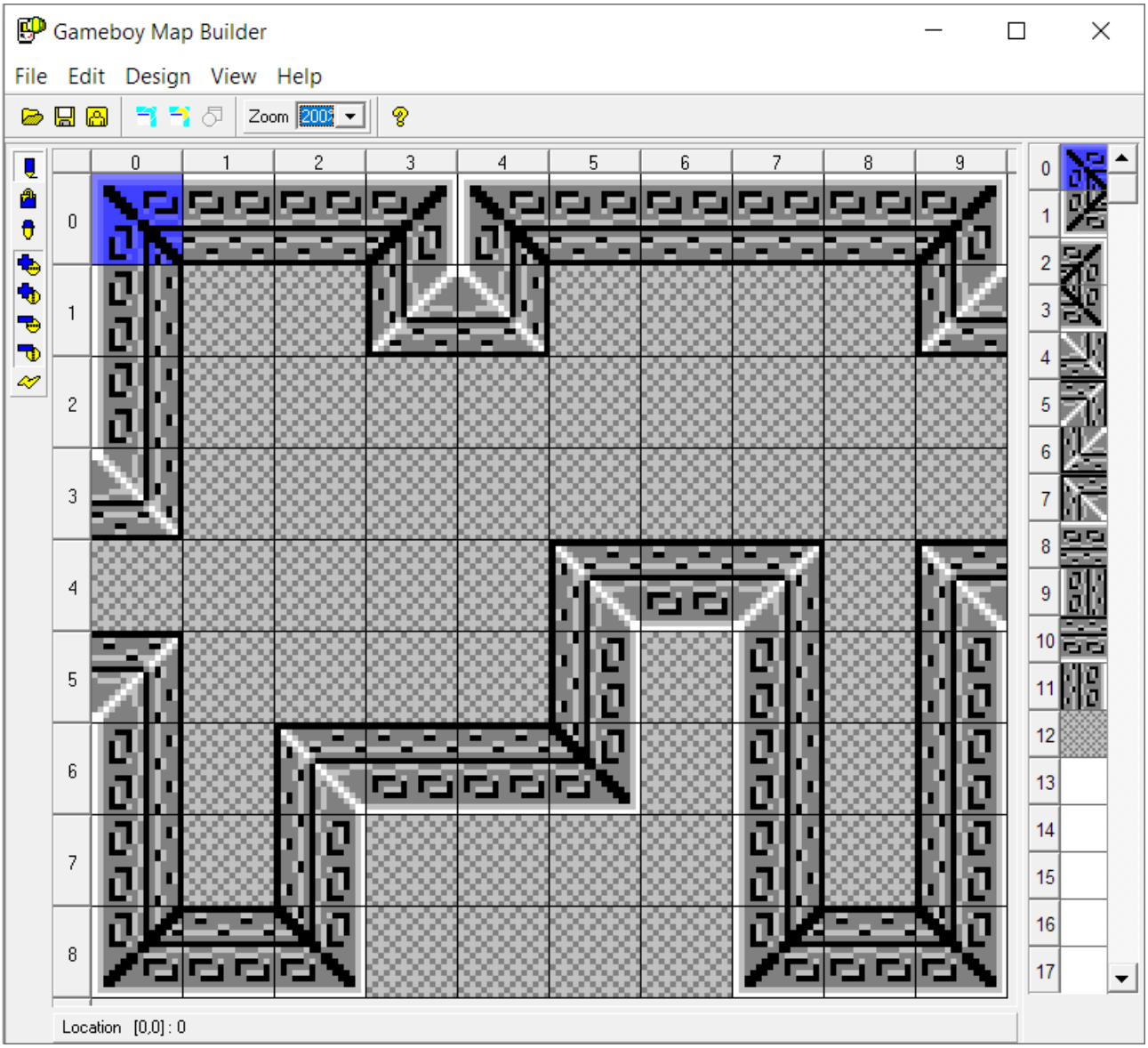



**Figura 6.9:** Mapa creado en GBMB.

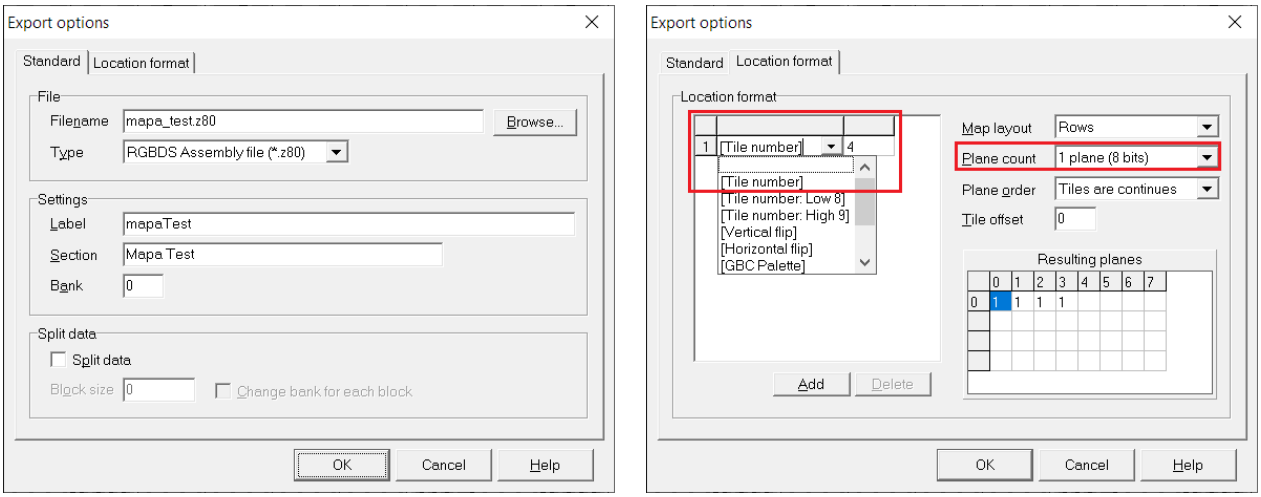



**Figura 6.10:** Exportar un tilemap en GBMB.

La sección izquierda sirve para elegir la información que se quiere exportar de cada tile, mientras que la sección inferior derecha de esta pestaña muestra como se distribuirá dicha información. Ahora mismo hemos elegido que se exporte como propiedad 1 el número de cada tile, que ocupa 4 bits (o 5, si tienes 32 distintos). En la sección del resultado, vemos que hay ocho casillas de la cuadrícula con índices encima y a su izquierda, y cuatro de esas casillas tienen un 1. Esto significa que cada tile se exportará como un byte, y que los bits 0 al 3 de cada byte representan la propiedad 1 (el número de tile). Podríamos cambiar la selección del número de planos para exportar 4 bits por tile en vez de 8, y ahorraríamos la mitad de memoria, aunque eso complicaría la lógica a la hora de dibujar, pero en el futuro añadiremos más tiles distintos, y más propiedades que exportar, como qué casillas se pueden pisar y cuales no, por lo que necesitaremos más de 4 bits por cada tile. Deja el resto de opciones en sus valores por defecto.

Añade el archivo exportado al proyecto y cámbiale la extensión a *.asm*.

Para dibujar el tilemap, deberás recorrerlo y usar su contenido para obtener las direcciones de los metatiles. Como cada metatile ocupa 4 bytes, para obtener la dirección de cada metatile a partir de su id tendrás que multiplicarlo por 4, y luego sumarlo a la dirección del índice de metatiles. El algoritmo para dibujar un tilemap completo es el siguiente:

```
puntero a tilemap
puntero a memoria de vídeo

inicio bucle
    obtén de de metatile del puntero a tilemap
    puntero a metatile = id x 4 + inicio índice metatiles
    dibuja metatile
    actualiza punteros
fin bucle
```

Implementa la función para dibujar un tilemap a partir de su dirección y de la dirección de memoria de vídeo donde irá el primer metatile a dibujar, y úsala para dibujar el tilemap que hemos creado en la pantalla.

### 6.2.1 Transiciones entre tilemaps

Para terminar esta sección vamos a hacer un método para cambiar el nivel actual por otro distinto. La forma más sencilla de cambiar el nivel sin que al jugador le parezca que ha cambiado de un frame a otro es hacer un fundido a blanco o a negro antes de cambiar los gráficos, y después deshacer el fundido. De esta forma no se ve como se cambia el fondo. Más adelante se explicará otro método de cambiar entre niveles desplazando la vista de uno a otro, para dar una sensación de continuidad.

Para hacer un fundido solo tenemos que cambiar la paleta de colores gradualmente. Antes del fundido, lo más probable es que la paleta tenga asignados los colores negro, gris oscuro, gris claro y blanco como los colores 3, 2, 1 y 0 respectivamente. Cuando hagamos el fundido habrá que cambiar la paleta del fondo (en $FF47) poco a poco, haciendo cada color de hardware un paso más claro o más oscuro, según se quiera hacer un fundido a blanco o a negro. La figura 6.11 como cambia la paleta durante un fundido a blanco, y 6.12 el efecto en el juego.

Para deshacer el fundido, solo hay que hacer el cambio inverso. Ahora puedes crear un segundo mapa, cargar uno de ellos y hacer el cambio usando el fundido. Crean una función que haga todo esto recibiendo como

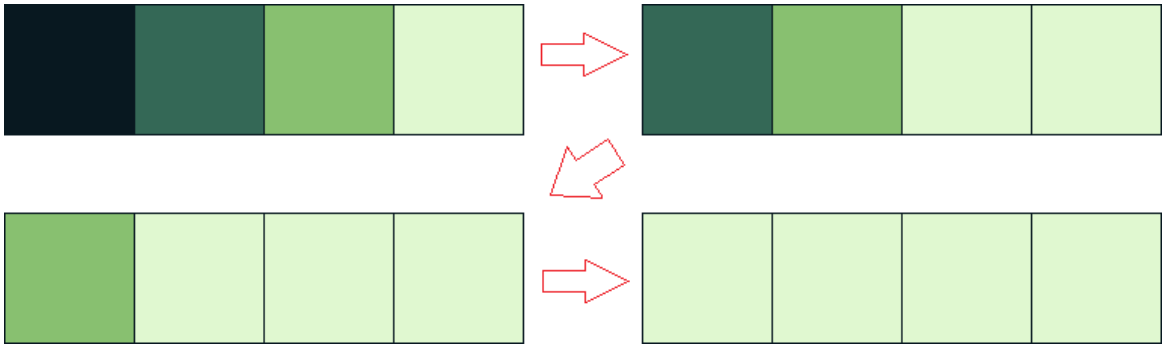


**Figura 6.11:** Cambio de paleta durante un fundido a blanco

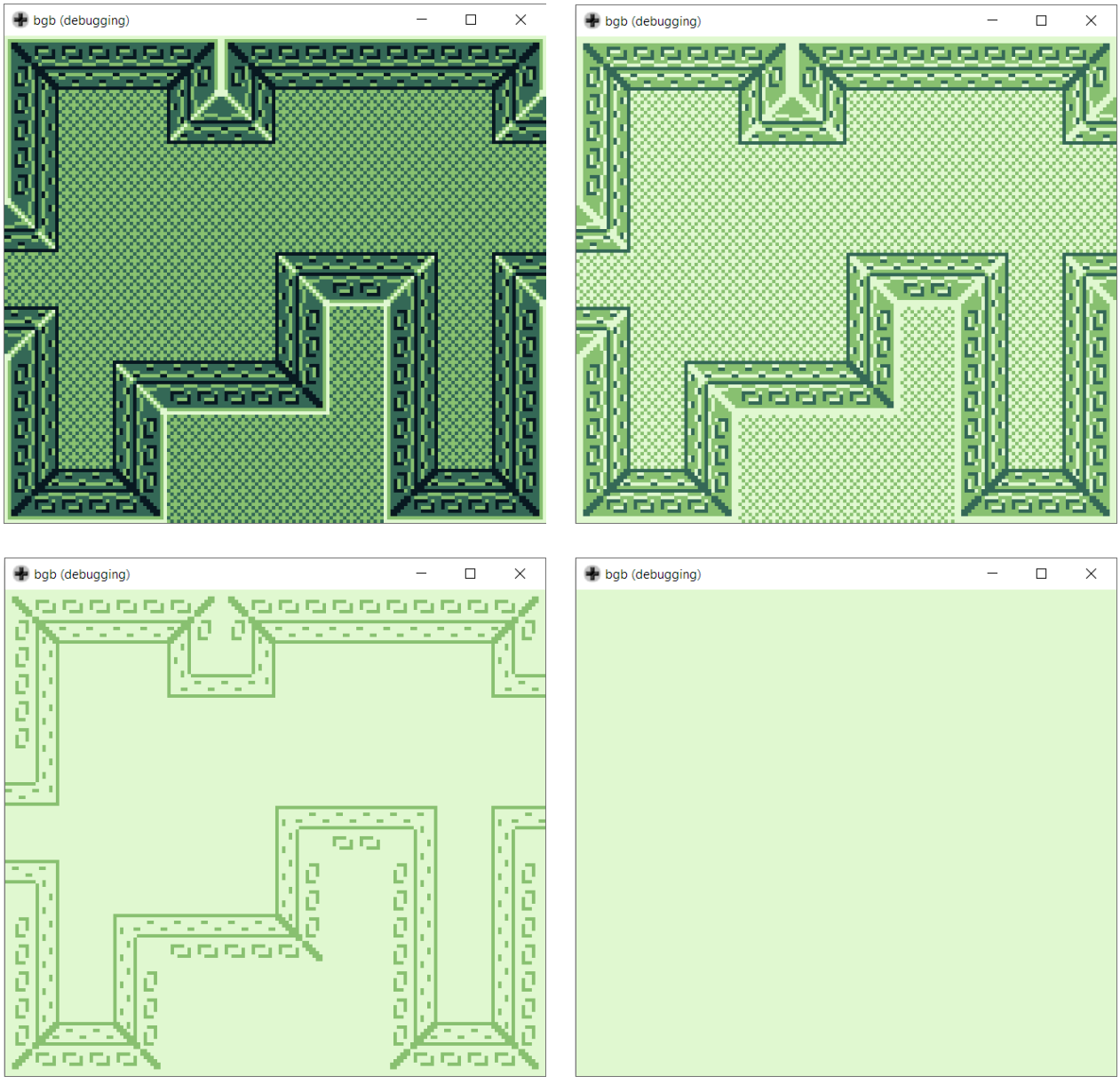



**Figura 6.12:** Fundido a blanco.

parámetro el tilemap al que cambiar.

### 6.2.2 Propiedades en tilemaps

Como ya dijimos anteriormente, vamos a añadir propiedades a cada tile para diferenciarlos entre los que son sólidos, como las paredes, y los que se pueden pisar, como el suelo.

Abre GBMB con el tileset que diseñamos, y haz clic en la opción *Location properties...* de la pestaña *File*. Se abrirá una ventana en la que crear propiedades como la que se ve en la imagen 6.13.

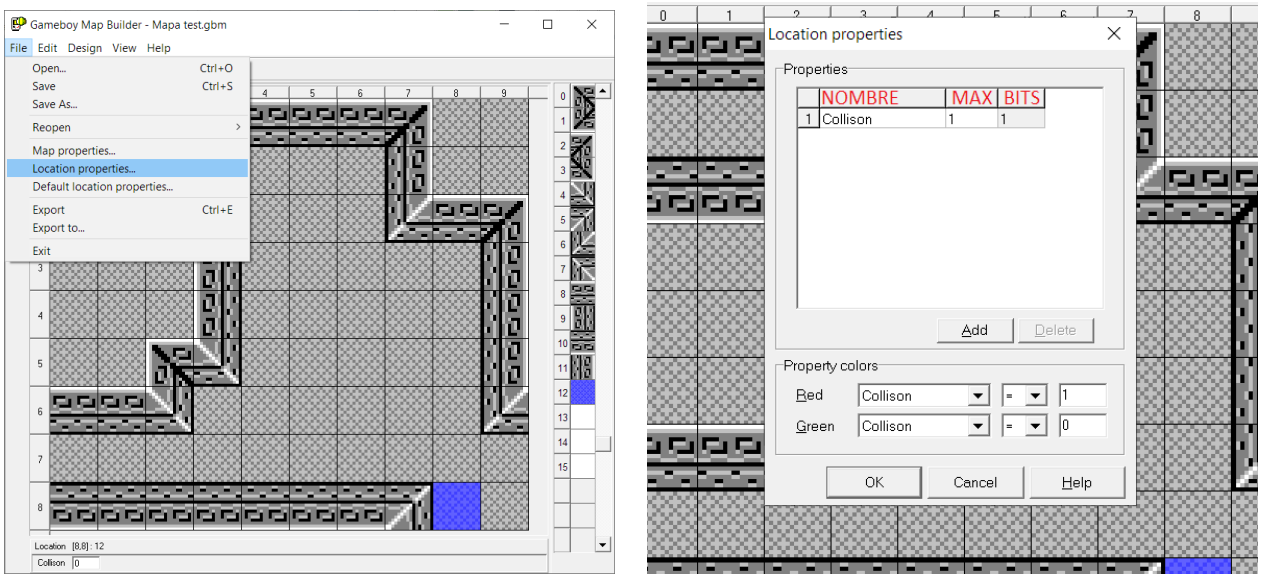



**Figura 6.13:** Propiedades del tilemap.

Las columnas de las propiedades no muestran nombres, pero la imagen anterior les proporciona unos para facilitar referirse a ellas. Elige un nombre para la nueva propiedad e introduce un valor de 1 en MAX. Este es el valor máximo que un tile puede tener en esta propiedad, empezando en 0. La última columna indica la cantidad de bits necesaria para representar esta propiedad, como la nuestra solo puede tener dos valores, solo se necesita un bit.

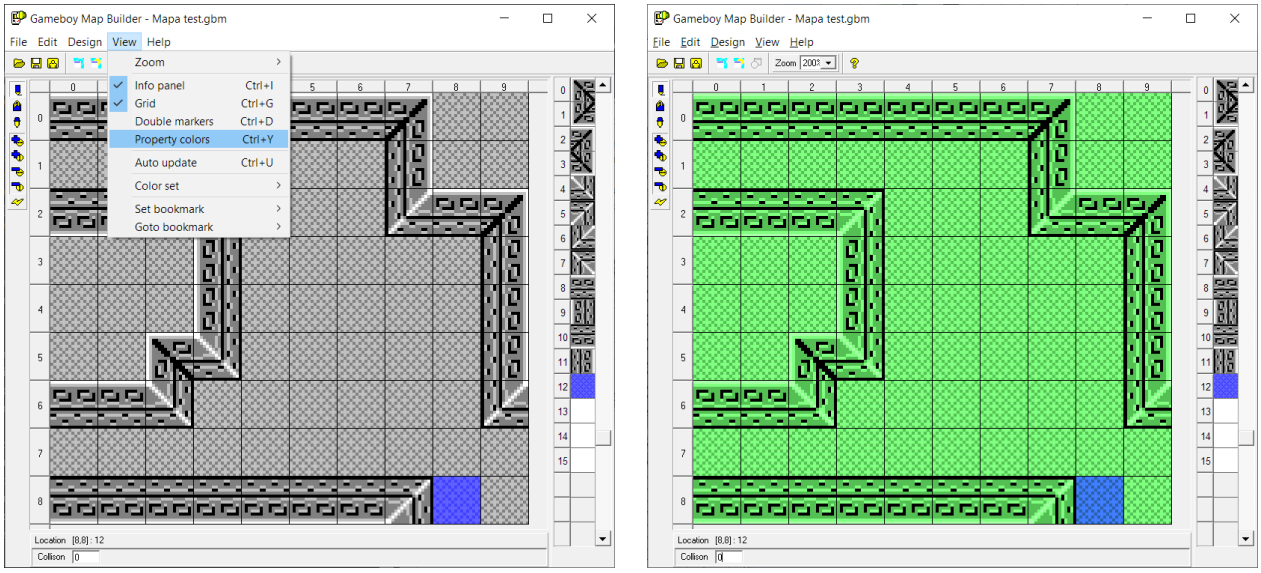



**Figura 6.14:** Colores según propiedad.

En la sección inferior de esta ventana, se pueden asignar colores a los tiles según sus propiedades para facilitar la identificación de qué tiles tienen ciertas características. Configura la primera fila, que da el color rojo, para cuando esta propiedad valga 1, y la siguiente fila, par el verde, cuando sea 0. Para ver los colores, activa la opción *Property colors* en la pestaña *View*.

Ahora mismo todos los tiles deberían verse de color verde, como en la figura 6.14 porque no les hemos asignado un valor a la propiedad. Puedes asignar el valor de cada propiedad que hayas creado para cada tile desde la opción *Default location properties...* en la pestaña *File*. En la ventana que se habrá abierto, asigna un uno a la propiedad para cada tile de pared, y asegúrate de que cada tile de suelo tenga un 0.

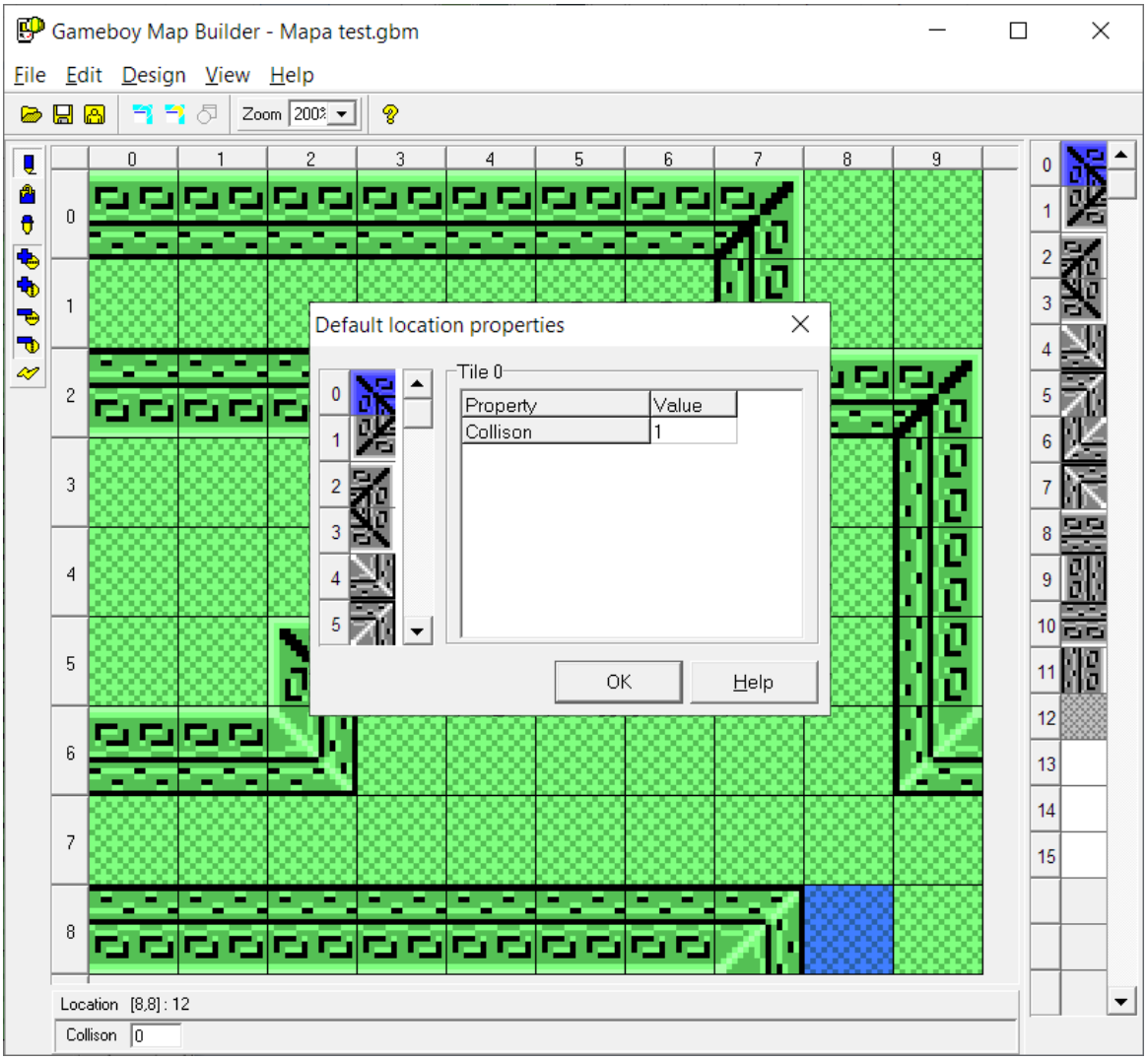



**Figura 6.15:** Asignar propiedades.

Una vez los hayas hecho, haz clic en *OK* para guardar los cambios. Verás que los colores del mapa no cambian a pesar de haber asignado valores nuevos a las propiedades de cada tile. Esto se debe a que sólo se han asignado al tileset, en la sección derecha, los tiles que pusimos antes siguen teniendo un valor de 0. Para guardar un mapa con los valores de propiedad adecuados, debes volver a poner los tiles de paredes, que serán rojos deberían tener el color rojo una vez los pongas. Alternativamente puede cambiar las propiedades de cada tile ya colocado en la barra inferior tras seleccionarlo.

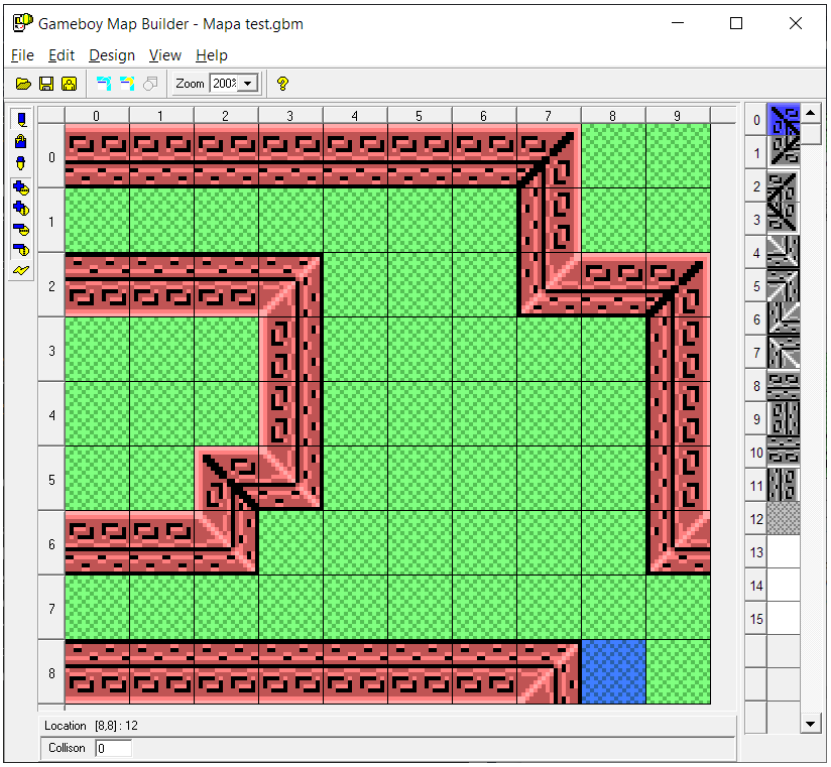



**Figura 6.16:** Colores asignados a cada tile.

Para terminar, vamos a exportar los nuevos mapas con las propiedades. Esta vez, en la sección de *Location format*, añade una segunda propiedad, haz que la primera sea el número de tile, y la segunda la que hemos creado. La tabla debería verse como la de la figura 6.17, con los cuatros bits del 0 al 3 con un 1, y el bit 4 con un 2, por la nueva propiedad.

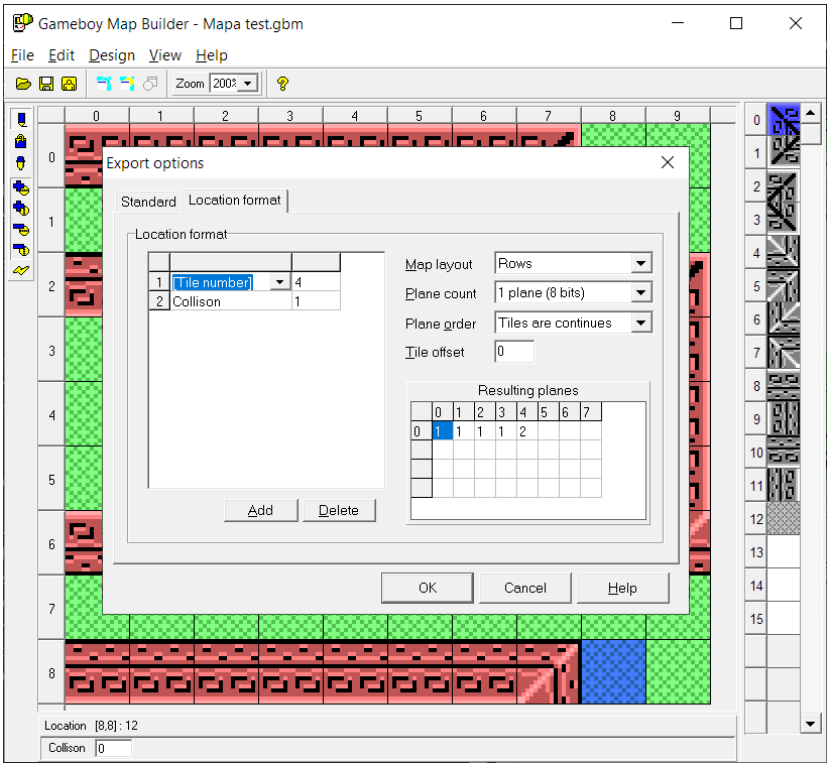



**Figura 6.17:** Colores asignados a cada tile.

Cuando añadas estos tilemaps al código, tendremos problemas si no actualizamos la función que obtiene la dirección de los metatiles a partir de su valor en el tilemap, pues se ha añadido un bit que no tiene relación con el número de tile. Antes de multiplicar y sumar el valor del tile al índice de metatiles, hay que poner a 0 todos los bits que no pertenecen al número de tile. Esto puede hacerse con una máscara de bits en la que aquellos que pertenezcan al número (del 0 al 3) estén a 1, y aquellos que no (del 4 al 7) estén a 0, y realizar una operación **AND** entre la máscara y el valor del tile para extraer el número de tile.

## 6.3 Movimiento en cuadrícula

Ya hemos diseñado un par de niveles, pero no nos hemos parado a ver todavía qué es un sokoban. Sokoban (encargado de almacén) es un juego que se lanzó en 1982 para PC-8801, un ordenador de Nippon Electric Company. En este juego el jugador debe empujar cajas hasta unas posiciones específicas en cada nivel moviéndose en horizontal y vertical en una cuadrícula.

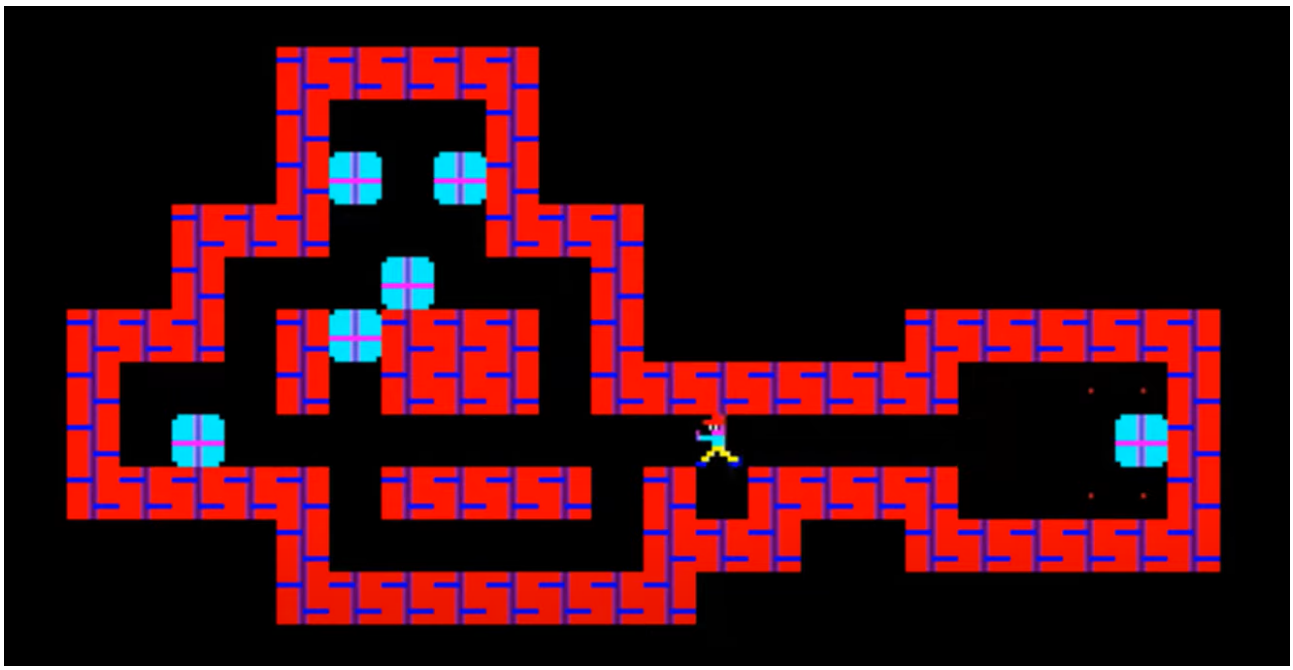


**Figura 6.18:** Juego Sokoban de NEC PC-8801

Un juego que siga estas mecánicas, ser un juego de puzles en el que el jugador se mueve en una cuadrícula y empuja cajas u otros objetos.

Para que nuestro personaje parezca moverse por una cuadrícula debemos modificar al menos uno de los sistemas que hemos programado. Las opciones que tenemos son las siguientes:

- Físicas: Podemos hacer que una velocidad de 1 modifique las coordenadas de las entidades en 16, moviendo así el personaje de golpe.
- Control: Cuando leamos los botones para actualizar la velocidad, en vez de dar una velocidad de 1, se le da una de 16.
- Dibujado: Antes de dibujar las entidades, se multiplican sus posiciones por 16 para ponerlas en la cuadrí-

cula.

De estas tres opciones, la más sensata y que nos dará menos problemas es modificar el sistema de dibujado. Querremos que internamente las coordenadas de la entidad no tomen valores más grandes que el ancho y alto en tiles del nivel, porque necesitaremos comprobar la posición del personaje en el mapa para ver si está intentando moverse hacia una pared o un suelo. La única opción de las anteriores que conserva las coordenadas X e Y de las entidades en valores bajos es la última. Para probar el juego a partir de ahora, querrás desactivar la generación de enemigos y todo lo demás que teníamos anteriormente para que no interfieran en la nueva funcionalidad.

El jugador ahora se moverá demasiado rápido, ya que su posición se actualiza en cada frame mientras tengamos un botón pulsado. Cambiemos el control del personaje para que solo se actualice su velocidad cuando los botones se hayan pulsado en el último frame, y no si se mantienen. Otro problema que tenemos ahora es que como el personaje ya no se mueve de forma continua, no se ven sus animaciones. Podemos arreglar esto cambiando la función de estado del personaje de forma que si el estado que fuera a devolver fuera 0, en su lugar devolviera el estado actual, de esta forma se reproducirá la animación aún cuando no se está moviendo.

Para finalizar esta sección, vamos a añadir al sistema de físicas una comprobación que mire si el metatile que ocupará el jugador en el siguiente paso es una pared o no, y en caso de que lo sea no actualice su posición. Para esto necesitaremos saber cuál es el tilemap actual, por lo que necesitaremos usar dos bytes de RAM en los que se guardará la dirección del tilemap desde la función que lo carga.

## 6.4 Reestructurar el bucle principal

El bucle principal de juego que creamos en el capítulo anterior tiene una estructura como la siguiente:

```
sistema de control      ;Actualiza velocidades
borrar entidades        ;Borra todas las entidades marcadas como muertas
sistema de físicas      ;Actualiza posiciones según la velocidad
sistema de animación    ;Actualiza sprites de cada entidad
sistema de dibujado     ;Actualiza la pantalla
sistema de colisiones   ;Comprueba qué entidades se superponen
gestor de partida       ;Comprueba si la partida ha terminado
esperar VBlank          ;Termina un frame
regreso al principio    ;Reinicia el bucle para el siguiente frame
```

Este bucle se ejecuta en cada frame, porque el juego que hacíamos se movía en tiempo real, las entidades se tenían que mover en cada frame, y por tanto las colisiones también debían comprobarse siempre. Un sokoban sin embargo no es un juego activo, las acciones solo suceden cuando el jugador pulsa una tecla para moverse, por lo que estar llamando a todos estos sistemas en cada frame es un desperdicio de energía para la consola. Vamos a reestructurar este bucle para que se ejecute únicamente lo mínimo necesario en cada frame, y el resto de sistemas se llamen solo si el jugador ha pulsado una tecla de movimiento.

El nuevo bucle quedaría así:

```
sistema de animaciones  ;Las animaciones se actualizan siempre
sistema de dibujado
comprobar entradas      ;Regreso al principio si no hay nuevos botones pulsados

gestor de entradas      ;Comprueba qué botón se ha pulsado y actúa acorde
                        ;Si no se ha pulsado ningún botón de dirección, regresa al principio
sistema de control      ;Actualiza velocidades de entidades independientes
borrar entidades        ;Borra todas las entidades marcadas como muertas
sistema de físicas      ;Actualiza posiciones
regreso al principio
```

De esta forma sólo se actualizan los sistemas cuando se pulsa un tecla.

Ahora mismo actualizamos la velocidad del personaje que controlamos usando un sistema de entrada que solo actualiza las velocidades y no hace nada si alguno de los otros botones está pulsado, o puede que hayas incluido el control del personaje en el sistema de control (o IA) de entidades. En cualquier caso, vamos a cambiar esto y moverlo a un nuevo gestor de entradas, que comprobará el estado de cada botón, y llamará a una función o a otra según cuál esté pulsado. En caso de que se hayan pulsado varios a la vez, habrá algunos que tendrán prioridad sobre los otros.

Este nuevo gestor requerirá de una lista de ocho funciones, una para cada botón, y usará una rutina como la siguiente para llamar a la adecuada:

```
   ld hl, funcionesBotones
   ld a, [flancoAscendente]
.loop
   bit 7, a
   jp nz, .llama
   rla
   inc hl
   inc hl
   jr. loop
.llama
   ld a, [hl+]
   ld h, [hl]
   ld l, a
jp hl
```

La lista de funciones deberá estar ordenada según como se almacenan los botones. Necesitamos diferenciar cuando se ha pulsado un botón de dirección y cuando otro para decidir en el bucle principal si regresar al principio o llamar al resto de sistemas. Podemos hacer esto usando como valor de retorno el flag de carry, que puede ser activado con la instrucción `scf` y desactivado con `or a`. Estas instrucciones podemos ponerlas dentro de las funciones asociadas a cada botón, o añadir un contador al gestor de entradas para que elija entre una y otra según qué valor tenía el contador cuando se encontró el botón pulsado.

## 6.5 Crea las cajas y los botones

Un elemento característico de un sokoban son las cajas, o cualquier objeto que haga una función similar. La función de las cajas es ser empujadas hasta sus puntos de destino, que podemos llamar botones. Una vez todos los botones estén pulsados, se completa el nivel y se pasa al siguiente. El jugador puede empujar las cajas, pero sólo de una en una, nunca dos a la vez, o lo que es lo mismo, una caja no puede empujar a otra.

Para implementar las cajas necesitamos una entidad que sea dibujable (necesitamos ver dónde están), y le daremos un tipo nuevo que será empujable, mientras que al jugador le daremos el tipo empujador. Puedes reutilizar los tipos colisionable y colisionador del capítulo anterior, ya que no usaremos el sistema de colisiones. Las cajas no tendrán el tipo físicas, ya que no tienen IA, en su lugar, les aplicaremos las físicas cuando vayan a ser empujadas. El motivo para esto es que el movimiento de las cajas depende del resultado de las físicas de de otras entidades, por lo que no se podrían actualizar todas las entidades de una sola pasada. Además si algún punto de una cadena de empuje no se puede mover, no debe moverse ninguno de los elementos de la cadena, cuyas físicas se habrían calculado antes.

Cuando hicimos las físicas para el movimiento en cuadrícula, añadimos una función que comprobaba si el movimiento era posible, según si la casilla a la que se estaba intentado mover la entidad era pared o suelo. Vamos a expandir esta función, de forma que si la casilla a la que se va a mover la entidad es de suelo, busque si hay alguna otra entidad en ella. En caso de que lo haya, si la entidad que se está moviendo es del tipo empujadora y la entidad en la casilla sobre la que se está moviendo es empujable, se copiarán los valores de velocidad de la primera entidad a la segunda, y se le aplicará la función de físicas a esta última. Si alguna de esas dos condiciones no se cumple, se devolverá que el movimiento no se puede realizar. Debido a que la función que comprueba si el movimiento es posible llama a la función de físicas como última acción, intentando mover una entidad, esta última debe también devolver si se ha podido mover o no, pues la anterior función de físicas de la cadena necesitará saberlo para poder mover o no su entidad.

La estructura de la función de físicas sería la siguiente:

```
fisicas una entidad:
    Calcula siguiente posición
    Movimiento posible?
    No -> RET            ;Devuelve "movimiento no realizado"
    Actualiza posición   ;Devuelve "movimiento realizado"
```

La función que calcula si el movimiento es posible tendría la siguiente estructura:

```
Movimiento posible?:
    La casilla de destino es suelo?
    No -> RET            ;Devuelve "no"
    Hay una entidad en la casilla de destino?
    No -> RET            ;Devuelve "sí"

    La entidad en la casilla de destino es empujable?
    No -> RET            ;Devuelve "no"
    La entidad moviéndose puede empujar?
    No -> RET            ;Devuelve "no"
```

```
Copia la velocidad de una entidad a otra
físicas una entidad (entidad en casilla destino)
RET                 ;Devuelve lo que devuleva físicas
```

Para hacer los botones vamos a necesitar crear un nuevo metatile que los represente en los niveles, por lo que deberás volver a GBTD y crearlo, y luego añade un par de estos metatiles a un mapa. En una partida, un nivel estará completado cuando todos los botones tengan una caja, por lo que necesitamos crear una nueva entidad que los represente. Esta entidad no necesitará ser dibujada ni tendrá animación, su diseño ya viene con el metatile que se dibuja en el mapa, pero sí necesita una función de comportamiento, para controlar si todos los botones están pulsados. Una forma de comprobar esto es tener un byte de memoria que se podría a 0 al principio de cada bucle, luego la IA de los botones aumentaría este valor en uno si el botón no está pulsado. Al final del bucle principal, si el valor sigue siendo 0, todos los botones estarían pulsados y el nivel completado.

Esta idea presenta varias complicaciones si partimos del código que tenemos. Para empezar, nuestro sistema de físicas no permite que haya dos entidades en la misma casilla, por lo que sería imposible empujar una caja sobre un botón. Además, si suponemos que vamos a usar la misma función que en el apartado anterior para buscar una entidad en la casilla del botón y determinar si está pulsado, esta siempre devolverá que hay una entidad, pues el botón está en esa casilla. Podemos resolver ambos problemas cambiando la función que busca una entidad para que solo devuelva entidades empujables, si suponemos que no va a haber otras entidades sólidas que no se puedan empujar. De este modo los botones o cualquier otra entidad estática no impedirá el movimiento sobre una casilla, y los botones sólo se activarán si hay cajas sobre ellos.

Ahora que podemos saber en cualquier momento si todos los botones están pulsados, es hora de crear una función en el gestor de partida para cambiar de nivel, y añadirla al bucle principal. Supondremos que los niveles estarán en una lista con los punteros al tilemap de cada uno, por lo que el gestor de partida tendrá que recordar en qué posición de la lista estaba el último nivel, y aumentar ese puntero en dos para obtener el siguiente nivel. Por ahora crea una lista únicamente con dos niveles, genera una o dos entidades de caja y botón al empezar la partida y prueba a completar el nivel para comprobar que se cambia el tilemap como esperábamos.

## 6.6 Codificación de niveles

Habrás visto que al completar el primer nivel, se empieza a cambiar automáticamente al siguiente. Esto se debe a que las cajas y los botones siguen en las mismas posiciones, por lo que el nuevo nivel se detecta automáticamente como completado y además, las entidades no cambian de color junto al fundido. Tenemos que añadir el cambio de la paleta OBP0 para hacer el fundido completo, y borrar todas las entidades al cambiar de nivel. Si haces esto, en el nuevo nivel no habrá ni personaje, ni cajas, ni botones, es hora de codificar las entidades dentro de los niveles.

Podríamos hacer que en la definición de un nivel estuviera además del tilemap una lista de entidades y que tras cargar el mapa se cargaran todas las entidades. Otra opción es aprovechar los bits libres de los metatiles para indicar si hay alguna entidad inicialmente en ellos. Crea una nueva propiedad de valor máximo 3 en GBTD y llámala **entidad** y dale al tile del botón el valor por defecto 3. Usaremos el valor 2 para indicar dónde hay cajas,

y el 1 para la posición de inicio del jugador. Un 0 significará que no hay ninguna entidad. Exporta los mapas poniendo la nueva propiedad en los bits 5 y 6.

Cuando estés cargando un mapa, comprueba para cada metatile si debe haber una entidad en esa casilla, y crea la correspondiente al número encontrado.

# Apéndice A Sistemas de numeración

Los números que usamos en el día a día están escritos en el sistema de numeración decimal, al que también se le conoce como de base 10, porque tiene 10 símbolos distintos, los números del 0 al 9. Este es un sistema posicional, en el que el mismo símbolo tiene valores distintos en función de en que parte del número se encuentre, esto es lo que aprendemos a diferenciar como la cifra de las unidades, la de las decenas, la de las centenas, y así hasta llegar a la cifra de más a la izquierda del número. La relación de valor entre una cifra contigua a otra es de 10, el mismo número que la base, y las cifras más a la izquierda, o más significativas son las que tienen más valor. De este modo podemos descomponer números como 1989 de la siguiente forma:

$$1 \cdot 10^3 + 9 \cdot 10^2 + 8 \cdot 10^1 + 9 \cdot 10^0 \tag{A.1}$$

Con un número de 4 cifras pueden representarse $10^4$ valores diferentes, del 0 al 9999. En general, con un número de n cifras pueden representarse $10^n$ valores diferentes.

## A.1 Hexadecimal

El sistema hexadecimal funciona del mismo modo, con el añadido de tener 16 símbolos en vez de 10. Dado que no tenemos más números de una cifra que los 10 del sistema decimal, para representar los 6 símbolos restantes se usan las letras de la A a la F, teniendo estas los valores de 10 a 15 en orden alfabético, es decir, A equivale a 10, B a 11, C a 12, D a 13, E a 14 y F a 15. Y al igual que en el sistema decimal, un símbolo tiene valores distintos según en qué posición de un número se encuentre, pero en vez de multiplicarse por 10, el valor se multiplica por 16. Al contar entonces, el siguiente número después del 9, sería A, al que le seguirían B, C, D, E y F, y tras este iría 10. Entonces se aumentaría de nuevo la cifra derecha hasta llegar a 1F, y el siguiente número sería 20. El número que iría después de 9F sería A0, y a FF le seguiría 100. Para obtener el valor de un número hexadecimal expresado en el sistema decimal, seguiríamos el proceso de multiplicar por potencias de 16, usando 0 de exponente para la cifra de más a la derecha y aumentándolo en uno para cada cifra subsiguiente. Por ejemplo, el número hexadecimal 67B4 tiene un valor de:

$$6 \cdot 16^3 + 7 \cdot 16^2 + 11 \cdot 16^1 + 4 \cdot 16^0 \tag{A.2}$$

En el contexto de este libro no es necesario convertir números de hexadecimal a decimal, pero si es importante que conozcas el orden de numeración, y que entre 50 y 60 hay 16 números, no 10.

Al igual que con el sistema decimal, con un número de n cifras pueden representarse $16^n$ cifras. Un número de 2 cifras hexadecimales puede valer entre 0 y 255, uno de cifras puede llegar hasta 65535.

## A.2 Binario

El sistema binario funciona exactamente igual que el decimal o el hexadecimal, pero usando solamente dos símbolos, el 0 y el 1. Dado que los componentes internos de un ordenador pueden estar en dos estados, según si circula corriente eléctrica o no, el sistema binario se usa para representar el funcionamiento de un ordenador,

y al mismo tiempo se construyen los componentes para que realicen operaciones lógicas y aritméticas en binario.

Al igual que con las anteriores bases, para obtener el valor de un número binario se deben multiplicar sus cifras por potencias de 2. Por ejemplo:

$$11011100 = 1 \cdot 2^7 + 1 \cdot 2^6 + 0 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0 = 220 \tag{A.3}$$

El mínimo de información que podemos tener en el sistema binario es una sola cifra, que puede representar un 1 o un 0. Esto es lo que se llama un bit. Un byte es una unidad de memoria compuesta de 8 bits, 8 cifras, por lo que puede tomar valores entre 0 y 255 ($2^8$ valores diferentes), es decir, que un número binario de 8 cifras puede representar exactamente el mismo rango de valores que un número hexadecimal de 2. Esto es un propiedad que surge debido a que 16 es potencia de 2, concretamente 16 es igual a $2^4$, por lo que una cifra hexadecimal es equivalente a 4 cifras binarias.

Gracias a esta característica, podemos traducir números de binario a hexadecimal y al revés sin necesidad de convertir cada vez el número completo a decimal. En su lugar, podemos traducir cada cifra hexadecimal por separado y juntar todas las traducciones a binario, que tendrán 4 bits. Si fuera al revés podríamos dividir un número binario de 4 en 4 cifras y convertir cada grupo a una cifra hexadecimal.

Verás que al trabajar con ordenadores es muy habitual representar y manejar en hexadecimal números que internamente son binarios. Esto se debe a que es mucho más rápido identificar el valor de un número si tiene una menor cantidad de cifras.

# Apéndice B RGBASM

Este anexo es una explicación de las características de rgbasm, el lenguaje que se usa en el libro para programar en Game Boy. Esta documentación se ha extraído y resumido de [4], y corresponde a la versión 0.6.1 de rgbasm. Lo más probable es que la mayoría de conceptos se hayan mantenido igual, pero si estás leyendo este libro e instalaste una versión distinta de rgbasm te recomiendo que vayas a la fuente original y elijas la documentación de la versión que estés usando.

## B.1 Símbolos

### B.1.1 Etiquetas

RGBASM distingue entre varios tipos de etiquetas, siendo estas globales, locales, y exportadas. Las etiquetas sirven para que no necesites calcular las direcciones de memoria en las que se encontrará cada instrucción a la que quieras saltar, en cuyo caso también necesitarías actualizar cada salto siempre que se mueva algo de sitio. Para declarar una etiqueta ésta debe ser lo más a la izquierda de la línea, debe empezar por una letra, y debe terminar con el símbolo de dos puntos. Si la etiqueta es local, se pueden omitir los dos puntos al final. Si una etiqueta no tiene ningún símbolo de punto (.) en ella, es global, y local si tiene uno. Al crear una etiqueta global se crea un ámbito perteneciente a ella, que termina en la declaración de la siguiente etiqueta global. No se puede declarar una etiqueta con más de un punto. El punto de una etiqueta local debe estar al principio de ella, si no lo está, la parte izquierda se interpreta como una etiqueta global y debe coincidir con el ámbito actual (la etiqueta global inmediatamente anterior). El nombre completo de una etiqueta local es siempre `ámbito.local`, por lo que en aquellos casos que se declare una etiqueta local poniendo el punto al principio, el nombre real de la etiqueta será el de la global del ámbito actual seguido de la local. Puedes referenciar una etiqueta local sin usar su parte de ámbito desde dentro de éste, pero debes incluirlo si quieres referenciarla desde otro. Dado que el ámbito forma parte del nombre de una etiqueta local, se pueden declarar varias en distintos ámbitos con el mismo nombre. Los siguientes son algunos ejemplos de etiquetas:

```
EtiquetaGlobal:   ;Inicia ámbito "EtiquetaGlobal"
.local            ;Nombre completo: EtiquetaGlobal.local
OtraGlobal:       ;Inicia ámbito "OtraGlobal"
.local            ;Nombre completo: OtraGlobal.local
.otraLocal:       ;Nombre completo: OtraGlobal.otraLocal

jr EtiquetaGlobal.local ;Es un uso correcto
jr .local            ;.local se interpreta como OtraGlobal.local

TerceraGlobal::   ;Inicia ámbito "TerceraGlobal", esta etiqueta se exporta
```

Fíjate en que los dos puntos no pertenecen al nombre de la etiqueta, no hay que usarlos al referenciarlas.

Por último tenemos las etiquetas exportadas, que se definen poniendo dos símbolos de dos puntos al final de la declaración de una etiqueta. Estas etiquetas pueden ser referenciadas desde archivos distintos a en el que

se definieron.

Al referenciar una etiqueta, su valor se resuelve durante el ensamblado a no ser que sea exportada, en cuyo caso se resuelve en el linker.

### B.1.2 Constantes

Las constantes representan valores numéricos que se resuelven durante el ensamblado. RGBASM implementa varios tipos de constantes:

#### B.1.2.1 Constantes fijas

Se definen con "`DEF nombre EQU valor`", y pueden usarse para cualquier elemento que no cambie, como el inicio de la memoria de vídeo u otras regiones de memoria.

```
DEF _VRAM EQU $8000
DEF _SCRN0 EQU $9800
DEF _SCRN1 EQU $9C00
```

El archivo *hardware.inc* define muchas de este tipo de constantes. Este tipo de constante no se puede redefinir con otra directiva igual, aunque si es necesario, se le puede cambiar el valor a una constante fija usando la palabra clave `REDEF` en vez de `DEF`.

#### B.1.2.2 Variables

A pesar de su nombre, algunas constantes pueden variar su valor. Este tipo de constante se define usando un símbolo de igual "**=**.[en] vez de la palabra clave `EQU`. También pueden definirse con operadores compuestos (**+=**, **-=**, ***=**...).

#### B.1.2.3 Constantes relativas

Este tipo de constantes se definen indicando la diferencia de valor entre la actual y la siguiente. Su uso más común es indicar la posición y tamaño de elementos en una estructura de datos. Funcionan definiendo una variable interna llamada **_RS** que se actualiza con cada definición. Las directivas que interactúan con esta variable son las siguientes:

- RSSET expresión: Asigna el valor de *expresión* a **_RS**.
- RSRESET: Asigna el valor 0 a **_RS**.
- DEF símbolo RB valor: Asigna a *símbolo* el valor de **_RS** y aumenta esta variable en *valor*.
- DEF símbolo RW valor: Asigna a *símbolo* el valor de **_RS** y aumenta esta variable en 2**valor*.
- DEF símbolo RL valor: Asigna a *símbolo* el valor de **_RS** y aumenta esta variable en 4**valor*.

## B.2 Secciones

Todo el código, en cualquier archivo, debe estar precedido por una directiva de sección. Esta directiva indica en qué lugar de la ROM poner el código a continuación, o en el caso de las secciones de RAM, las etiquetas.

El linker (rgblink) es el que en última instancia decide dónde colocar cada sección, siguiendo las limitaciones impuestas por sus declaraciones. Si estas limitaciones son contradictorias o imposibles de cumplir, lanza un error. Dos secciones declaradas en lugares distintos no pueden tener el mismo nombre, incluso en archivos diferentes, excepto si se usan algunas opciones específicas.

Todas las declaraciones de secciones siguen el formato:

```
SECTION nombre, tipo<[dirección]>, <opciones>
```

El atributo `nombre` debe ser una cadena de texto entre comillas dobles. Los atributos `[dirección]` y `opciones` no son obligatorios, y cada opción distinta debe estar separada por comas. El atributo `[dirección]` indica la dirección exacta en la que debe comenzar la sección.

Los distintos tipos de sección son:

- ROM0: Banco 0 de la ROM, va de $0000 a $3FFF. Si la opción **–tiny** se activa en el linker, se expande hasta $7FFF.
- ROMX: Cualquier banco distinto al 0 de la ROM, las direcciones van de $4000 a $7FFF. Si la opción **–tiny** se activa en el linker, este tipo es equivalente a ROM0. El banco exacto elegido depende del linker, pero puede indicarse con la opción **BANK[banco]**, y puede ir de 1 a 511.
- VRAM: Un banco de la memoria de vídeo, con direcciones entre $8000 y $9FFF. El banco puede ser 0 o 1, aunque el banco 1 no está disponible si se está ensamblando para una Game Boy original (DMG).
- SRAM: Un banco de RAM de guardado externa, en el cartucho. Va de $A000 a $BFFF, y puede haber hasta un máximo de 16 bancos, numerados del 0 al 15.
- WRAM0: Banco 0 de la RAM de trabajo, va de $C000 a $CFFF. si la opción **–wramx** se activa en el linker, se expande hasta $DFFF.
- WRAMX: Cualquier banco distinto al 0 de la RAM de trabajo, las direcciones van de $D000 a $DFFF. Si la opción **–wramx** se activa en el linker, este tipo es equivalente a WRAM0. El banco exacto elegido depende del linker, pero puede indicarse con la opción **BANK[banco]**, y puede ir de 1 a 7.
- OAM: Sección de RAM para los objetos (sprites). Las direcciones están entre $FE00 y $FE9F.
- HRAM: Sección alta de la RAM. Abarca las direcciones entre $FF80 y $FFFE.

Como se ha mencionado más arriba, dos secciones no pueden tener el mismo nombre salvo en algunas excepciones. La primera de ellas es declarar la sección como fragment. Para hacer esto sólo hay que escribir **FRAGMENT** justo después de **SECTION** y antes del nombre de la sección. Esto hará que el linker ponga todas las secciones con este nombre juntas. Ten en cuenta que todas las secciones con el mismo nombre que una que hayas definido como fragment deben ir acompañadas de **FRAGMENT** también.

## B.3 Cadenas constantes

## B.4 Macros

Una macro es un bloque de código que se asigna a un nombre, igual que los símbolos, y puede ser invocado en otras partes del código. Invocar una macro equivale a copiar el código definido en esta. Para invocar una macro, su nombre debe estar indentado.

El siguiente es un ejemplo de cómo definir una macro:

```
MACRO NombreMacro
    contenido
ENDM
```

`MACRO` inicia la definición de la macro, `NombreMacro` es el nombre que se le asigna, y el que se usará para invocarla, `contenido` es todo el contenido de la macro, y `ENDM` marca el final de la definición. Cuando se escriba `NombreMacro` en alguna parte del código, se copiará `contenido` en su lugar. Es posible invocar una macro desde otra, el contenido de una macro puede invocar a la misma.

Las macros admiten argumentos, pero estos no se listan en la definición. La macro recibe los argumentos en la invocación separados por comas. Para usar argumentos en el código de una macro, hay que usar las expresiones **\1** a **\9** para los primeros 9 parámetros. Para usar más parámetros las expresiones tienen la forma **\<n>**, donde **n** es el número de argumento, por ejemplo el décimo se escribiría como **\<10>**.

Ten en cuenta que los argumentos no se evalúan antes de invocarse a la macro, se copian como texto tal cual donde se usen. Esto puede tener efectos no deseados, por ejemplo:

```
MACRO MultiplicaPorDos
    DB \1 * 2
ENDM

    MultiplicaPorDos 1 + 2
    ;Se expande como DB 1 + 2 * 2
    ;El resultado de la operación es 5, no 6.
```

Para evitar estos problemas puedes poner paréntesis alrededor del argumento en la macro, o en la propia invocación.

También es posible definir etiquetas dentro de una macro:

```
MACRO LoopyMacro
       xor a
.loop
       ld  [hl+], a
       dec c
       jr  nz, .loop
ENDM
```

Esto producirá un error si la macro es invocada más de una vez dentro del mismo ámbito. La expresión **\@** se expandirá a una cadena de texto única dentro de la invocación de la macro, por lo que puedes sustituir el código anterior por:

```
MACRO LoopyMacro
       xor a
.loop\@
       ld  [hl+], a
       dec c
```

```
        jr   nz, .loop\@
ENDM
```

De esta forma \@ hará que la etiqueta sea distinta en cada invocación.

### B.4.1 Bloques de código repetibles

Puedes usar la directiva REPT para generar bloques de código que se repitan. Esta directiva se usa de la siguiente forma:

```
REPT N
    código
ENDR
```

El código incluido entre `REPT` y `ENDR` se repetirá `N` veces, donde N es un valor conocido en tiempo de ensamblado. Una utilidad de esta directiva es generar tablas sin tener que escribir cada entrada, por ejemplo es posible crear una tabla con los cuadrados de cada número entre 0 y 100 de la siguiente forma:

```
DEF X = 0
REPT 101
    DW X*X
    DEF X += 1
ENDR
```

Otra forma de crear bloques repetibles es con la directiva `FOR`, que también produce una variable que recorre el rango de valores que le indiquemos. La tabla anterior puede generarse usando `FOR` de la siguiente forma:

```
FOR X, 101
    DW X*X
ENDR
```

Hay tres formas de crear un bucle FOR, que permiten cambiar el punto de inicio, final, y cuanto cambiar la variable en cada iteración:

- `FOR N, fin`: N toma los valores entre 0 y `fin-1`.
- `FOR N, inicio, fin-1`: N toma los valores entre `inicio` y `fin-1`.
- `FOR N, inicio, fin, paso`: N toma los valores entre `inicio` y `fin-1`, aumentando su valor en `paso` en cada iteración.

Los tres argumentos de `FOR`, `inicio`, `fin` y `paso` deben conocerse en tiempo de ensamblado.

También puedes usar la expresión \@ dentro de un bloque repetible, puedes usar bloques repetibles en una macro, y puedes anidar bloques repetibles.

### B.4.2 Código condicional

Las directivas `IF`, `ELIF`, `ELSE` y `ENDC` permiten crear código condicional, de forma que se ignoren partes del código durante el ensamblado. Estas directivas permiten crear estructuras complejas de macros, y pueden anidarse y combinarse con código repetible. Un bloque condicional se define de la siguiente manera:

```
IF condición 1
    código 1
ELIF condición 2
    código 2
ELIF condición 3
    código 3
ELSE
    código 4
ENDC
```

Las directivas `ELIF` y `ELSE` son opcionales, en caso de estar presentes, `ELSE` debe ser la última directiva antes de `ENDC`, y todos los `ELIF` deben estar después del primer `IF`. Puede haber tantos bloques `ELIF` como se desee. En el archivo final sólo se incluirá como máximo un bloque de código de la estructura, aquel cuya condición se cumpla primero, o el de `ELSE` en caso de estar presente.

# Apéndice C Instrucciones

En esta sección se listan y explican todas las instrucciones de las que dispone el Sharp SM83, el procesador de una Game Boy. Es importante tener en cuenta que las instrucciones no son como funciones de un lenguaje de programación de alto nivel. `ld` no es una instrucción que recibe dos parámetros, el origen y el destino, `ld a, b` es una instrucción, y `ld c, d` es otra distinta. Cada instrucción está identificada por un opcode (código de operación), uno o dos bytes que el procesador usa para decodificar la instrucción. El opcode de `ld a, b` es $78, mientras que el de `ld c, d` es $4A. Es evidente que a pesar de que ambas instrucciones hagan lo mismo (copiar un byte de un registro a otro) internamente son completamente diferentes. Es importante recordar esto pues es posible que si pensamos en `ld` como una función, creamos erróneamente que algunas instrucciones existan. Por ejemplo, el opcode de la instrucción `ld [de], a` es $12, sin embargo la instrucción `ld [de], b` no existe. No es posible copiar el valor del registro B sobre la dirección de memoria indicada por el registro DE.

Puedes consultar el listado completo de opcodes del Sharp SM83 en la web [2]. Los opcodes de la primera tabla son de un solo byte, mientras que los de la segunda son de dos, siendo el primero siempre $CB.

En las instrucciones siguientes, se agrupan varias de ellas usando nombres genéricos para algunos registros y datos. Estos nombres son los siguientes:

- **r8**: Los registros de 8 bits A, B, C, D, E, H, L.
- **r16**: Los registros de 16 bits AF, BC, DE, HL, SP.
- **n8**: Un byte de datos ubicado en memoria inmediatamente después del opcode.
- **n16**: Dos bytes de datos ubicado en memoria inmediatamente después del opcode.
- **e8**: Un byte de datos ubicado en memoria inmediatamente después del opcode, con signo.
- **u3**: Un valor entre 0 y 7. Usado en las instrucciones de bit. Corresponde a 3 bits del código de operación.
- **cc**: Condición. Una instrucción que use esto depende del estado de los flags. Las posibles condiciones son:
  - z: Si Z es 1.
  - nz: Si Z es 0.
  - c: Si C es 1.
  - nc: Si C es 0.

## C.1 Instrucciones de carga (load) de 8 bits

Las instrucciones de carga de 8 bits copian un byte de un origen a un destino. El origen puede ser un registro, un dato inmediato, o una dirección de memoria. El destino puede ser un registro o una dirección de memoria.

### ld r8, r8'

Copia el valor del registro r8' en r8.

### ld r8, n8

Copia el dato inmediato n8 en r8.

### ld r8, [hl]

Copia el byte en la dirección indicada por el registro HL en el registro r8.

### ld [hl], r8

Copia el valor del registro r8 en la dirección indicada por el registro HL.

### ld [hl], n8

Copia el dato inmediato n8 en la dirección indicada por el registro HL.

### ld a, [bc]

Copia el byte en la dirección indicada por el registro BC en el registro A.

### ld a, [de]

Copia el byte en la dirección indicada por el registro DE en el registro A.

### ld [bc], a

Copia el valor del registro A en la dirección indicada por el registro BC.

### ld [de], a

Copia el valor del registro A en la dirección indicada por el registro DE.

### ld a, [n16]

Copia el byte en la dirección indicada por n16 en el registro A.

### ld [n16], a

Copia el valor del registro A en la dirección indicada por n16.

### ld a, [c]

Copia el byte en la dirección $FF00+C en el registro A.

### ld [c], a

Copia el valor del registro A en la dirección $FF00+C.

### ldh a, [n8]

Copia el byte en la dirección $FF00+n8 en el registro A.

### ldh [n8], a

Copia el valor del registro A en la dirección $FF00+n8.

### ld a, [hl-]

Copia el byte en la dirección indicada por el registro HL en el registro A y reduce HL en uno.

### ld [hl-], a

Copia el valor del registro A en la dirección indicada por el registro HL y reduce HL en uno.

### ld a, [hl+]

Copia el byte en la dirección indicada por el registro HL en el registro A y aumenta HL en uno.

### ld [hl+], a

Copia el valor del registro A en la dirección indicada por el registro HL y amenta HL en uno.

## C.2 Instrucciones de carga (load) de 16 bits

Las instrucciones de carga de 16 bits copian dos bytes a un registro de 16 bits desde otro registro o desde dos direcciones de memoria consecutivas.

### ld r16, n16

Copia el dato inmediato de 16 bits n16 en el registro r16. r16 no puede ser AF.

### ld [n16], sp

Copia el valor del registro SP en la dirección indicada por n16.

### ld sp, hl

Copia el valor del registro HL en el registro SP.

### push r16

Reduce SP en dos y copia en la dirección resultante el registro r16. r16 no puede ser SP.

### pop r16

Copia la posición de memoria indicada por SP y el siguiente en el registro r16, y aumenta SP en dos. r16 no puede ser SP.

## C.3 Instrucciones aritméticas de 8 bits

Las instrucciones aritméticas de 8 bits realizan operaciones con registros y datos de 8 bits. Todas ellas usan el registro A como operador y actualizan los flags. Todas ellas pueden escribirse en rgbasm de dos formas, `operación a, n` y `operación n`. Dado que el registro A es implícito al usarse siempre, en este libro se usa la segunda opción.

### add r8

Suma los registros A y r8 y guarda el resultado en A.

### add [hl]

Suma el registro A y la posición de memoria indicada por HL y guarda el resultado en A.

### add n8

Suma el registro A y el dato inmediato n8 y guarda el resultado en A.

### adc r8

Suma los registros A y r8 y el bit de acarreo, y guarda el resultado en A.

### adc [hl]

Suma el registro A y la posición de memoria indicada por HL y el bit de acarreo, y guarda el resultado en A.

### adc n8

Suma el registro A y el dato inmediato n8 y el bit de acarreo, y guarda el resultado en A.

### sub r8

Resta el registro r8 al registro A y guarda el resultado en A.

### sub [hl]

Resta la posición de memoria indicada por HL al registro A y guarda el resultado en A.

### sub n8

Resta el dato inmediato n8 al registro A y guarda el resultado en A.

### sbc r8

Resta el registro r8 y el bit de acarreo al registro A y guarda el resultado en A.

### sbc [hl]

Resta la posición de memoria indicada por HL y el bit de acarreo al registro A y guarda el resultado en A.

### sbc n8

Resta el dato inmediato n8 y el bit de acarreo al registro A, y guarda el resultado en A.

### cp r8

Resta el registro r8 al registro A y no guarda el resultado.

### cp [hl]

Resta la posición de memoria indicada por HL al registro A y no guarda el resultado.

### cp n8

Resta el dato inmediato n8 al registro A y no guarda el resultado.

### inc r8

Aumenta el registro r8 en uno. No modifica el flag de acarreo.

### inc [hl]

Aumenta el valor de la posición de memoria indicada por HL en uno. No modifica el flag de acarreo.

### dec r8

Reduce el registro r8 en uno. No modifica el flag de acarreo.

### dec [hl]

Reduce el valor de la posición de memoria indicada por HL en uno. No modifica el flag de acarreo.

### and r8

Realiza una operación **AND** bit a bit entre los registros A y r8 y guarda el resultado en A.

### and [hl]

Realiza una operación **AND** bit a bit entre el registro A y la posición de memoria indicada por HL y guarda el resultado en A.

### and n8

Realiza una operación **AND** bit a bit entre el registro A y el dato inmediato n8 y guarda el resultado en A.

### or r8

Realiza una operación **OR** bit a bit entre los registros A y r8 y guarda el resultado en A.

### or [hl]

Realiza una operación **OR** bit a bit entre el registro A y la posición de memoria indicada por HL y guarda el resultado en A.

### or n8

Realiza una operación **OR** bit a bit entre el registro A y el dato inmediato n8 y guarda el resultado en A.

### xor r8

Realiza una operación **XOR** bit a bit entre los registros A y r8 y guarda el resultado en A.

### xor [hl]

Realiza una operación **XOR** bit a bit entre el registro A y la posición de memoria indicada por HL y guarda el resultado en A.

### xor n8

Realiza una operación **XOR** bit a bit entre el registro A y el dato inmediato n8 y guarda el resultado en A.

### ccf

Invierte el flag de acarreo y pone a cero los flags N y H.

### scf

Activa el flag de acarreo y pone a cero los flags N y H.

### daa

Convierte el registro A a decimal codificado en binario, suponiendo que su valor actual es el resultado de una operación de otros dos valores en decimal. El decimal codificado en binario es un sistema de representación que usa un nibble para representar los números del 0 al 9, usando los valores binarios del %0000 al %1001. Con este sistema un byte puede representar los números decimales del 00 al 99. Esta instrucción funciona sumando $06 y/o $60 según si los nibbles bajo y alto son mayores de 9 respectivamente. Si el nibble alto cumple dicha condición, se activa el flag de acarreo. Por ejemplo, $6B se convertiría en $71, $E4 en $44, y $CF en $35. Esta es la única instrucción de la que dependen los flags N y H, y tras su ejecución no modifica el flag N, y pone el flag H a 0.

### cpl

Invierte todos los bits del registro A y activa los flags N y H.

## C.4 Instrucciones aritméticas de 16 bits

Las instrucciones aritméticas de 16 bits realizan operaciones con registros de 16 bits, a excepción de AF.

### add hl, r16

Suma los registros HL y r16 y guarda el resultado en HL. Esta instrucción modifica los flags N, H y C.

### inc r16

Aumenta el registro r16 en uno. Esta instrucción no modifica los flags.

### dec r16

Reduce el registro r16 en uno. Esta instrucción no modifica los flags.

### add sp, e8

Suma el dato inmediato con signo e8 al registro SP y guarda el resultado en SP.

### ld hl, sp+e8 Suma el dato inmediato con signo e8 al registro SP y guarda el resultado en HL.

## C.5 Instrucciones de rotación y desplazamiento

Estas instrucciones modifican el contenido de un registro de 8 bits o de una dirección de memoria desplazando sus bits.

### rra

Rota el contenido del registro A un bit a la derecha. El contenido del bit 0 se copia en el flag de acarreo y el anterior contenido de este se copia al bit 7.

### rla

Rota el contenido del registro A un bit a la izquierda. El contenido del bit 7 se copia en el flag de acarreo y el anterior contenido de este se copia al bit 0.

### rrca

Rota el contenido del registro A un bit a la derecha. El contenido del bit 0 se copia en el flag de acarreo y en el bit 7.

### rlca

Rota el contenido del registro A un bit a la izquierda. El contenido del bit 7 se copia en el flag de acarreo y en el bit 0.

### rr r8

Rota el contenido del registro r8 un bit a la derecha. El contenido del bit 0 se copia en el flag de acarreo y el anterior contenido de este se copia al bit 7.

### rr [hl]

Rota el contenido de la posición de memoria indicada por hl un bit a la derecha. El contenido del bit 0 se copia en el flag de acarreo y el anterior contenido de este se copia al bit 7.

### rl r8

Rota el contenido del registro r8 un bit a la inzquierda. El contenido del bit 7 se copia en el flag de acarreo y el anterior contenido de este se copia al bit 0.

### rl [hl]

Rota el contenido de la posición de memoria indicada por hl un bit a la izquierda. El contenido del bit 7 se copia en el flag de acarreo y el anterior contenido de este se copia al bit 0.

### rrc r8

Rota el contenido del registro r8 un bit a la derecha. El contenido del bit 0 se copia en el flag de acarreo y en el bit 7.

### rrc [hl]

Rota el contenido de la posición de memoria indicada por hl un bit a la derecha. El contenido del bit 0 se copia en el flag de acarreo y en el bit 7.

### rlc r8

Rota el contenido del registro r8 un bit a la izquierda. El contenido del bit 7 se copia en el flag de acarreo y en el bit 0.

### rlc [hl]

Rota el contenido de la posición de memoria indicada por hl un bit a la izquierda. El contenido del bit 7 se copia en el flag de acarreo y en el bit 0.

### sra r8

El contenido del registro r8 se desplaza a la derecha un bit. El contenido del bit 0 se copia en el flag de acarreo y el contenido del bit 7 se mantiene igual.

### sra [hl]

El contenido de la posición de memoria indicada por hl se desplaza a la derecha un bit. El contenido del bit 0 se copia en el flag de acarreo y el contenido del bit 7 se mantiene igual.

### sla r8

El contenido del registro r8 se desplaza a la izquierda un bit. El contenido del bit 7 se copia en el flag de acarreo y se guarda un cero en el bit 0.

### sla [hl]

El contenido de la posición de memoria indicada por hl se desplaza a la izquierda un bit. El contenido del bit 7 se copia en el flag de acarreo y se guarda un cero en el bit 0.

### srl r8

El contenido del registro r8 se desplaza a la derecha un bit. El contenido del bit 0 se copia en el flag de acarreo y se guarda un cero en el bit 7.

### srl [hl]

El contenido de la posición de memoria indicada por hl se desplaza a la derecha un bit. El contenido del bit 0 se copia en el flag de acarreo y se guarda un cero en el bit 7.

### swap r8

Intercambia el contenido de los dos nibbles del registro r8.

### swap [hl]

Intercambia el contenido de los dos nibbles de la posición de memoria indicada por hl.

## C.6 Instrucciones de bit

Estas instrucciones comprueban o modifican el valor de un único bit de un registro o una dirección de memoria.

### bit u3, r8

Activa el flag Z si el bit u3 del registro r8 es cero, y pone el flag a cero si no lo es.

### bit u3, [hl]

Activa el flag Z si el bit u3 de la posición de memoria indicada por hl es cero, y pone el flag a cero si no lo es.

### res u3, r8

Pone a 0 el bit u3 del registro r8.

### res u3, [hl]

Pone a 0 el bit u3 de la posición de memoria indicada por hl.

### set u3, r8

Pone a 1 el bit u3 del registro r8.

### set u3, [hl]

Pone a 1 el bit u3 de la posición de memoria indicada por hl.

## C.7 Instrucciones de salto

Estas instrucciones cambian el valor del contador de programa, cambiando el orden normal de ejecución.

### jp n16

Copia el dato de 16 bits n16 en PC.

### jp hl

Copia el valor de HL en PC. A veces se ve esta instrucción de la forma `jp [hl]`, lo cual puede llevar a confusión ya que los corchetes se suelen usar para denotar lectura de memoria.

### jp cc, n16

Copia el dato de 16 bits n16 en PC si se cumple la condición cc.

### jr e8

Suma el dato inmediato e8 a PC.

### jr cc, e8

Suma el dato inmediato e8 a PC si se cumple la condición cc.

### call n16

Se hace push a la pila de la dirección de memoria posterior a esta instrucción y se copia n16 a PC.

### call cc, n16

Si se cumple la condición cc se hace push a la pila de la dirección de memoria posterior a esta instrucción y se copia n16 a PC.

### ret

Se hace pop de la pila y se copian los dos bytes obtenidos en PC.

### ret cc

Si se cumple la condición cc se hace pop de la pila y se copian los dos bytes obtenidos en PC.

### reti

Se hace pop de la pila y se copian los dos bytes obtenidos en PC. También activa las interrupciones.

### rst vec

Se hace push a la pila de la dirección de memoria posterior a esta instrucción y se copia vec a PC. vec viene definido por el opcode y no es un dato en memoria. vec puede tomar los valores $00, $08, $10, $18, $20, $28, $30 y $38.

## C.8 Instrucciones de control de la CPU

### nop

Esta instrucción no hace nada, en el ciclo de ejecución que se tarda en leer la instrucción la CPU sólo aumenta PC en uno.

### halt

Detiene la CPU hasta que sucede una interrupción. Mientras la CPU está detenida consume mucha menos energía.

### stop

Detiene el sistema hasta que sucede una interrupción, ahorra más energía que HALT. El resultado producido por esta interrupción cambia en función de una serie de factores demasiado complicados para esta explicación y por si no fuera poco, usarla indebidamente puede dañar la consola permanentemente, por lo que no se usa en ningún juego licenciado de la Game Boy.

### di

Desactiva las interrupciones.

### ei

Activa las interrupciones.

# Bibliografía


[1] Richard P. Brent. «Note on Marsaglia's Xorshift Random Number Generators». En: *Journal of Statistical Software* 11.5 (2004), págs. 1-5. DOI: 10.18637/jss.v011.i05. URL: https://www.jstatsoft.org/index.php/jss/article/view/v011i05.

[2] *Gameboy CPU (SM83) instruction set*. https://gbdev.io/gb-opcodes/optables/ y https://web.archive.org/web/20231126002803/https://gbdev.io/gb-opcodes/optables/. Accedidas el 26/11/2023.

[3] *Pan Docs*. https://gbdev.io/pandocs/single.html y https://web.archive.org/web/20240125185937/https://gbdev.io/pandocs/single.html. Accedidas el 25/01/2024.

[4] *rgbasm(5) — language documentation*. https://rgbds.gbdev.io/docs/v0.6.1/rgbasm.5 y https://web.archive.org/web/20231229174652/https://rgbds.gbdev.io/docs/v0.6.1/rgbasm.5. Accedidas el 29/12/2023.